Data Synchronization & Replication

Mobile and distributed computing systems

Johan Kristiansson



Topics covered in this Lecture

Theory, Concepts, Trade-offs, and Challenges

- CAP theorem revisited
- Consistency models

Causality, Ordering, and Conflict-Free Replication

- Causality & Ordering: Happens-before relationship: *Which change came first?*
- Logical Clocks, Lamport Clocks, Vector Clocks
- Conflict-Free Replicated Data types (CRDT)
- Mathematical foundations

Collaborative Editing *(Lecture 2)*

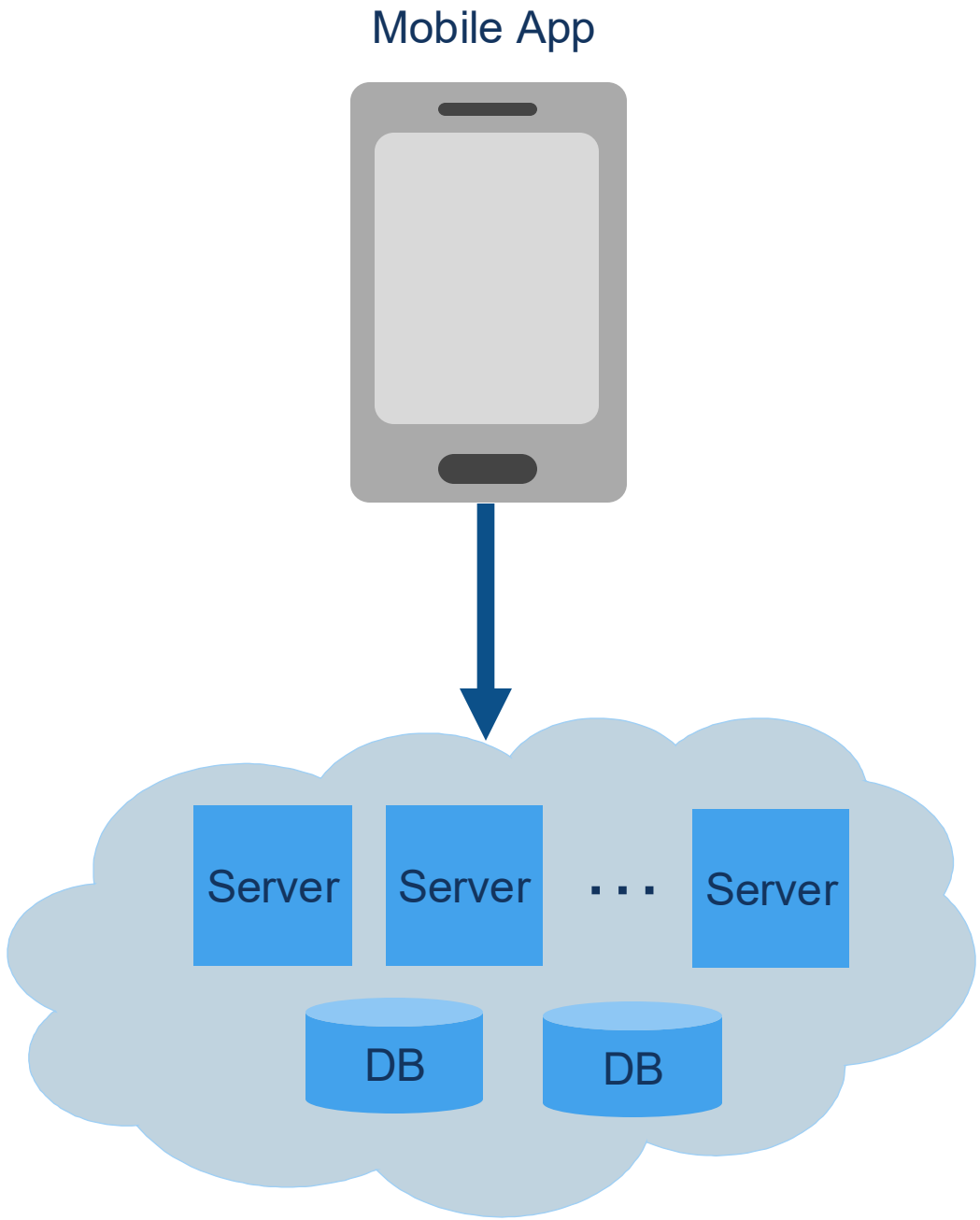
- Operational Transformation (OT)
- Sequential CRDTs

Mobile Apps depending on Internet connectivity

Daily Frustrations

- Can't access your notes or documents when subway commute
- Form data disappears after typing for 20 minutes
- Spinning wheels - ***Waiting... waiting... Timeout!***
- Something went wrong: Work lost, start over …
- International travel: expensive roaming or no data plan
- Rural areas - Spotty connection = unusable apps
- Wasted time re-doing lost work

Some Mobile Apps useless without Internet access!



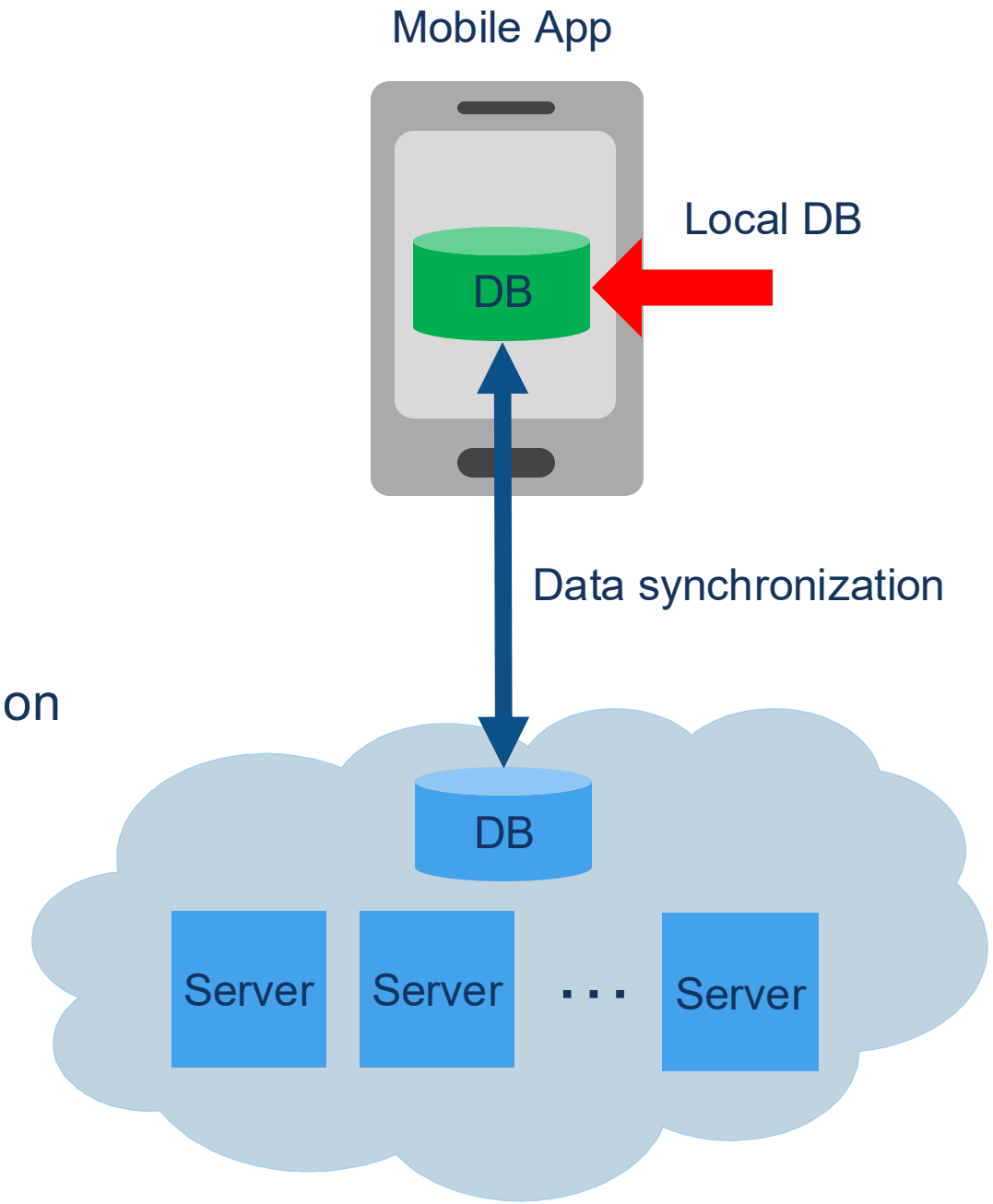
Offline/Mobile-First Apps

Core Architecture Principles

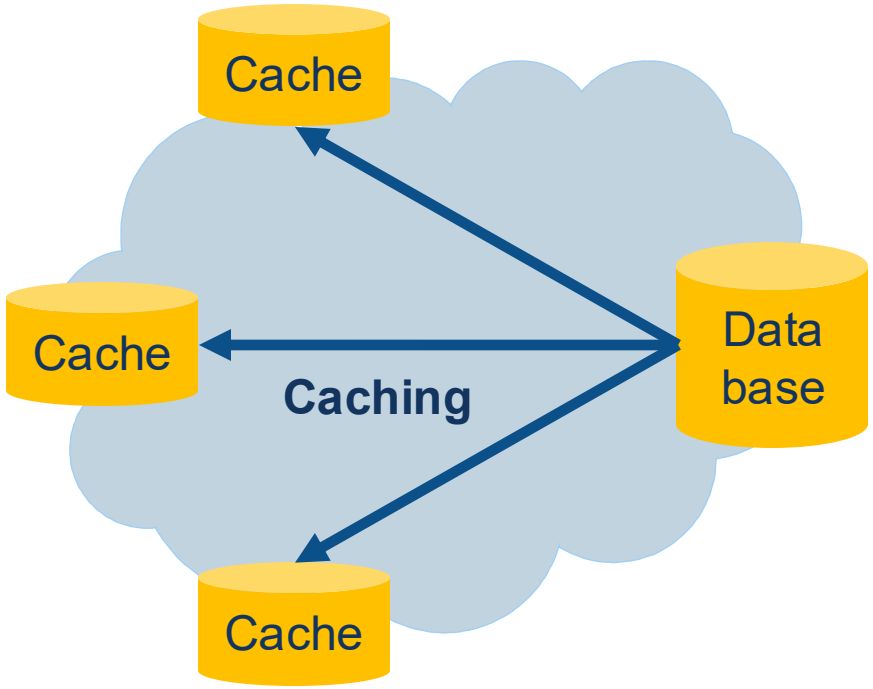
- Store everything on device/edge serve first
- Cache everything: data, and user actions
- Optimistic updates: UI responds instantly, sync happens later
- Sync when possible: Background synchronization when connected
- Conflict resolution

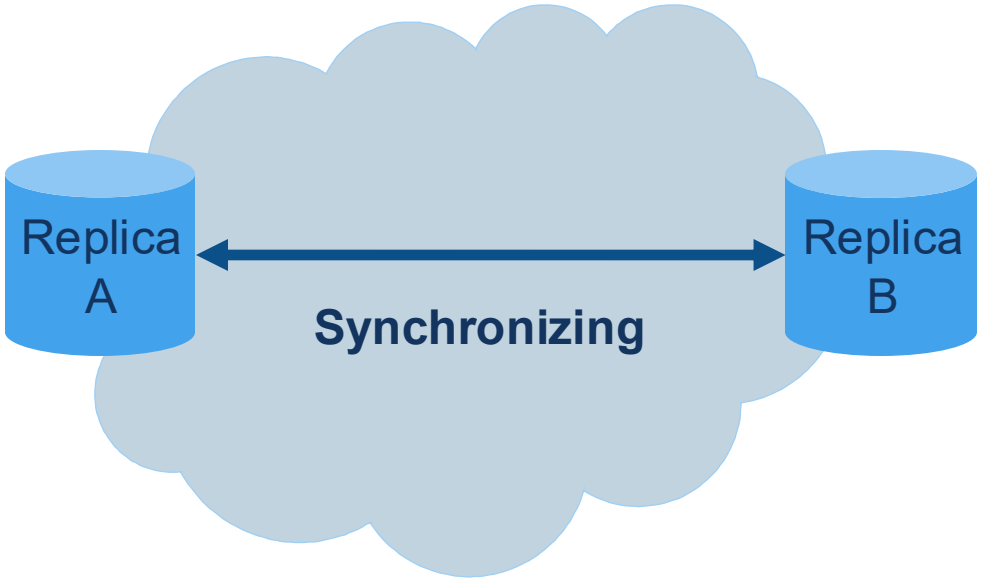
Example of applications

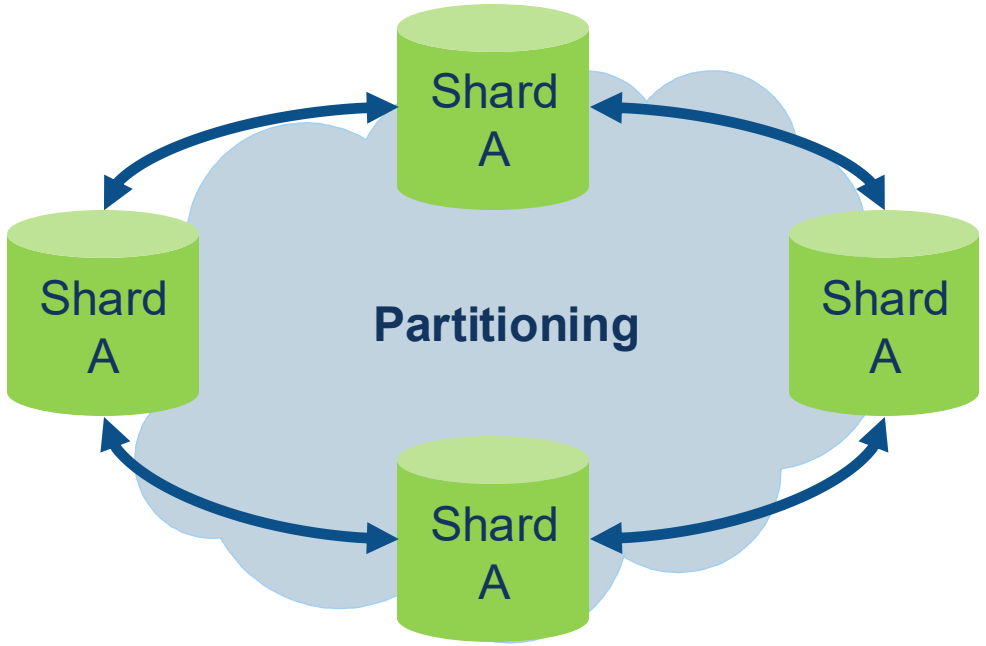
- Apple notes apps, Spotify, Google Maps, MS Office, Slack, Gmail etc.. etc …



Replication vs Caching vs Sharding







Data Sharding vs Data Replication

Data Sharding

- **Problem:** Too much data for one node
- **Solution:** Split data across nodes
- **Purpose:** Scalability

Data: [A B C D E F G H I J K L]

Node1: [A B C D]

Node2: [E F G H]

Node3: [I J K L]

Data Replication

- **Problem**: Node might fail
- **Solution:** Copy data to multiple nodes
- **Purpose:** High Availability & Fault Tolerance

Data: [A B C D E F G H I J K L]

Node1: [A B C D E F G H I J K L]

Node2: [A B C D E F G H I J K L]

Node3: [A B C D E F G H I J K L]

CAP-Theorem (revisited)

Consistency

- All nodes see the same data simultaneously

Availability

- System remains operational

Partition Tolerance

- System continues despite network failures

In the presence of a network partition, a distributed system must **choose** between ***Consistency*** and ***Availability***

Why?

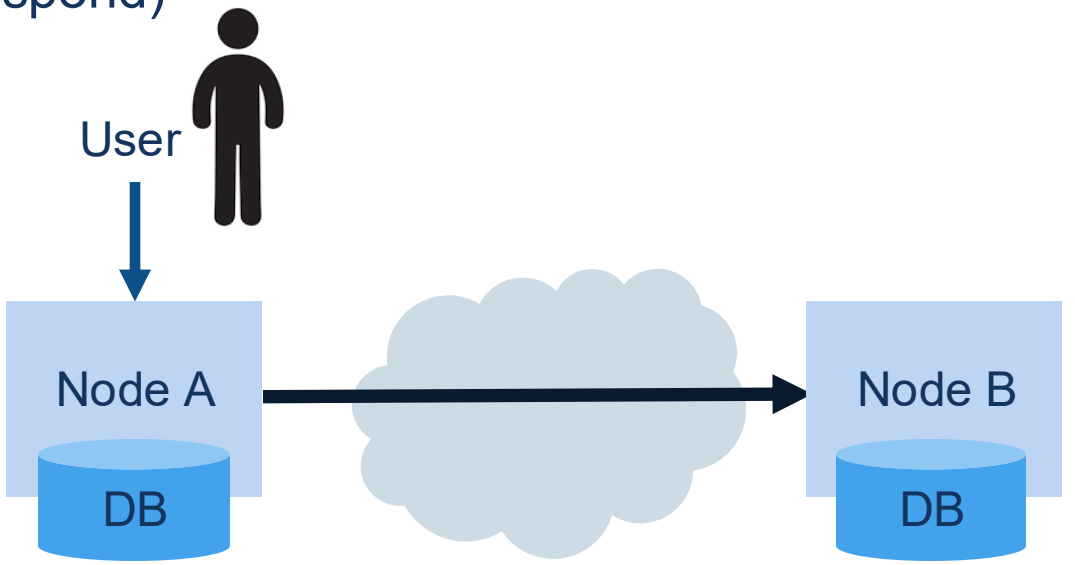
Time: Can’t set a global state instantaneously

Speed of light: Can't instantly sync distant nodes

Computer networks: Messages can be lost/delayed

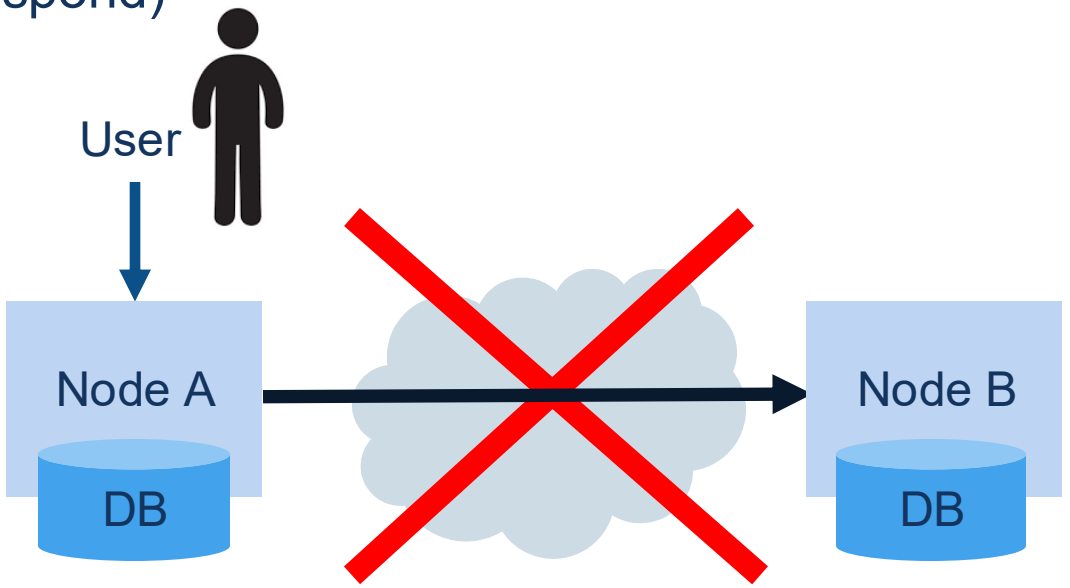
CAP Proof (by Contradiction)

- Assume you have all three (C + A + P):
- Network partitions (P must be handled)
- User requests write on Node A (A says must respond)



CAP Proof (by Contradiction)

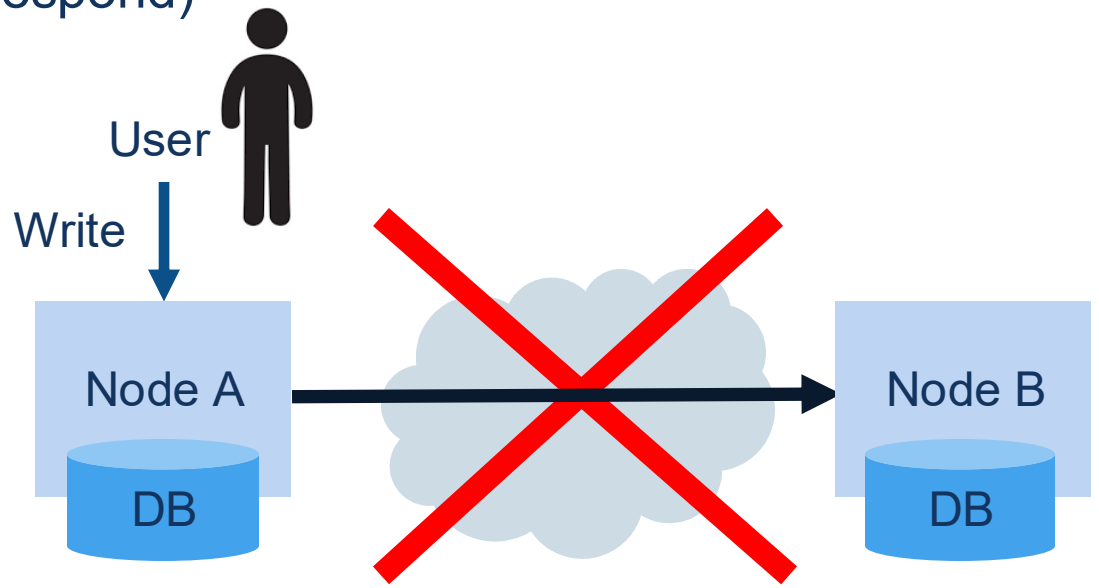
- Assume you have all three (C + A + P):
- Network partitions (P must be handled)
- User requests write on Node A (A says must respond)
- Node A can't reach Node B (partition!)



CAP Proof (by Contradiction)

- Assume you have all three (C + A + P):
- Network partitions (P must be handled)
- User requests write on Node A (A says must respond)
- Node A can't reach Node B (partition!)
- Two choices:
  - **Accept write:** Violates C (inconsistent with Node B)

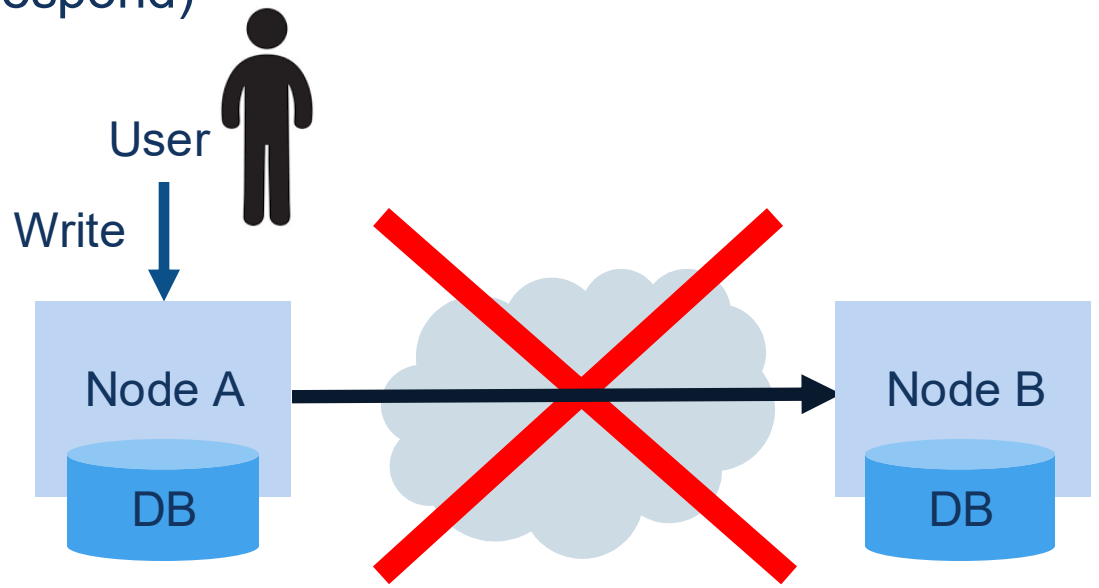
    Reject write: Violates A (unavailable)



CAP Proof (by Contradiction)

- Assume you have all three (C + A + P):
- Network partitions (P must be handled)
- User requests write on Node A (A says must respond)
- Node A can't reach Node B (partition!)
- Two choices:
  - **Accept write:** Violates C (inconsistent with Node B)

    Reject write: Violates A (unavailable)



Contradiction! Can't satisfy both choices

Trade-offs

CP Systems (Consistency + Partition Tolerance)

- **Trade-off:** Wait for consistency, system might be temporarily unavailable
- Correctness is important than availability (e.g. banking)
- **Example:** MongoDB, Etcd, Zookeeper

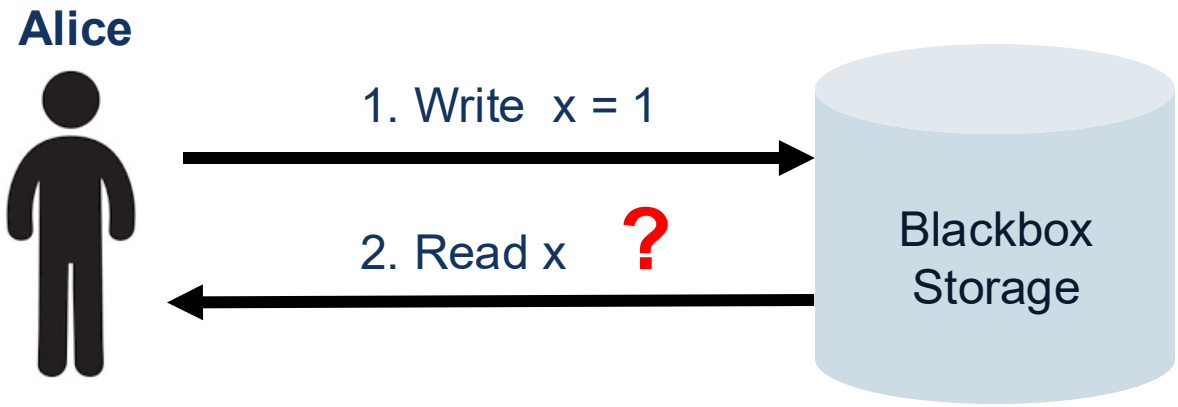
AP Systems (Availability + Partition Tolerance)

- **Trade-off:** Always available, system might be inconsistent
- Availability is more important than perfect consistency
- Example: Offline-first apps, Kademlia

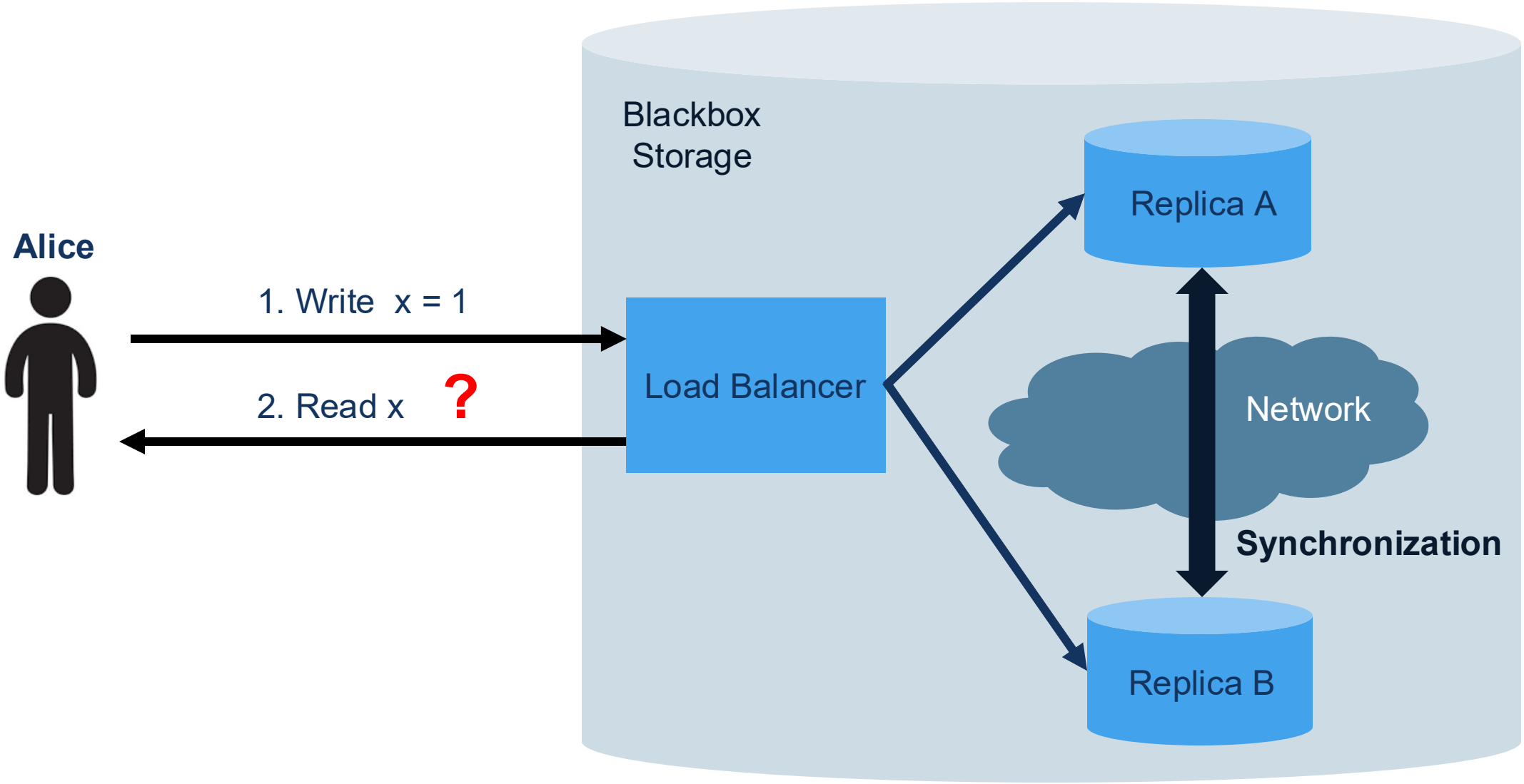
CA Systems (Consistency + Availability)

- **Trade-off:** The only way to prevent network partitions is to use single-node systems (not distributed)
- **Example:** Single PostgreSQL instance

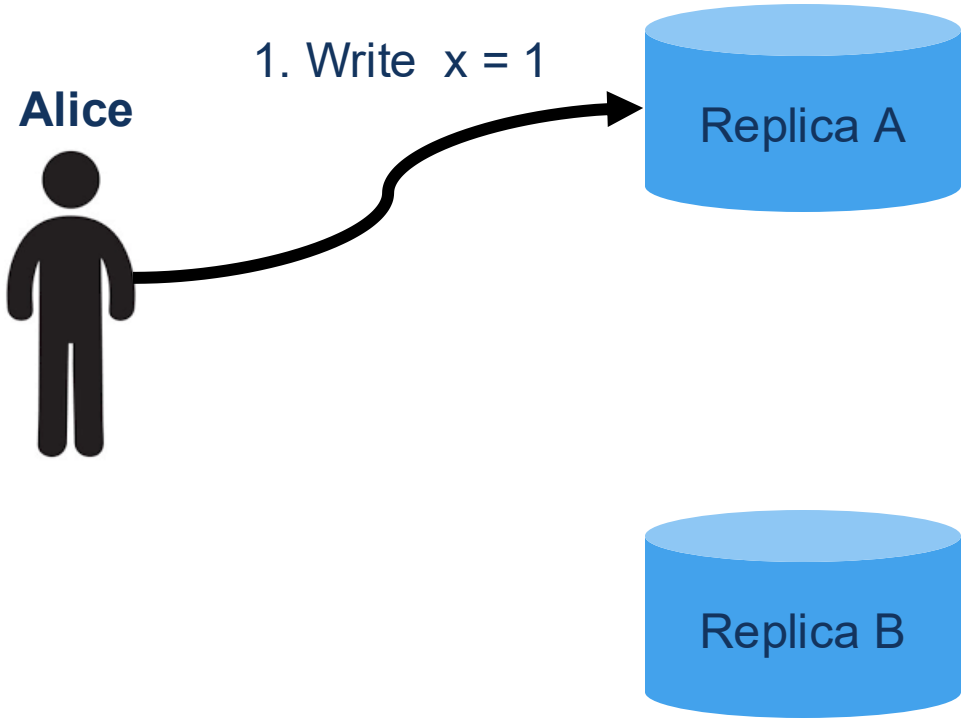
What will be return by the read?



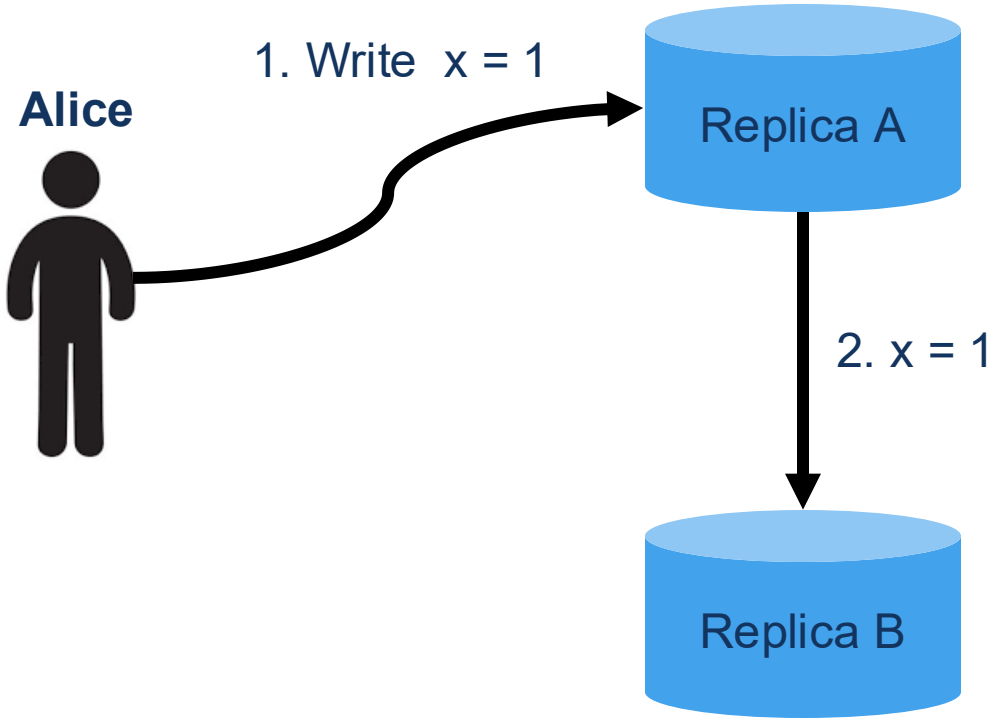
Blackbox is a Distributed System



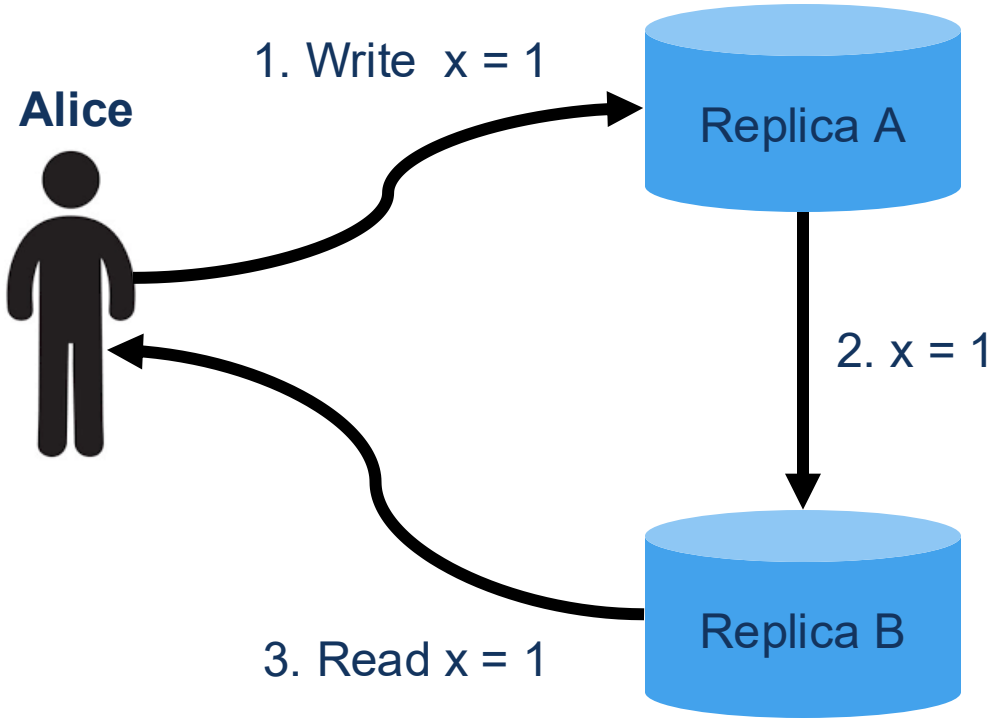
What will be return by the read?



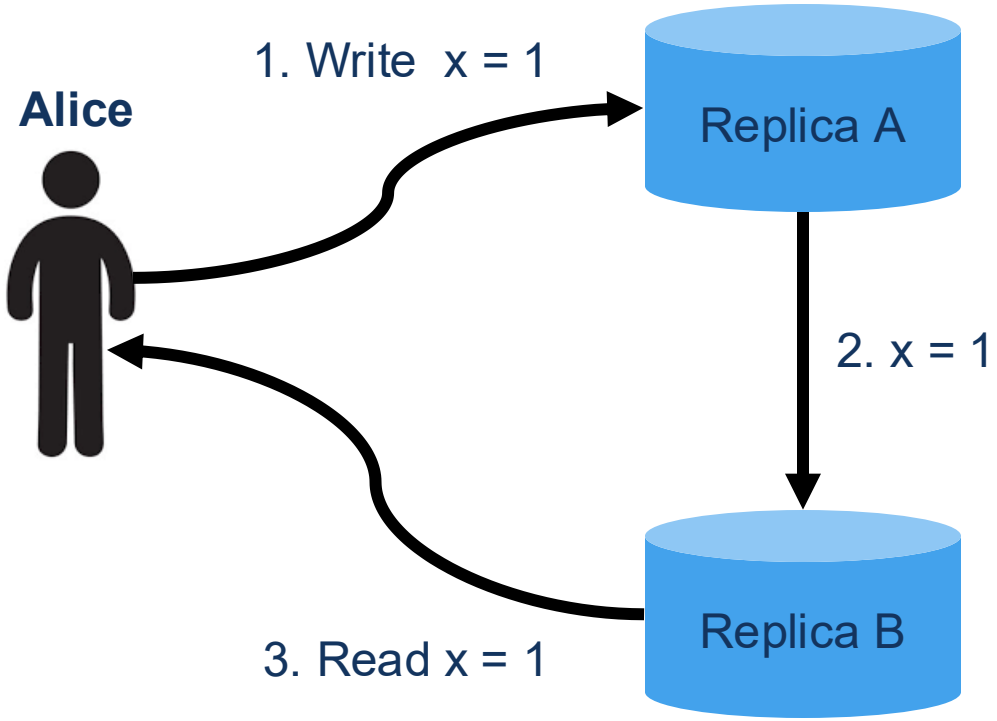
What will be return by the read?

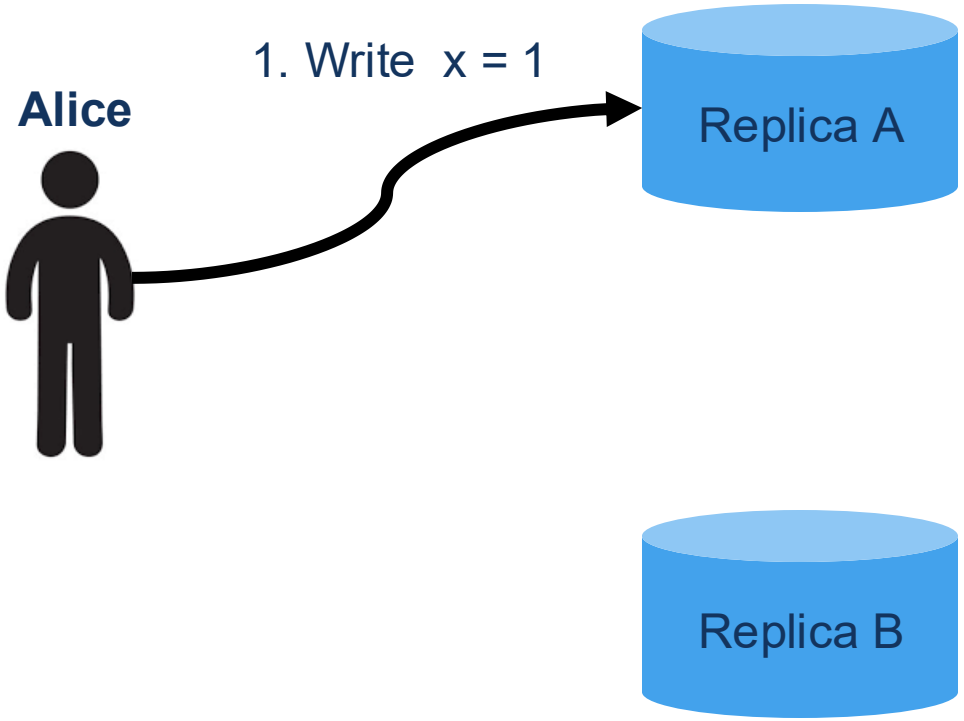


What will be return by the read?

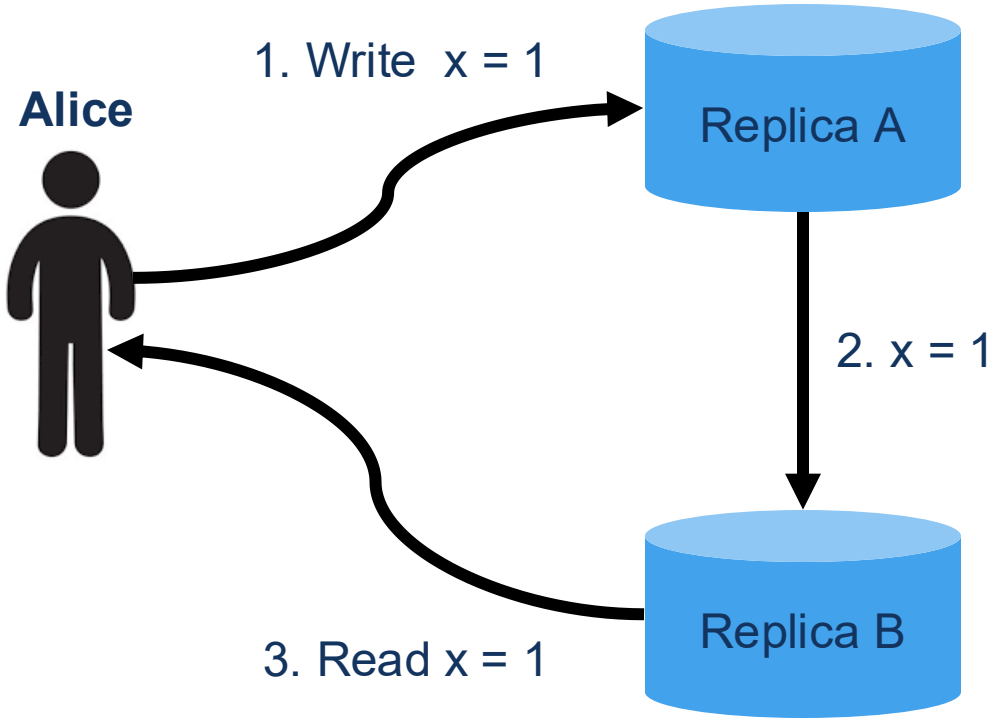


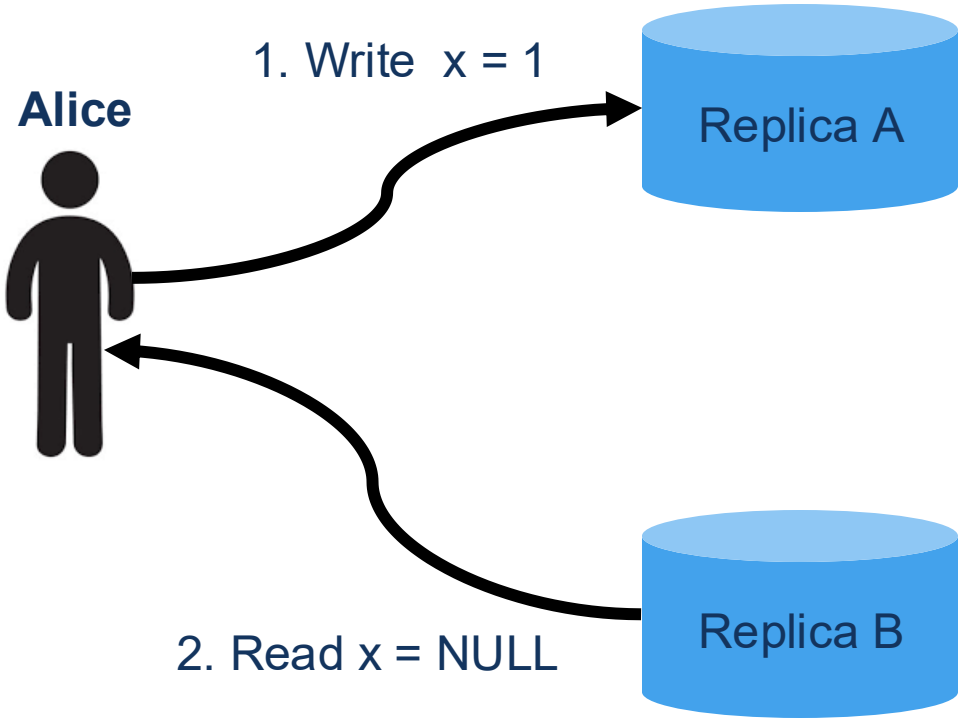
What will be return by the read?



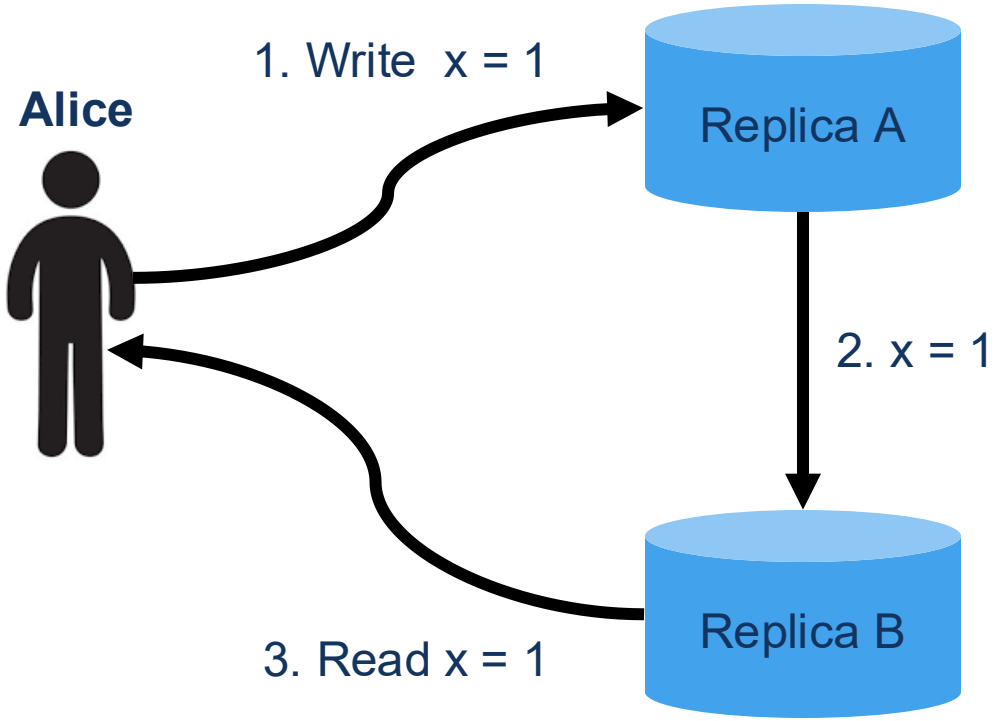


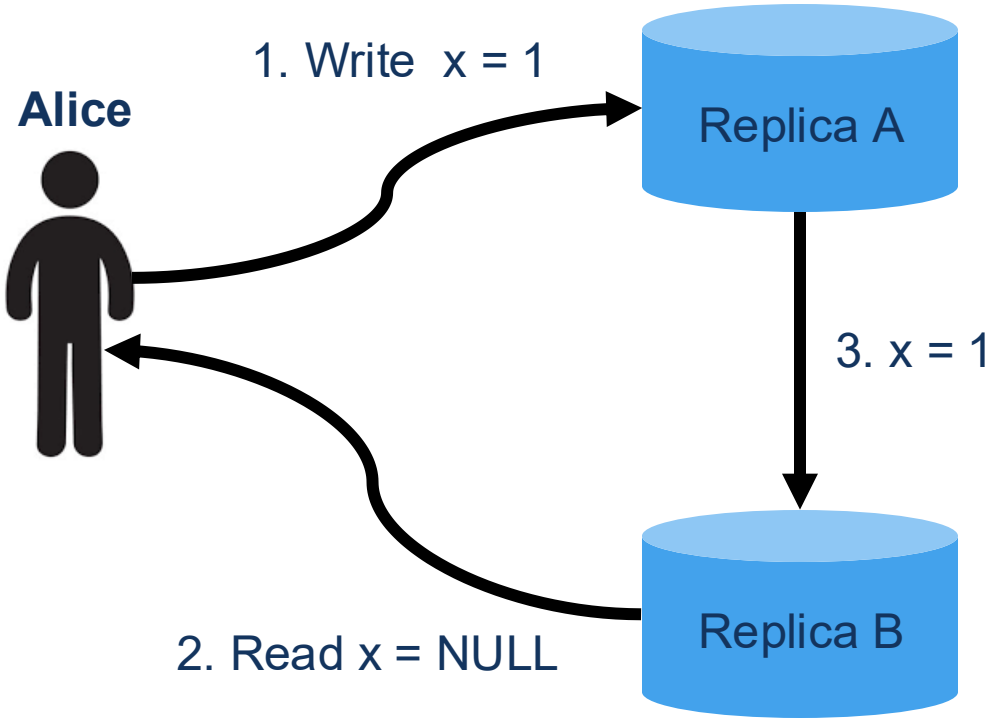
What will be return by the read?



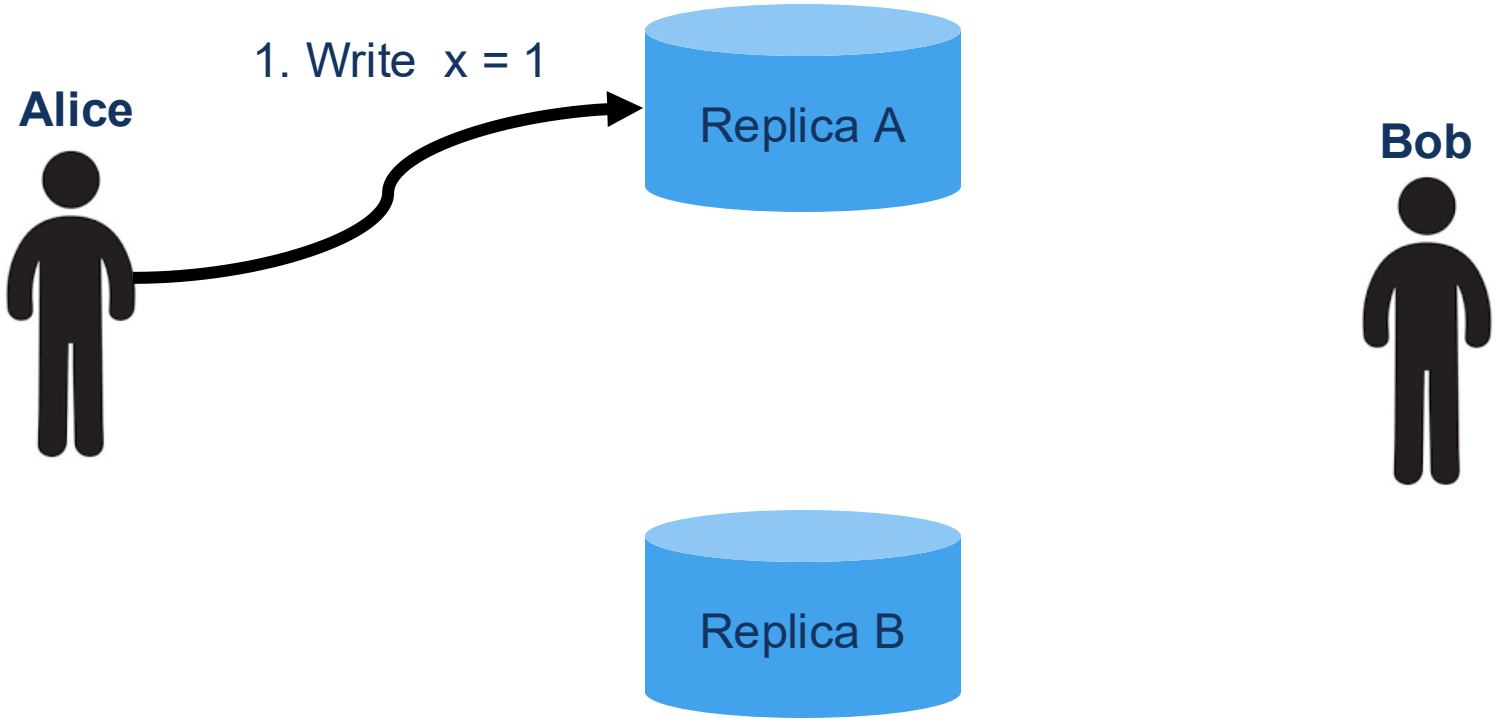


What will be return by the read?

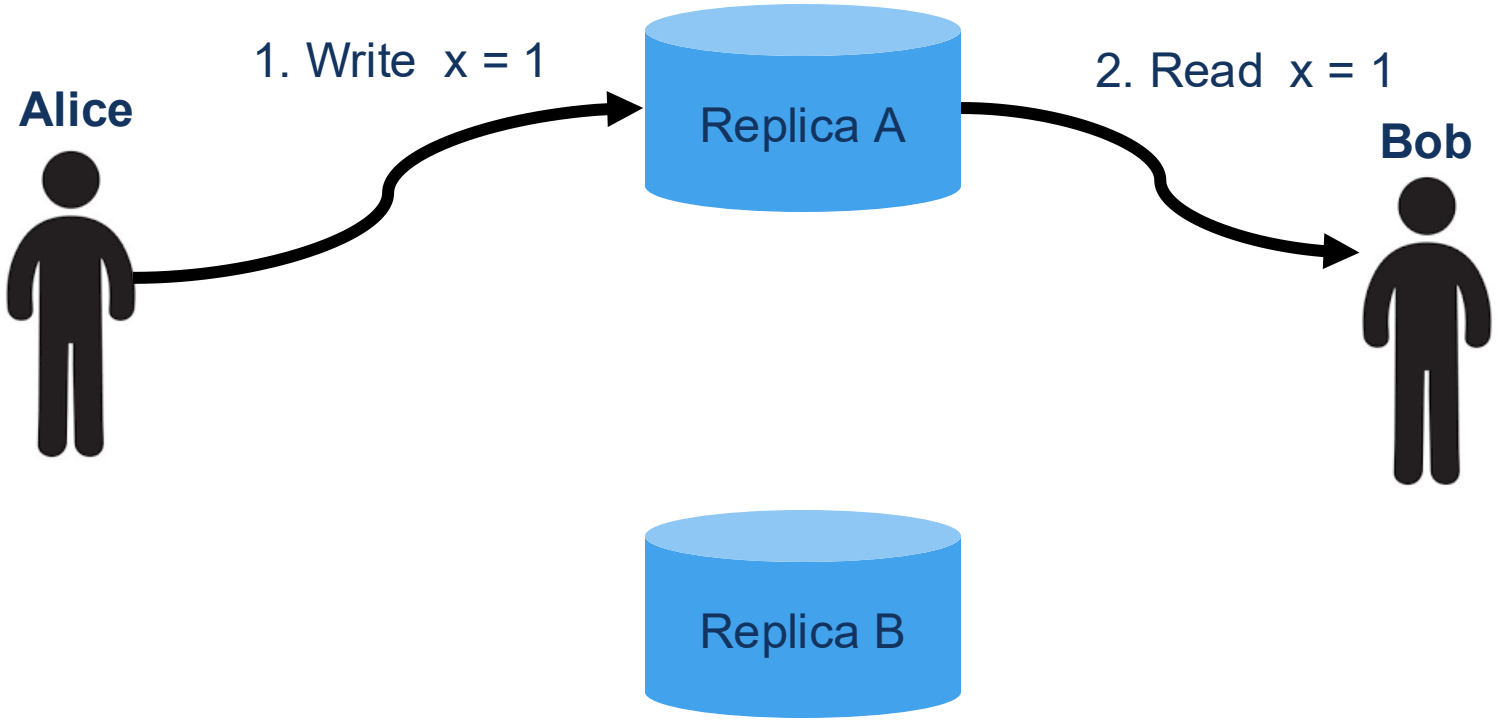




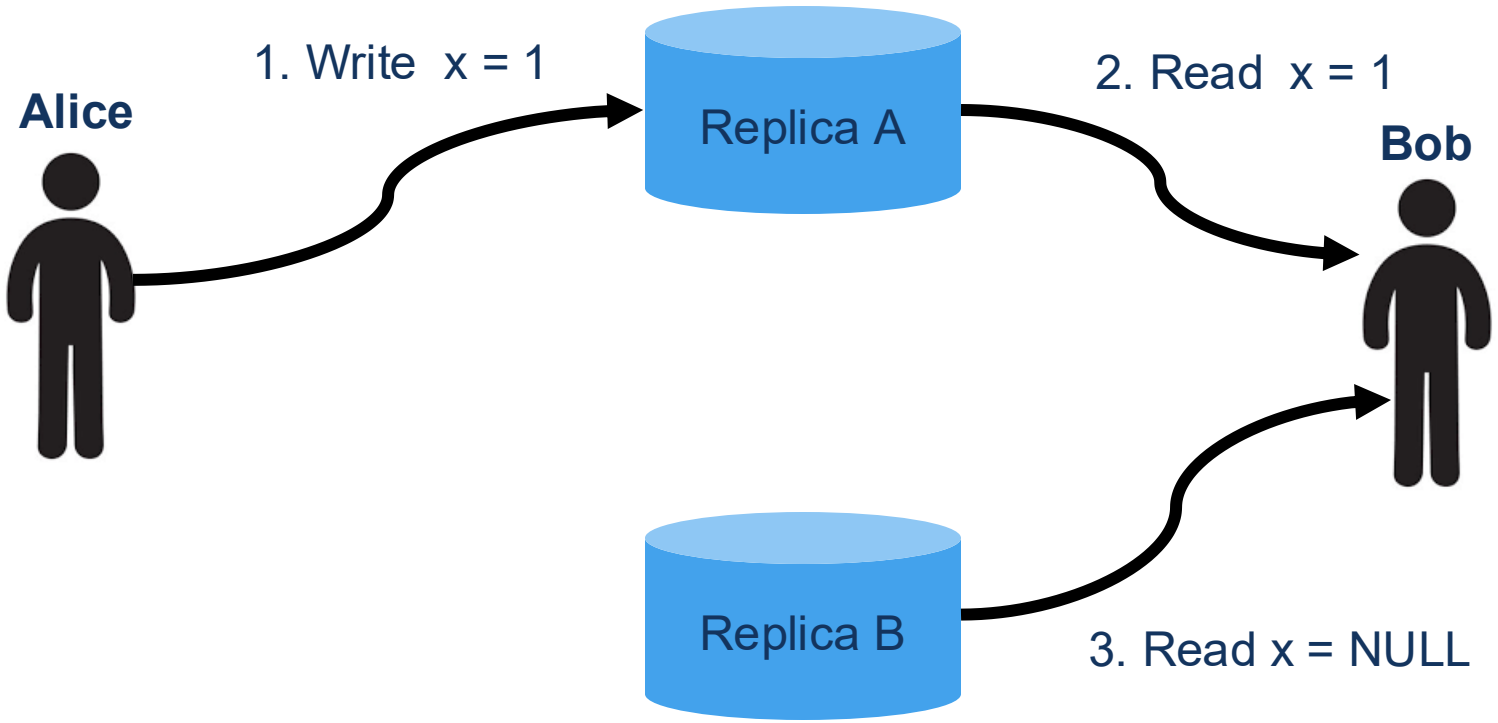
Non-Monotonic Reads



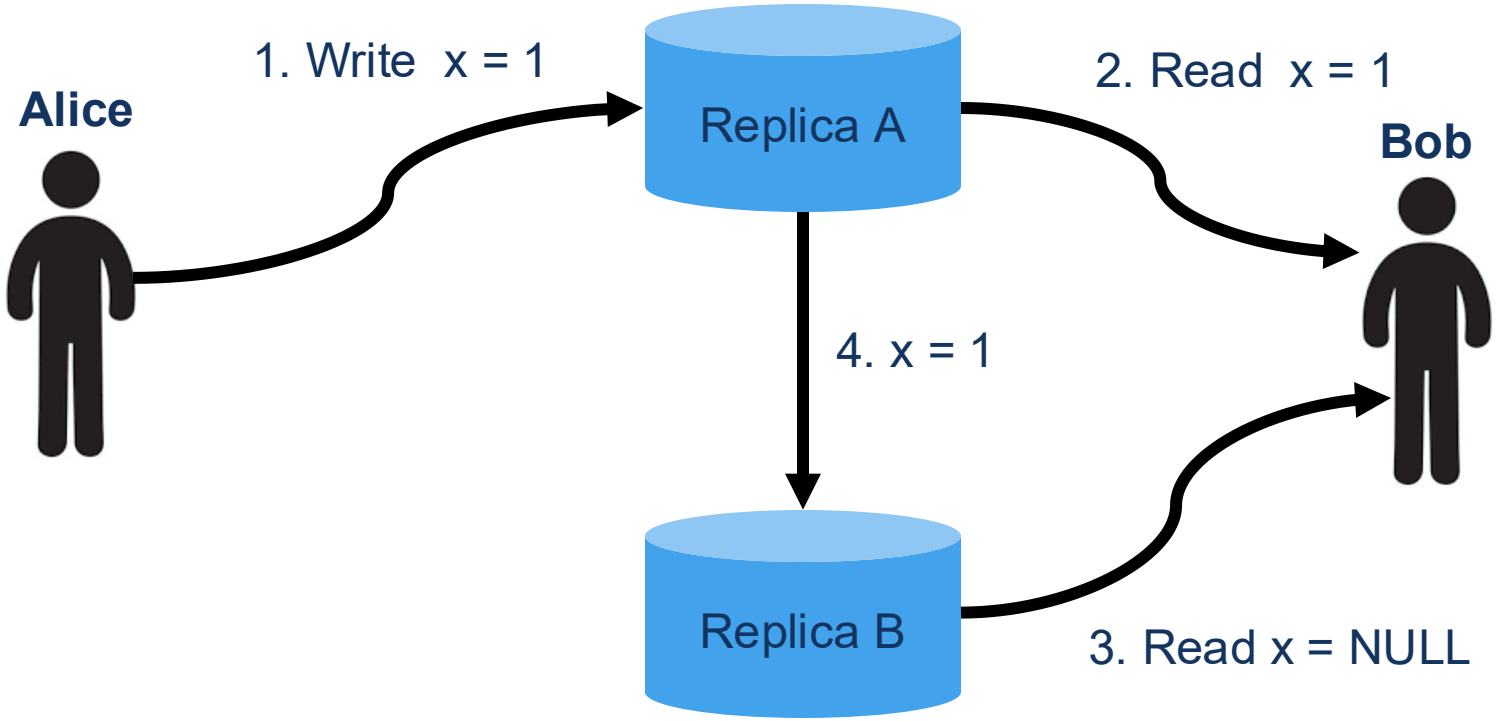
Non-Monotonic Reads



Non-Monotonic Reads

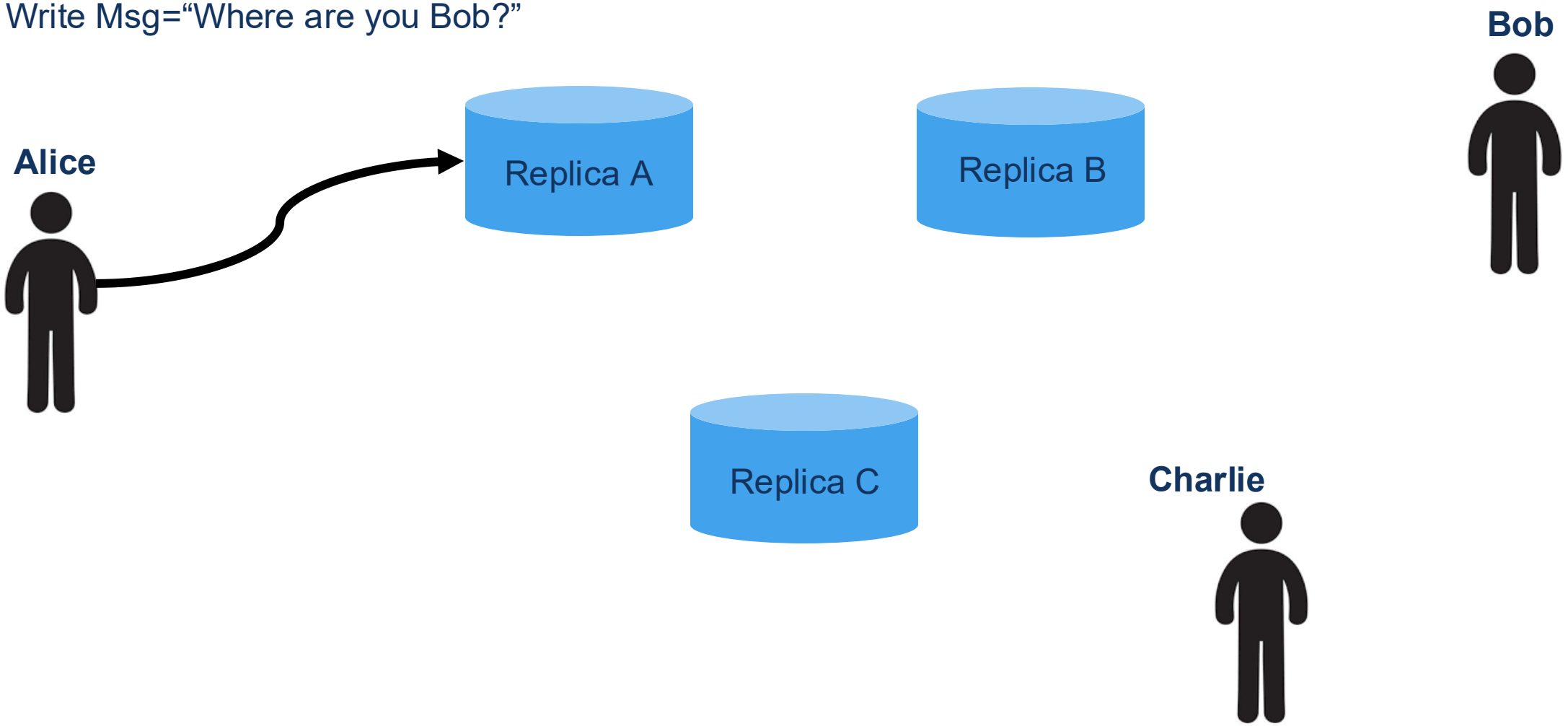


Non-Monotonic Reads



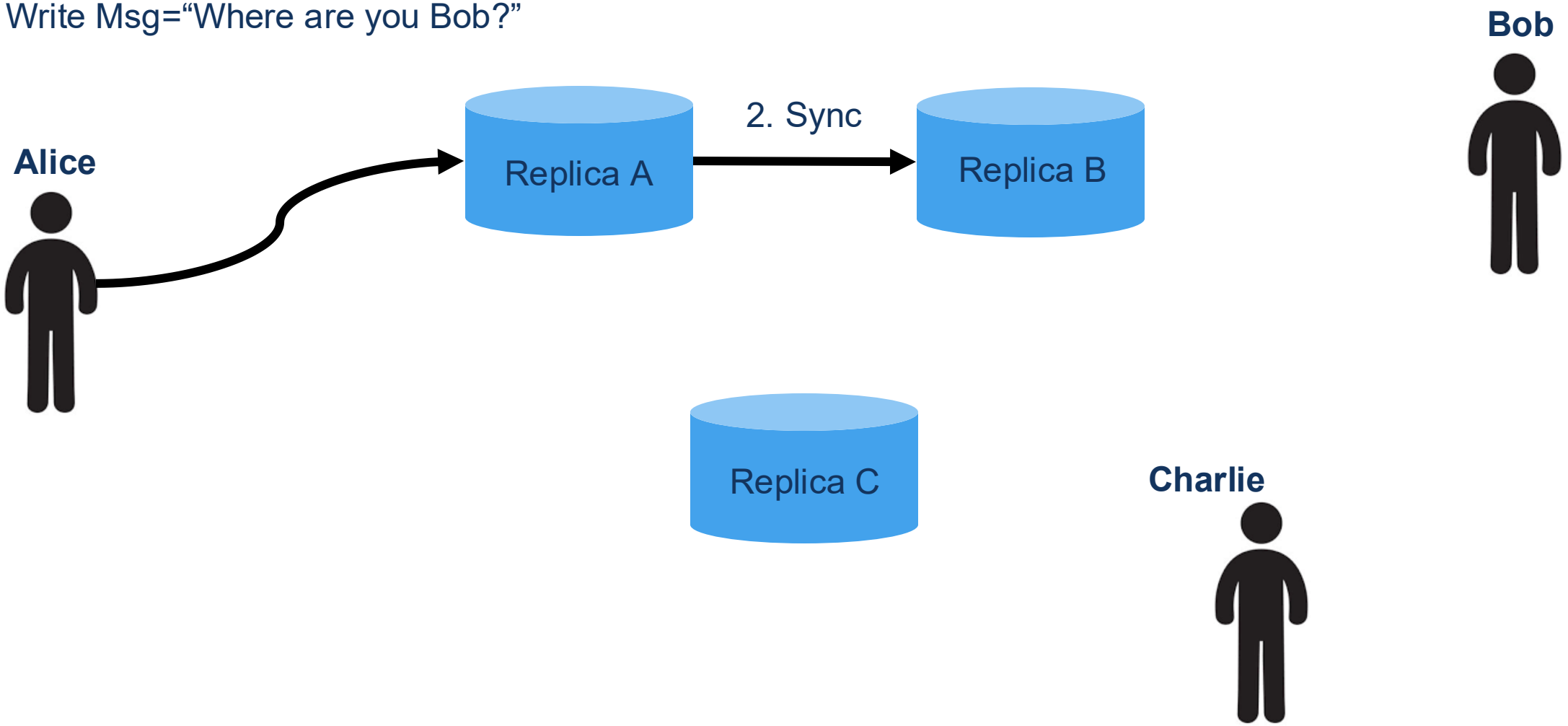
Violation of Casuality

1. Write Msg="Where are you Bob?"



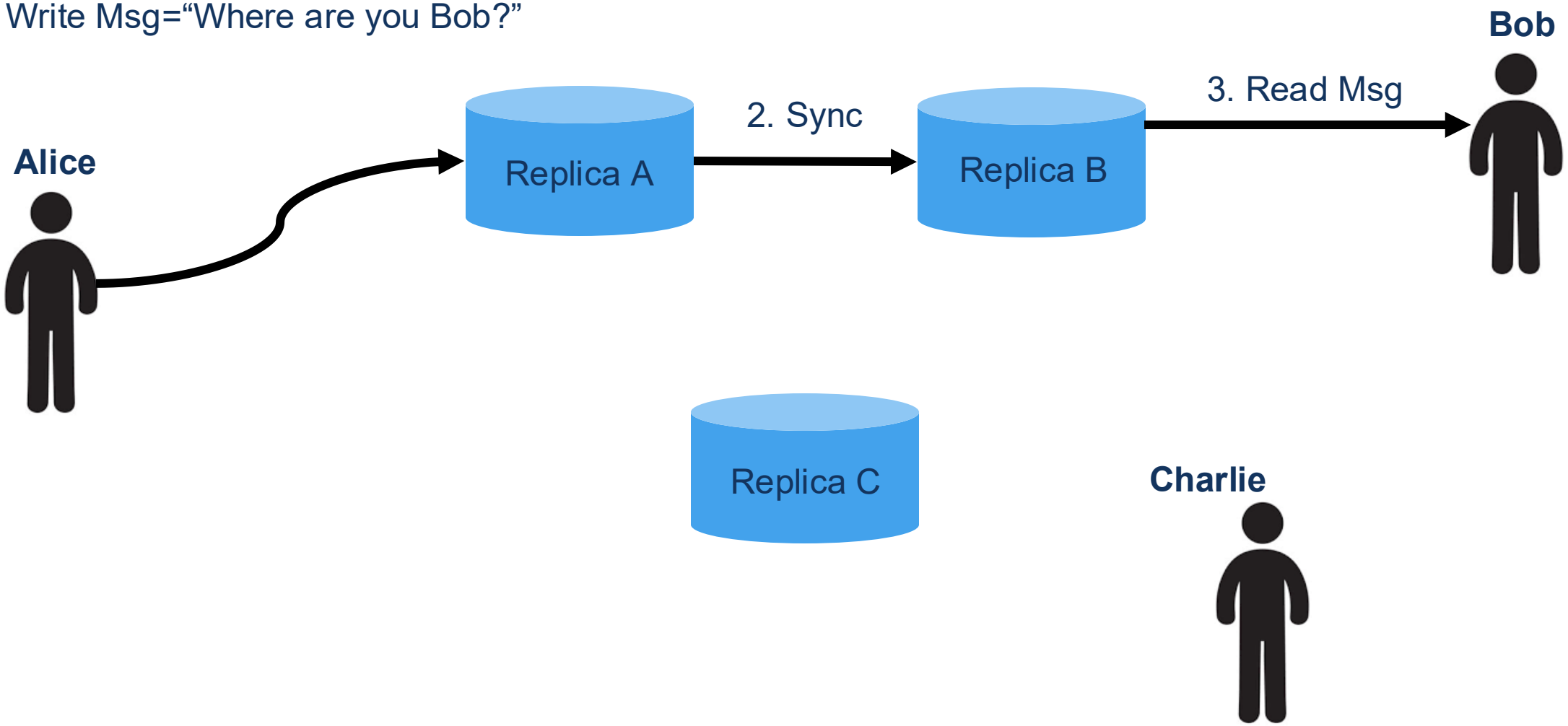
Violation of Casuality

1. Write Msg="Where are you Bob?"



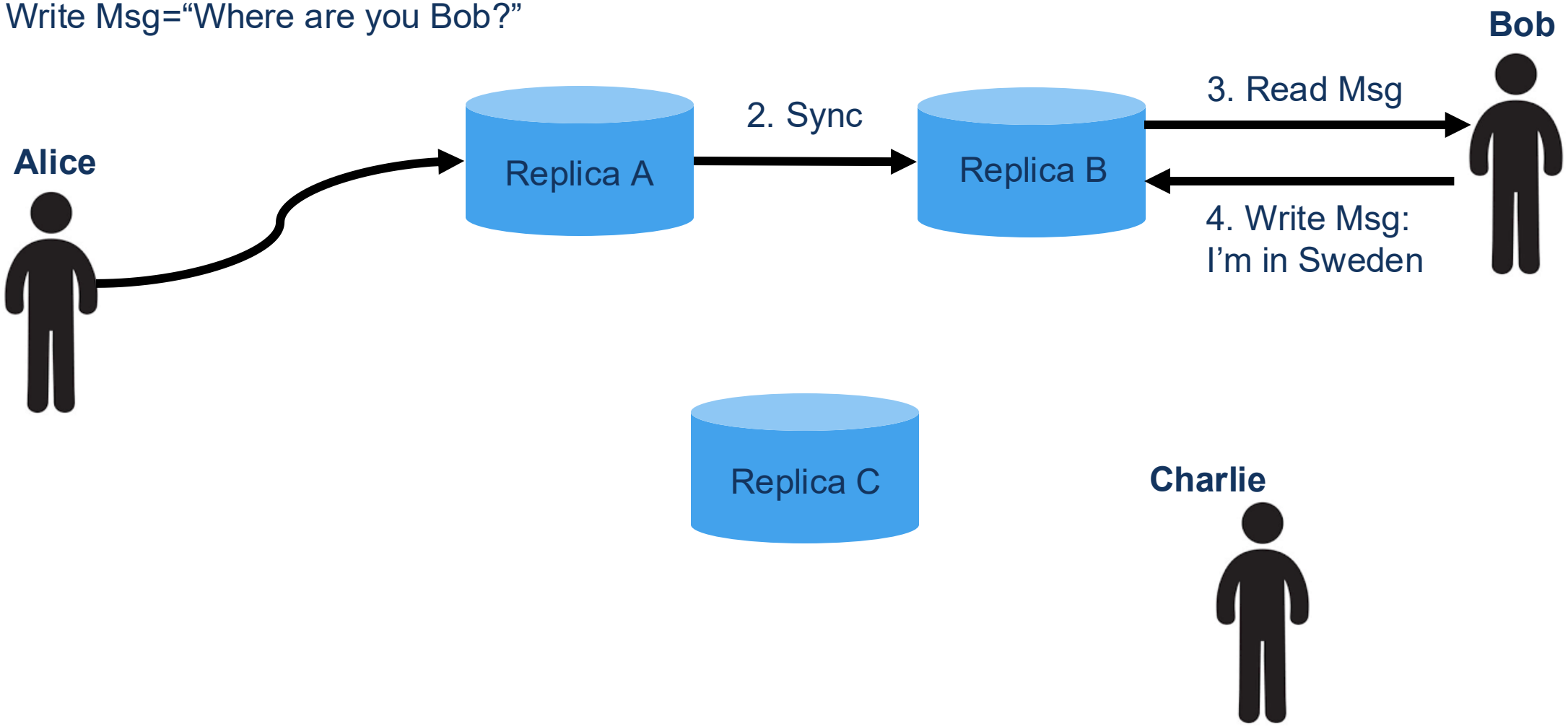
Violation of Casuality

1. Write Msg="Where are you Bob?"

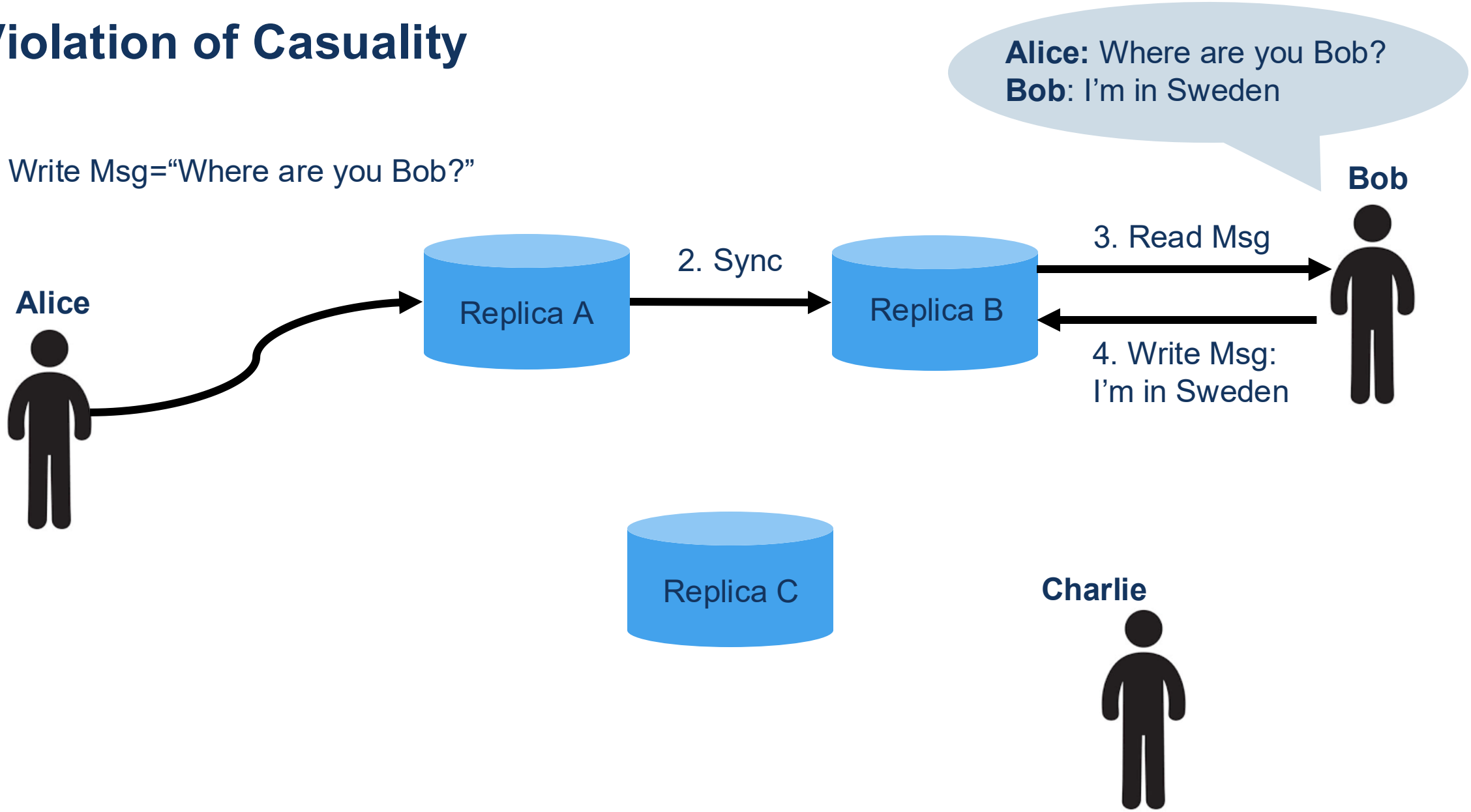


Violation of Casuality

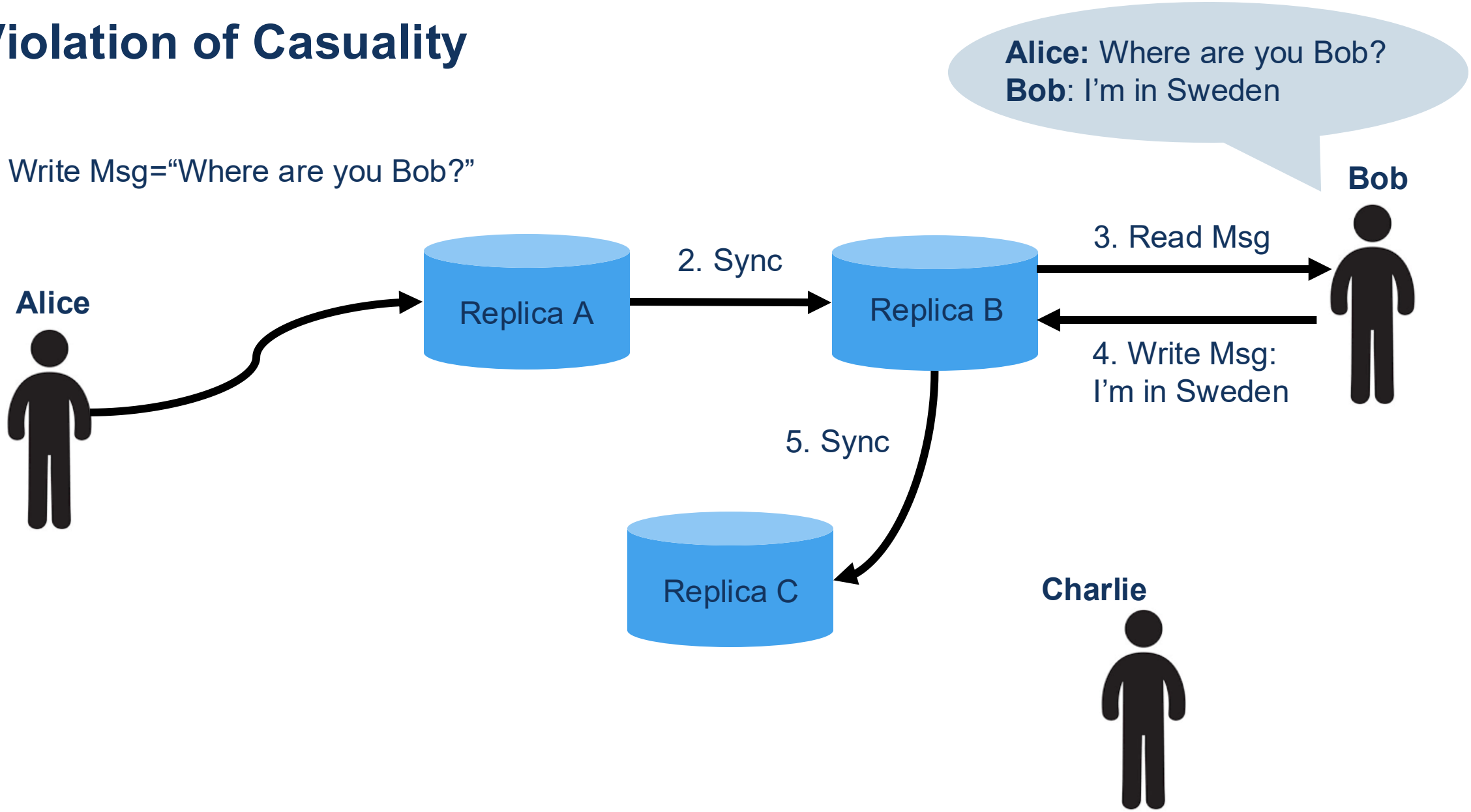
1. Write Msg="Where are you Bob?"



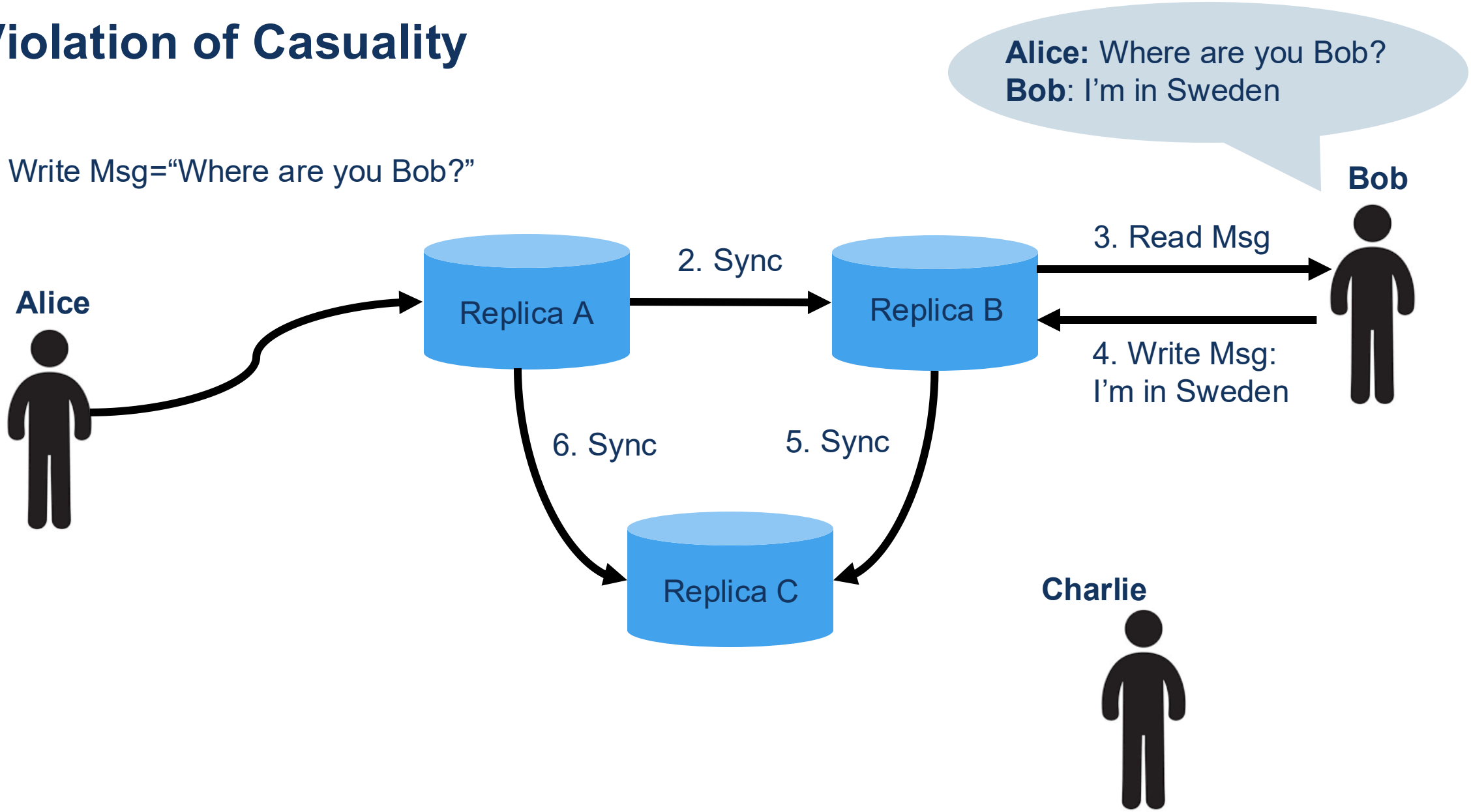
Violation of Casuality



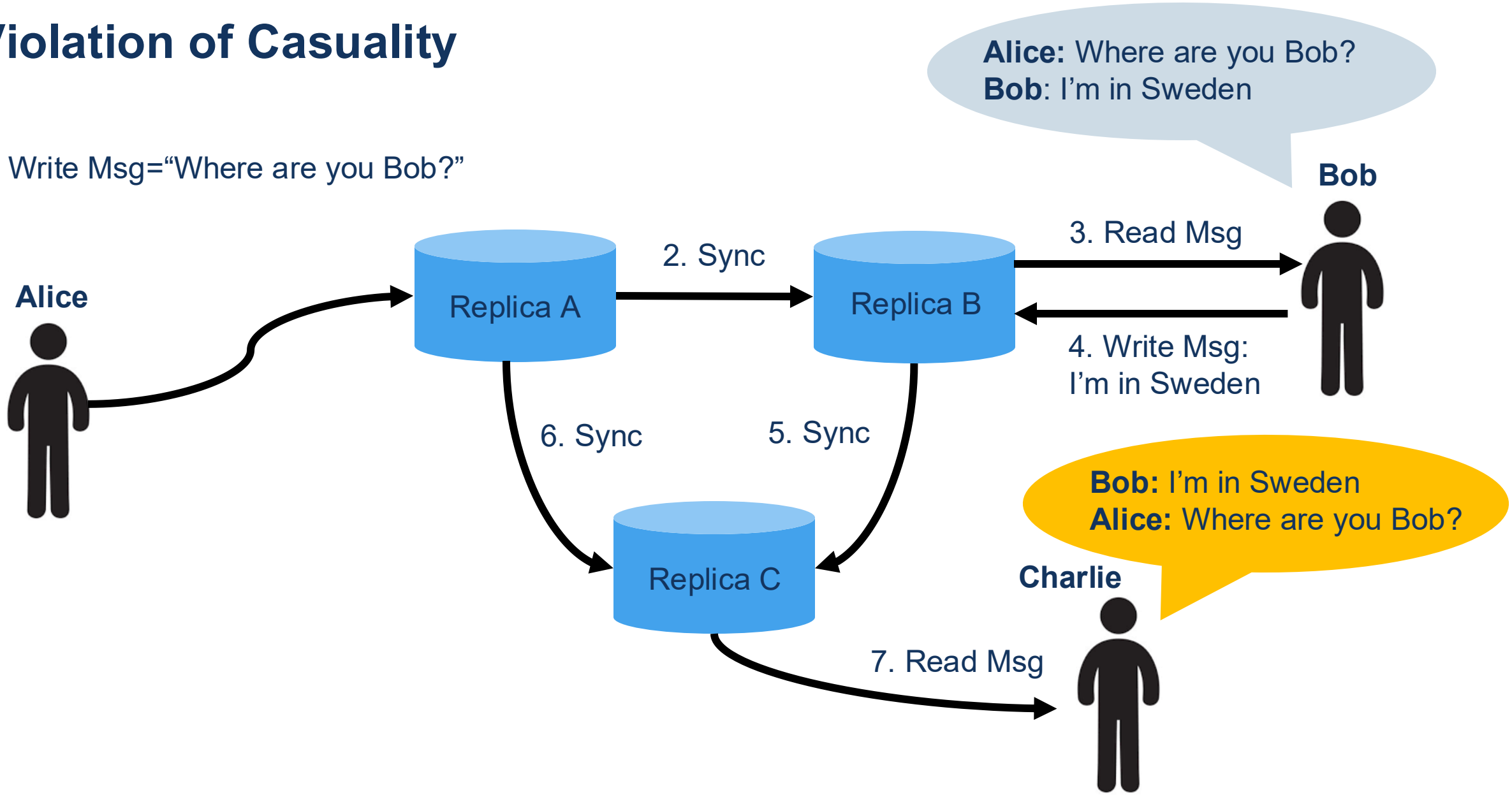
Violation of Casuality



Violation of Casuality

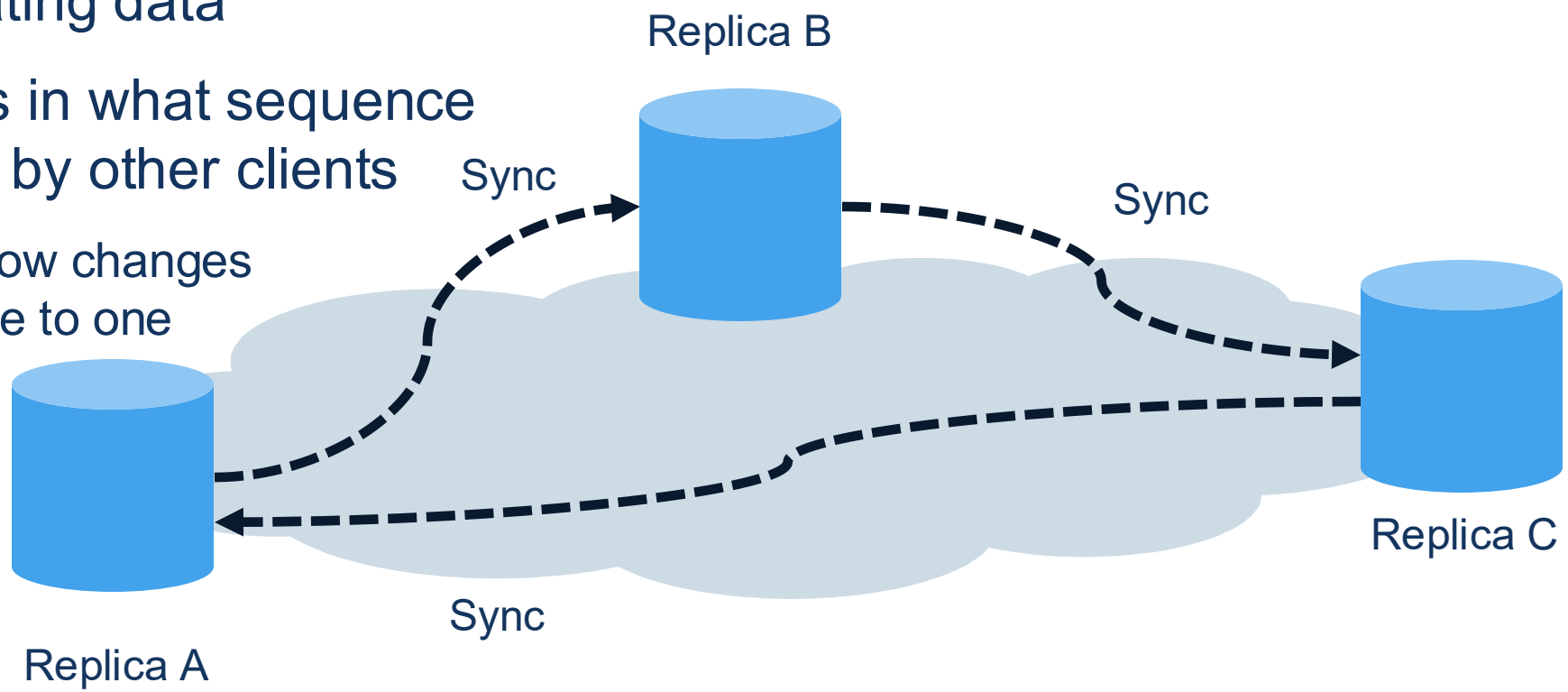


Violation of Casuality

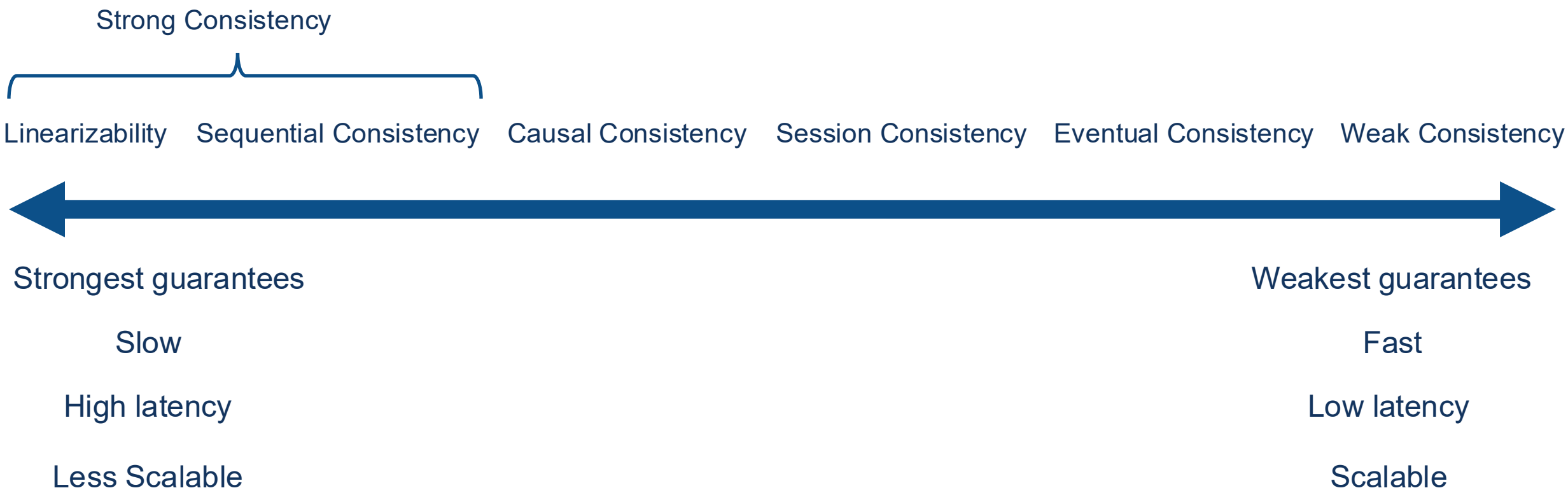


Consistency Models

- Due to network latency, partitions, and processing time, users may see updates at different time
- A consistency model dictates what users *should* expect when reading/writing/updating data
- Ordering determines in what sequence operations are seen by other clients
  - It's about when and how changes become visible relative to one another

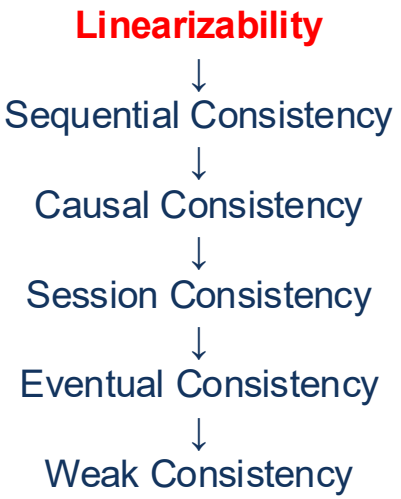


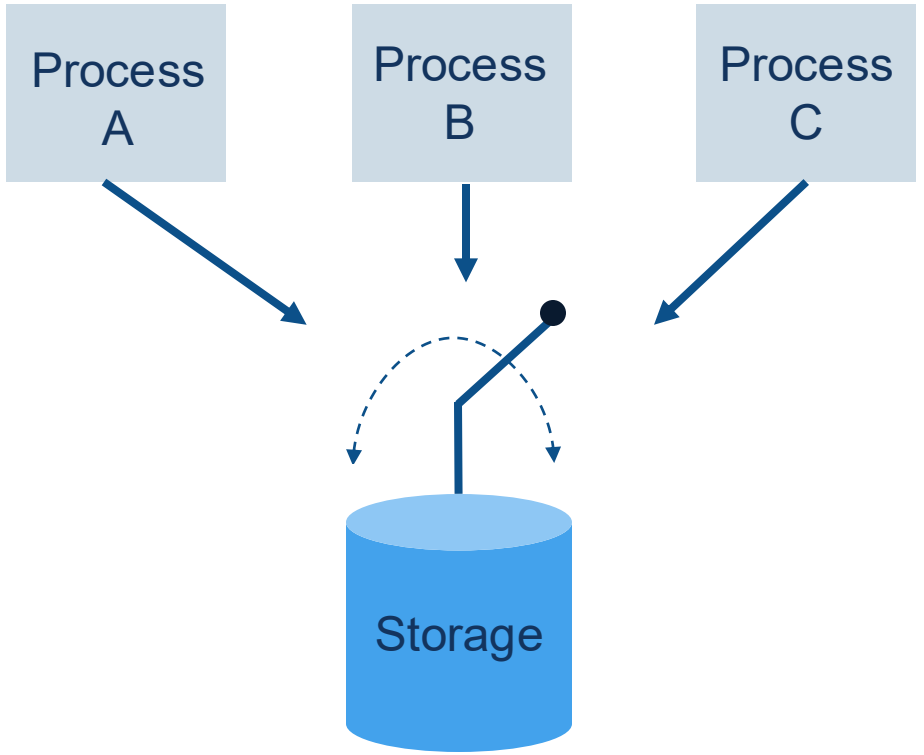
Consistency Models



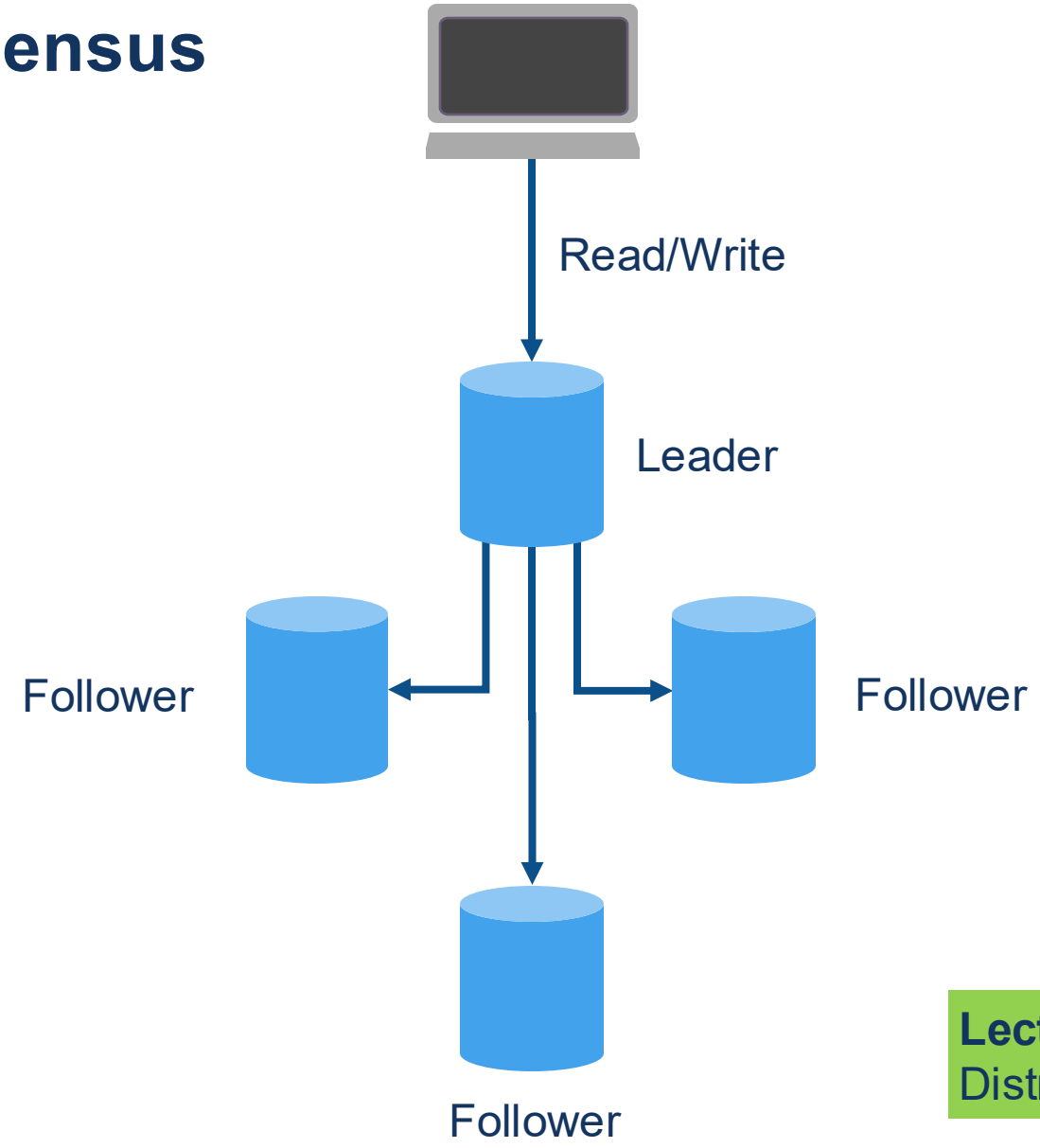
Linearizability

- All operations appear to execute ***atomically*** in ***real-time order***
- The system behaves as if it were not distributed at all, all operations behave as if executed on a single copy of the data, even though data is replicated across multiple nodes
- Total order respecting real-time order
- Completed and acknowledge operations are instantly visible to all clients





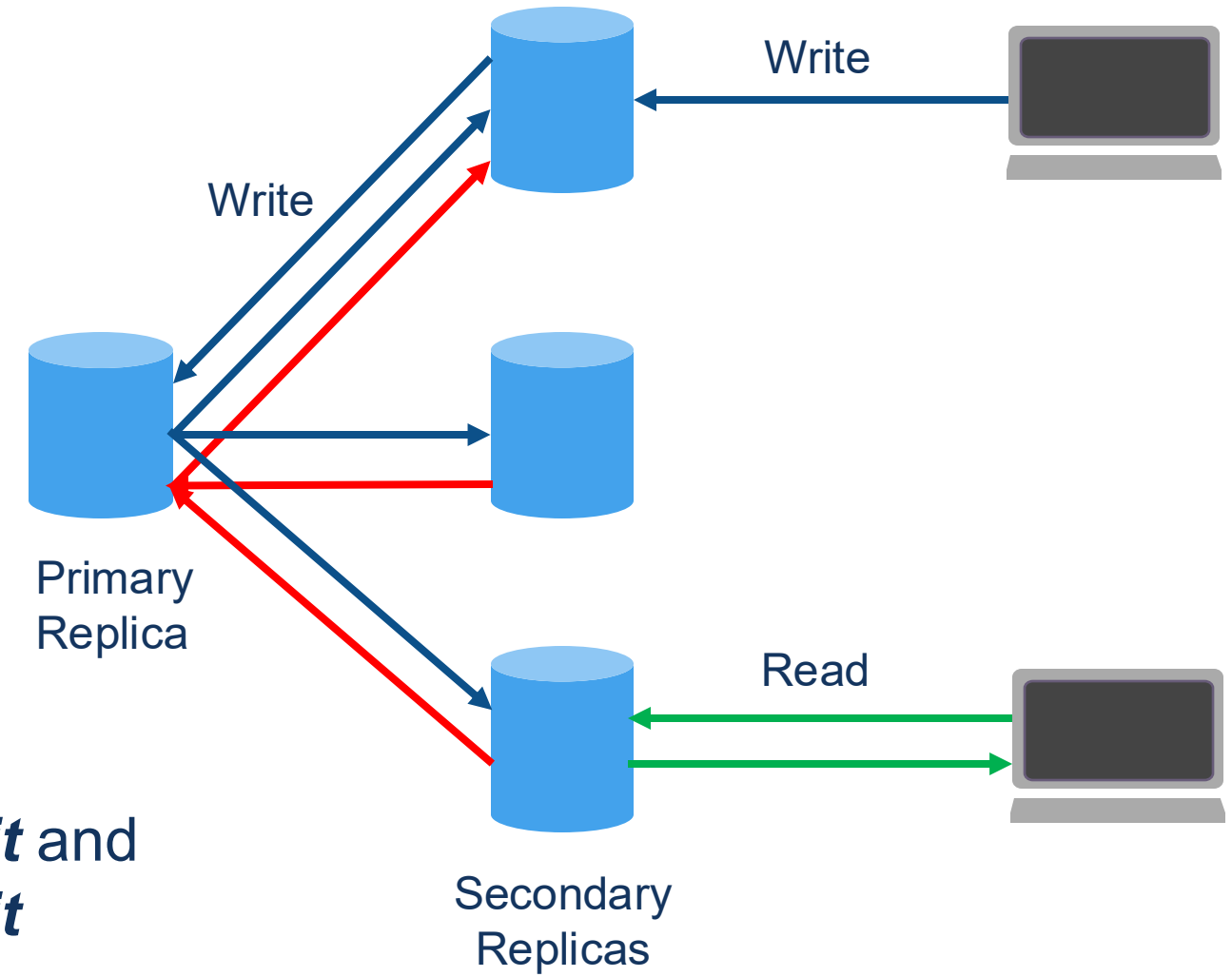
Distributed Consensus



Linearizability
↓
Sequential Consistency
↓
Causal Consistency
↓
Session Consistency
↓
Eventual Consistency
↓
Weak Consistency

Lecture 7: Leader Election and Distributed Consensus *(Olov)*

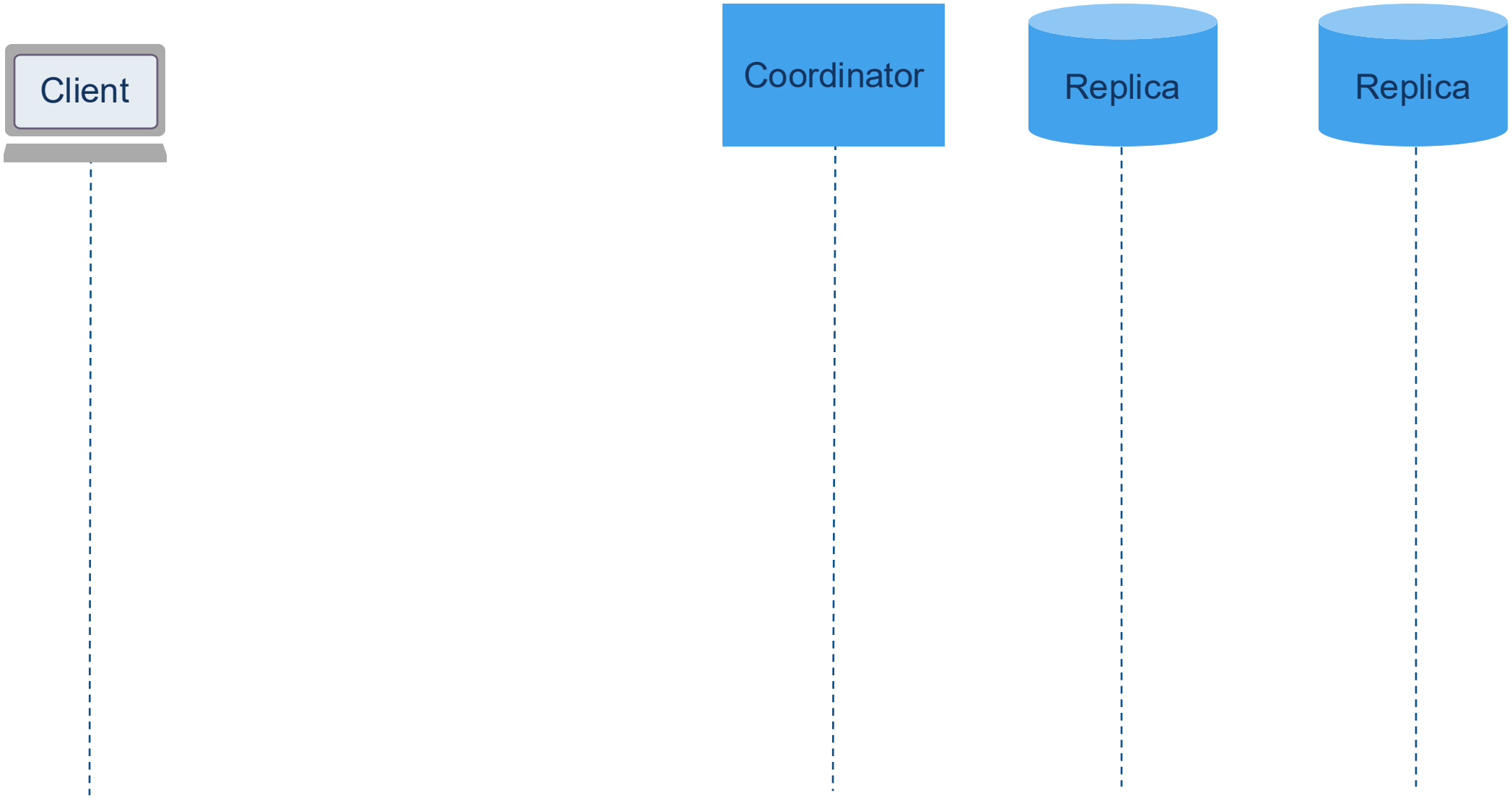
Atomicity



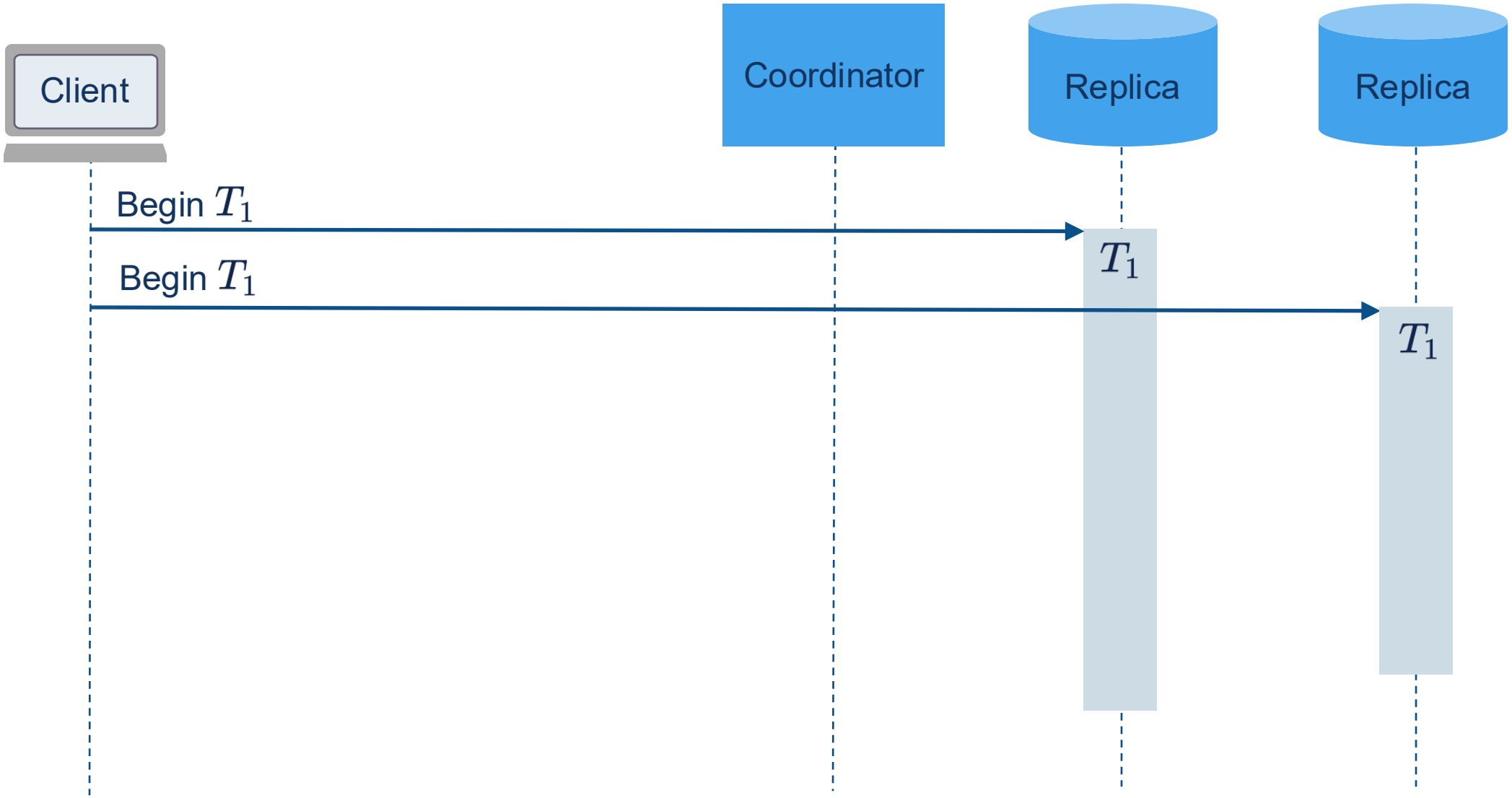
2-Phase Commit and
3-Phase Commit

Linearizability
↓
Sequential Consistency
↓
Causal Consistency
↓
Session Consistency
↓
Eventual Consistency
↓
Weak Consistency

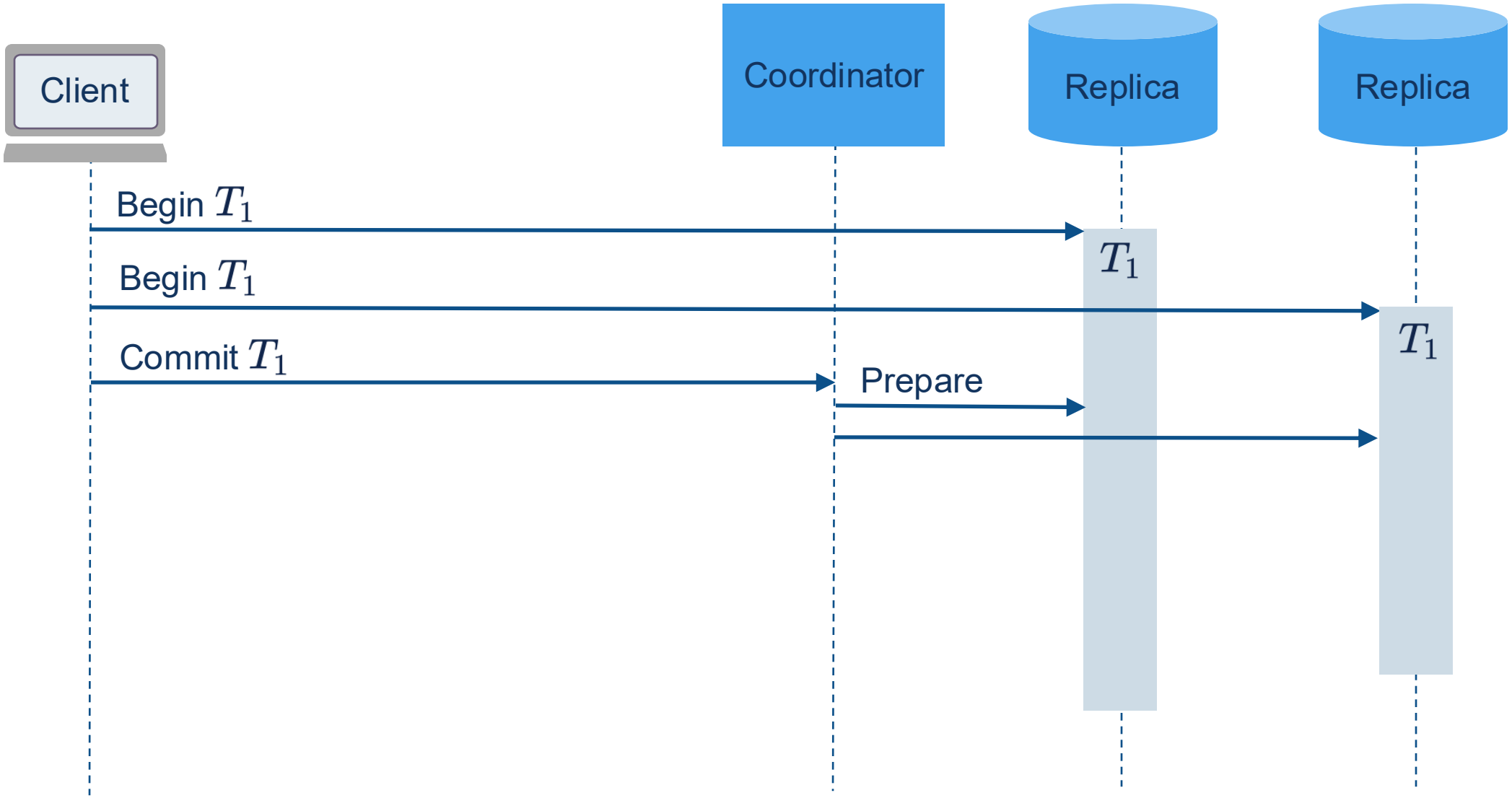
2-Phase Commit



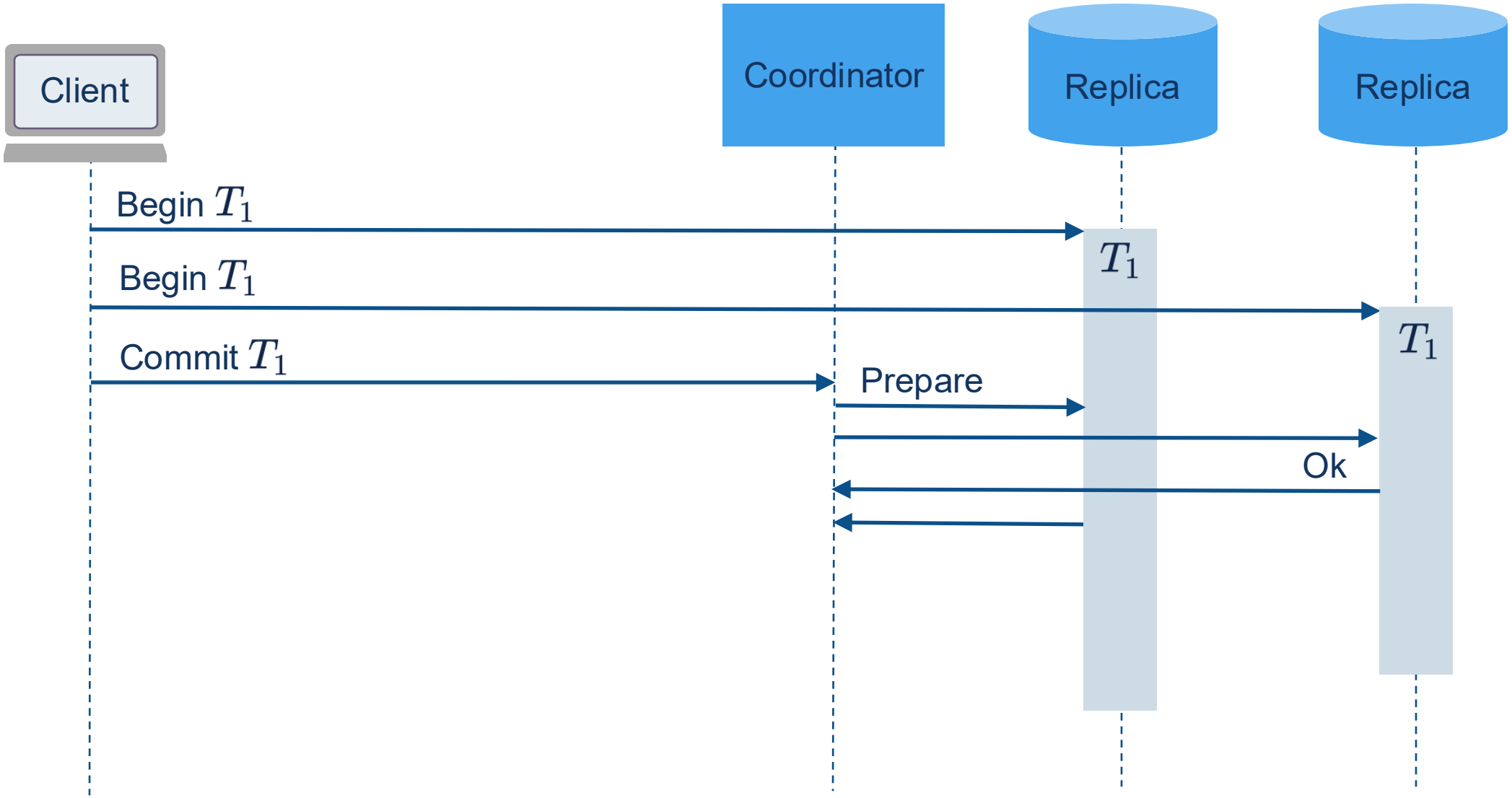
2-Phase Commit



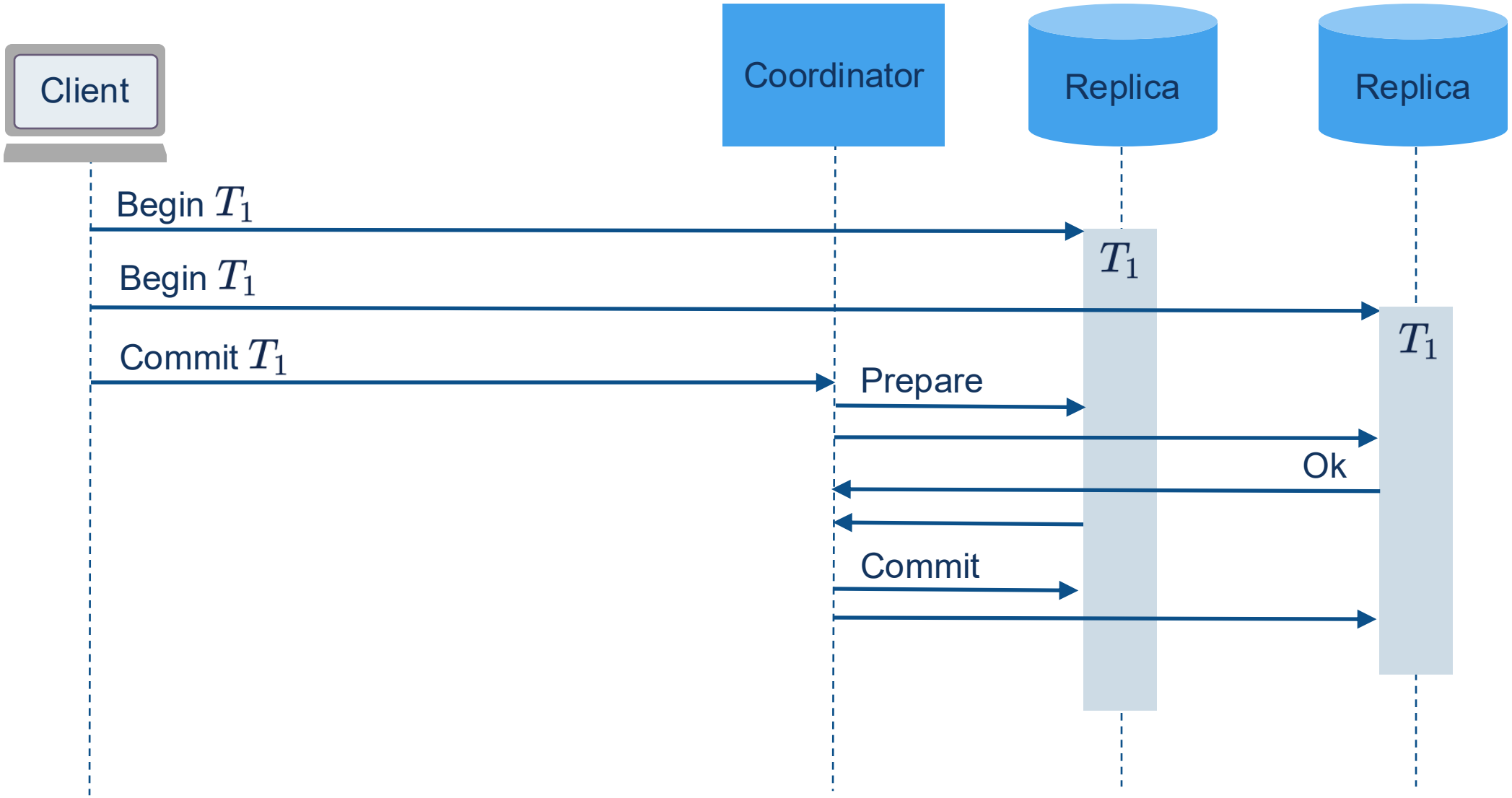
2-Phase Commit



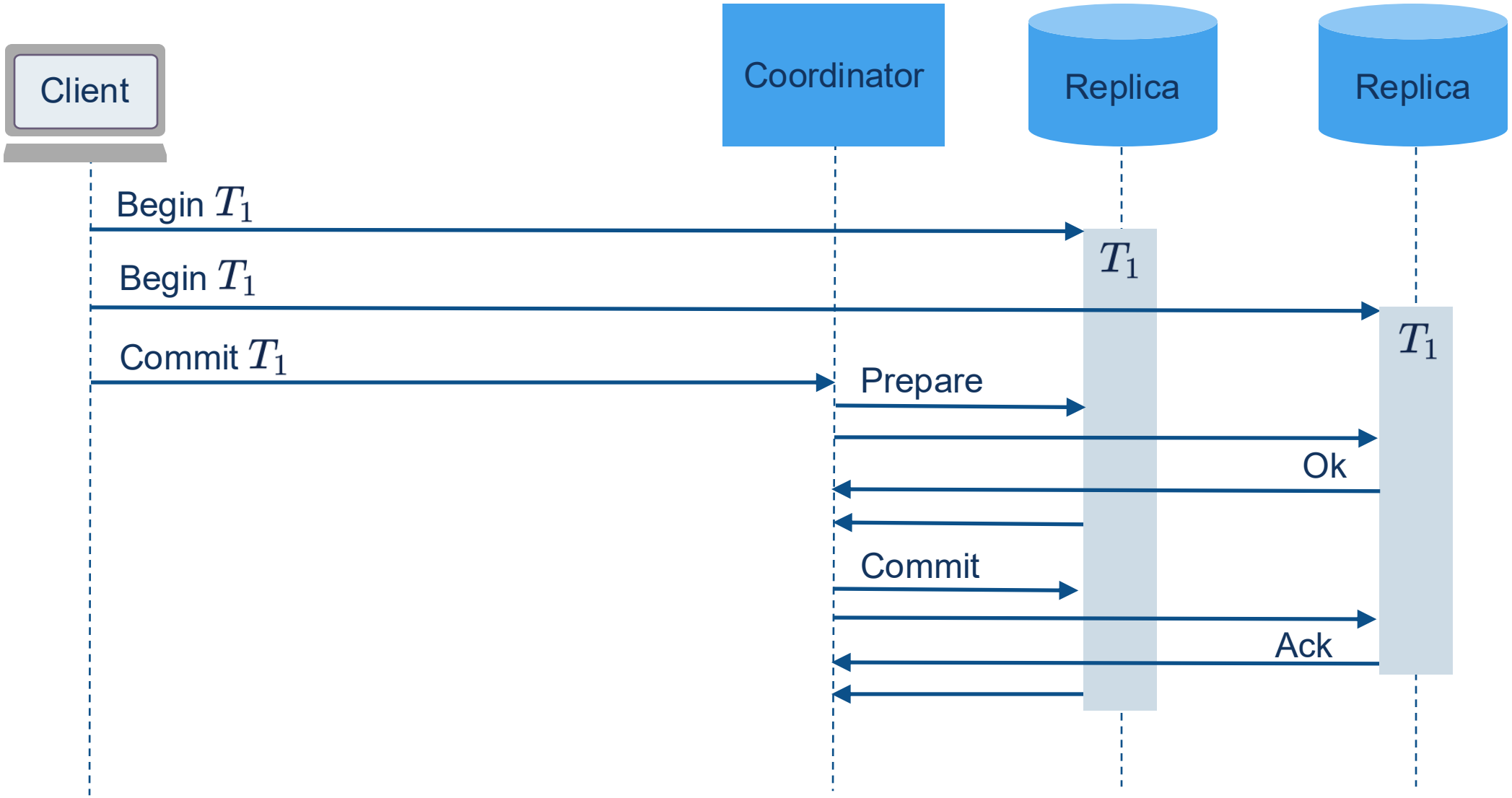
2-Phase Commit



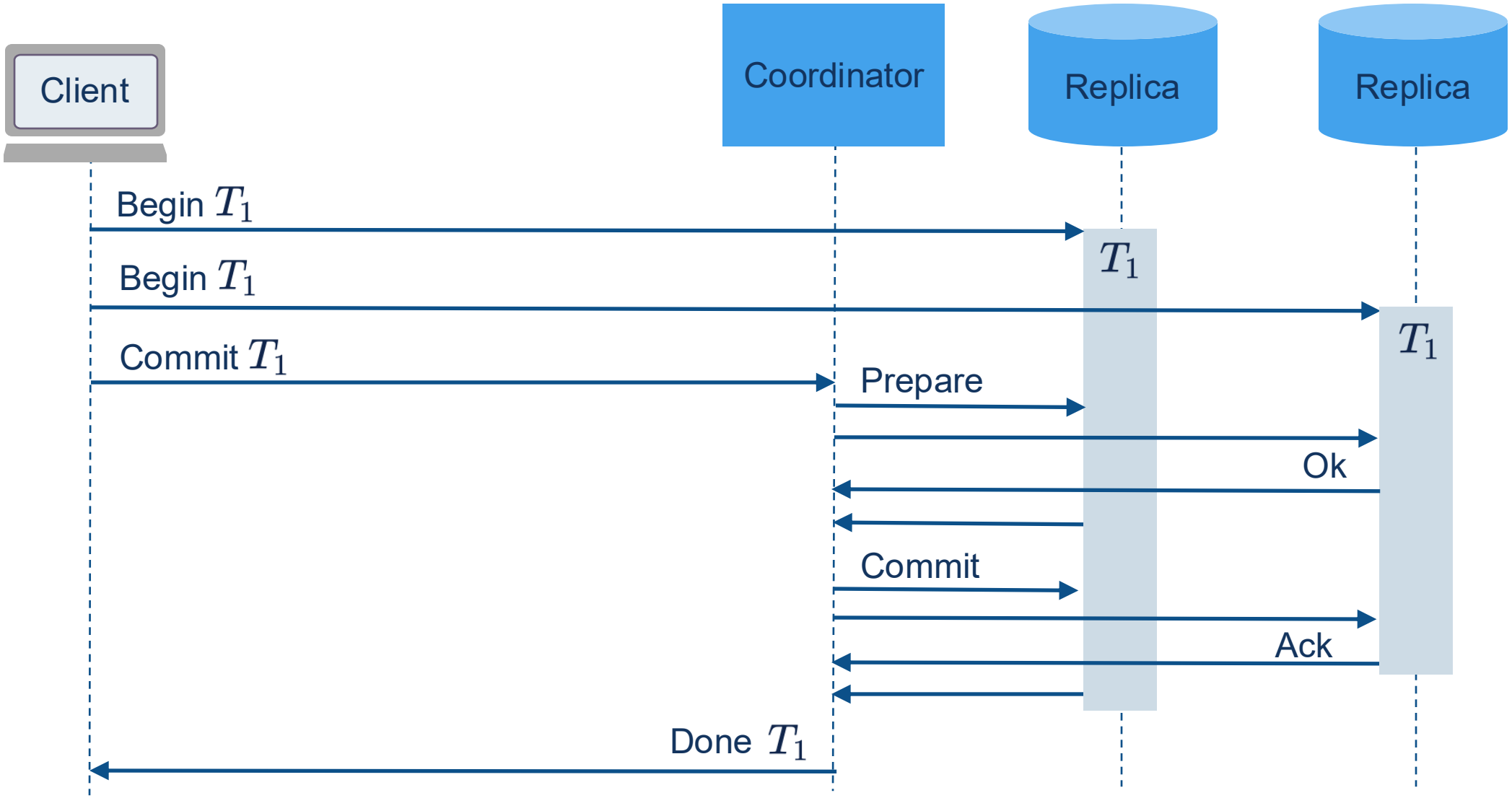
2-Phase Commit



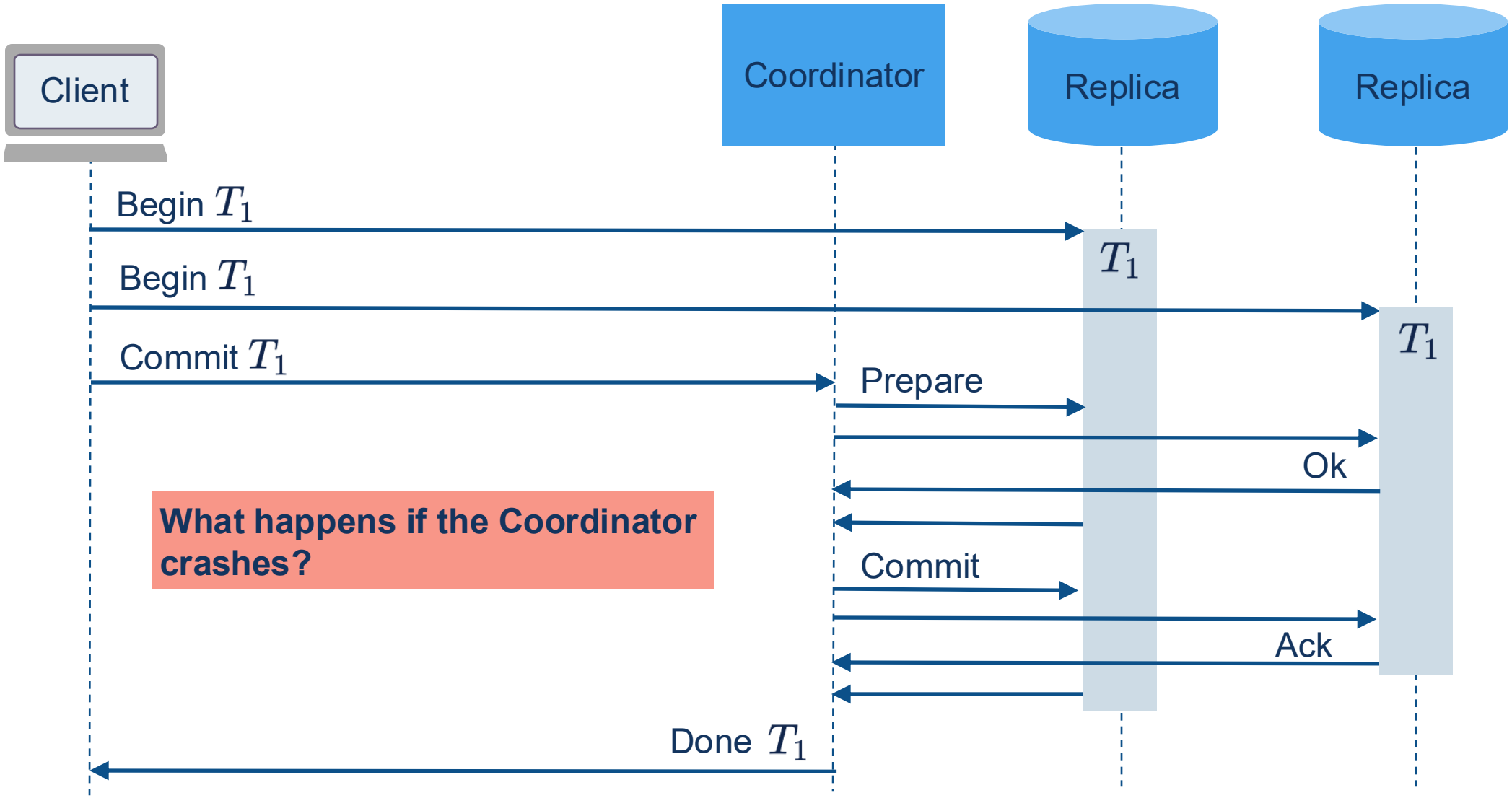
2-Phase Commit



2-Phase Commit



2-Phase Commit



Sequential Consistency

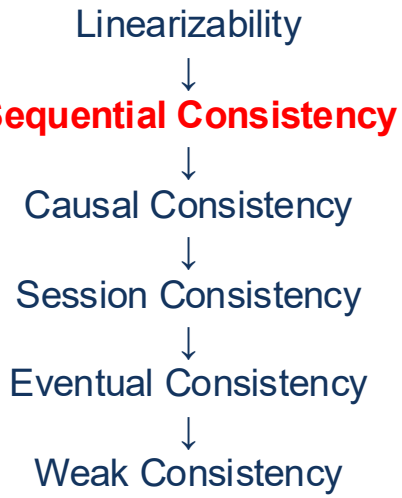
All operations appear in some total order respecting program order

Guarantee: Everyone agrees on total order, but may not match wall time

Example: Distributed database

Cost: Higher availability, better performance than strong consistency

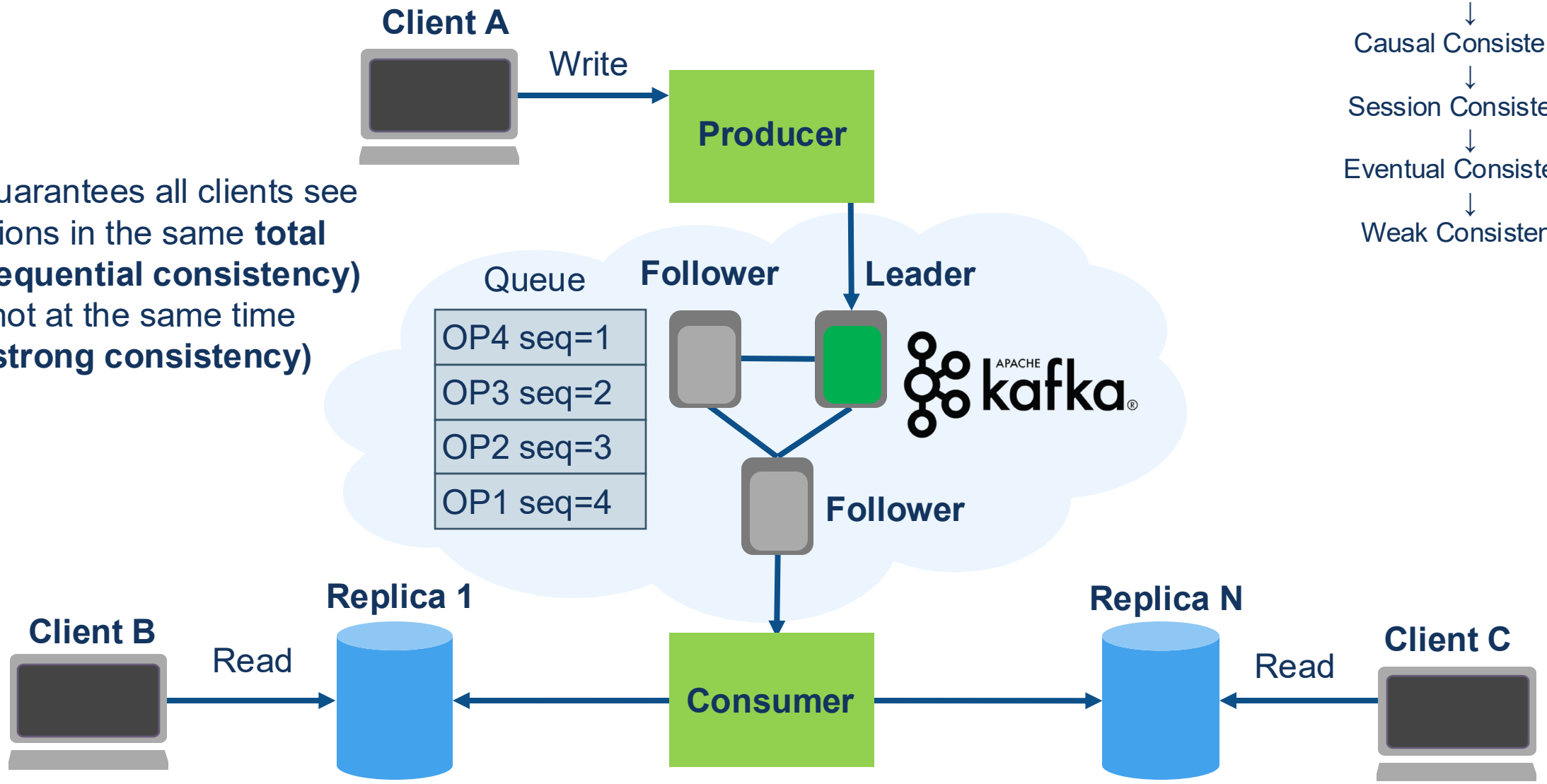
Implementation: Sequencer, blockchains



Sequential Consistency

Linearizability
↓
Sequential Consistency
↓
Causal Consistency
↓
Session Consistency
↓
Eventual Consistency
↓
Weak Consistency

Kafka guarantees all clients see operations in the same **total order (sequential consistency)** but not at the same time **(not strong consistency)**

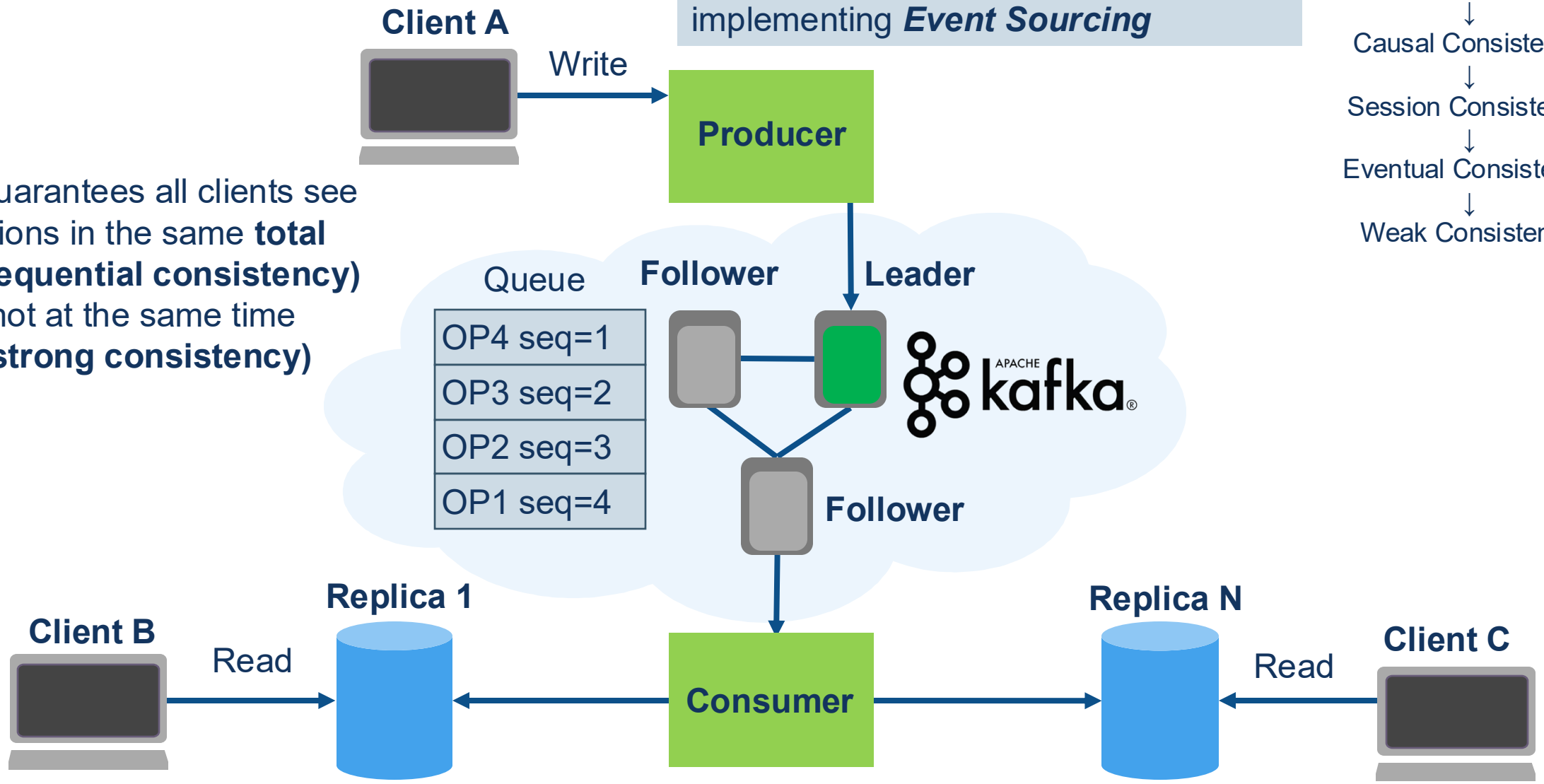


Sequential Consistency

Distributed log systems like Kafka provide the strong ordering and durability guarantees required for implementing ***Event Sourcing***

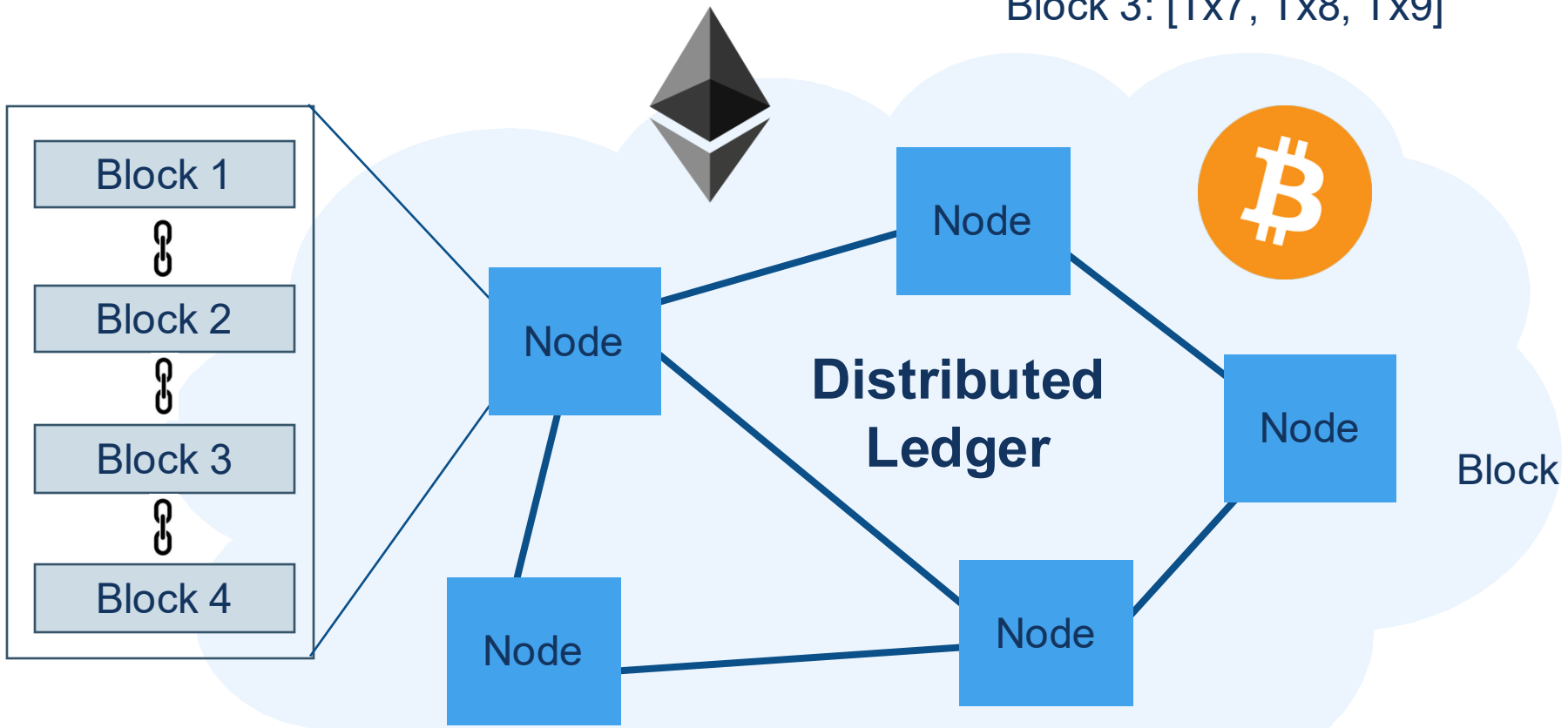
Linearizability
↓
Sequential Consistency
↓
Causal Consistency
↓
Session Consistency
↓
Eventual Consistency
↓
Weak Consistency

Kafka guarantees all clients see operations in the same **total order (sequential consistency)** but not at the same time **(not strong consistency)**



Sequential Consistency

Block 1: [Tx1, Tx2, Tx3]
Block 2: [Tx4, Tx5, Tx6]
Block 3: [Tx7, Tx8, Tx9]



Block 1 → Block 2

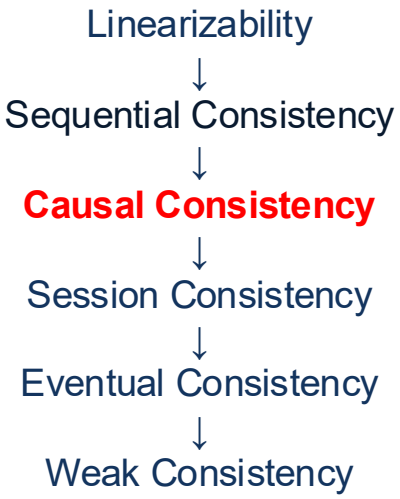
Block 1 → Block 2 → Block 3 → Block 4

All nodes will eventually have Block 1→2→3→4 but not at the same time **(Sequential Consistency)**

Linearizability
↓
Sequential Consistency
↓
Causal Consistency
↓
Session Consistency
↓
Eventual Consistency
↓
Weak Consistency

Lecture 8: Blockchains and Byzantine Fault Tolerance *(Christoffer)*

Causal Consistency



Definition: All operations appear in causal order, preserving happens-before relationships between operations

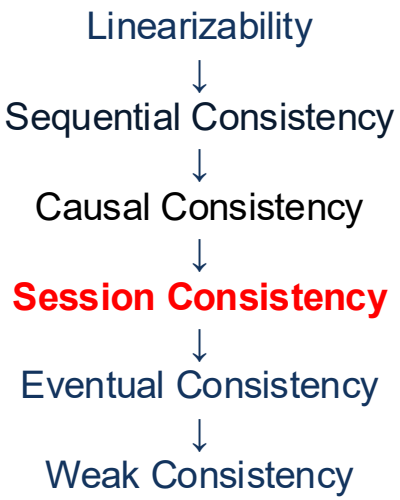
Guarantee: Causally related operations seen in same order, unrelated operations may differ across nodes

Example: Social media comments/replies, chat applications,

Cost: Better performance than sequential, weaker guarantees than total order

Implementation: Vector clocks, dependency graphs, Happens-Before

Session Consistency



Definition: Guarantees that within a session, reads and writes are seen in the order issued (Read Your Writes, Monotonic Reads, Monotonic Writes, Writes Follow Reads), but not across sessions

Guarantee: Causally related operations seen in same order, unrelated operations may differ across nodes

Example: Social media comments/replies, chat applications

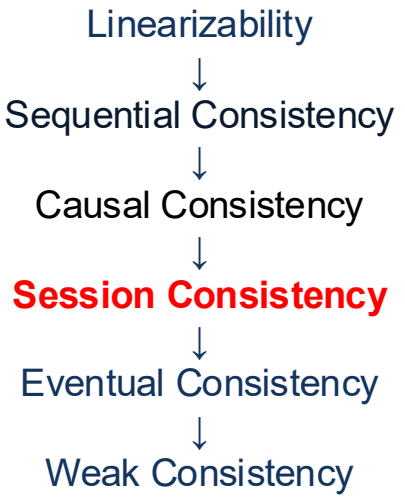
Cost: Better performance than sequential, weaker guarantees than total order

Implementation: Vector clocks, dependency graphs, sequence IDs

Session consistency actually provides four specific guarantees:

- Read Your Writes - You see your own updates
- Monotonic Reads - You never go backwards
- Monotonic Writes - Your writes apply in order
- Writes Follow Reads - Your writes come after what you've read

Session vs Causal Consistency



Causal Consistency Example

Alice: "Post: Who wants pizza?"
Bob: "Reply: I do!"
Carol: MUST see post before reply (causally related)

Session Consistency Example

Alice: Updates profile picture
Alice: Immediately sees new picture ✓
Bob: Might still see old picture
Carol: Might see old picture

Only Alice is guaranteed to see her change

Shopping Cart Example

Causal Consistency:

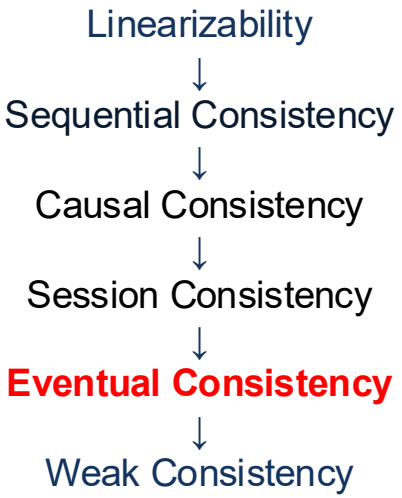
- You add item and text friend "I added it"
- Friend reads your message then checks cart
- Friend MUST see the item (causal: add → message → check)

Session Consistency:

- You add item → You see it
- Friend in same account → May not see it yet

Eventual Consistency

- Ensures that all replicas will eventually become consistent if no new updates are made, but not the order or causal visibility
- Replicas may diverge temporarily but will eventually become consistent; no promises about when or in what order
- Example: Distributed caches, NoSQL databases
- Maximum availability (works during partitions), lowest latency, but applications must handle inconsistency

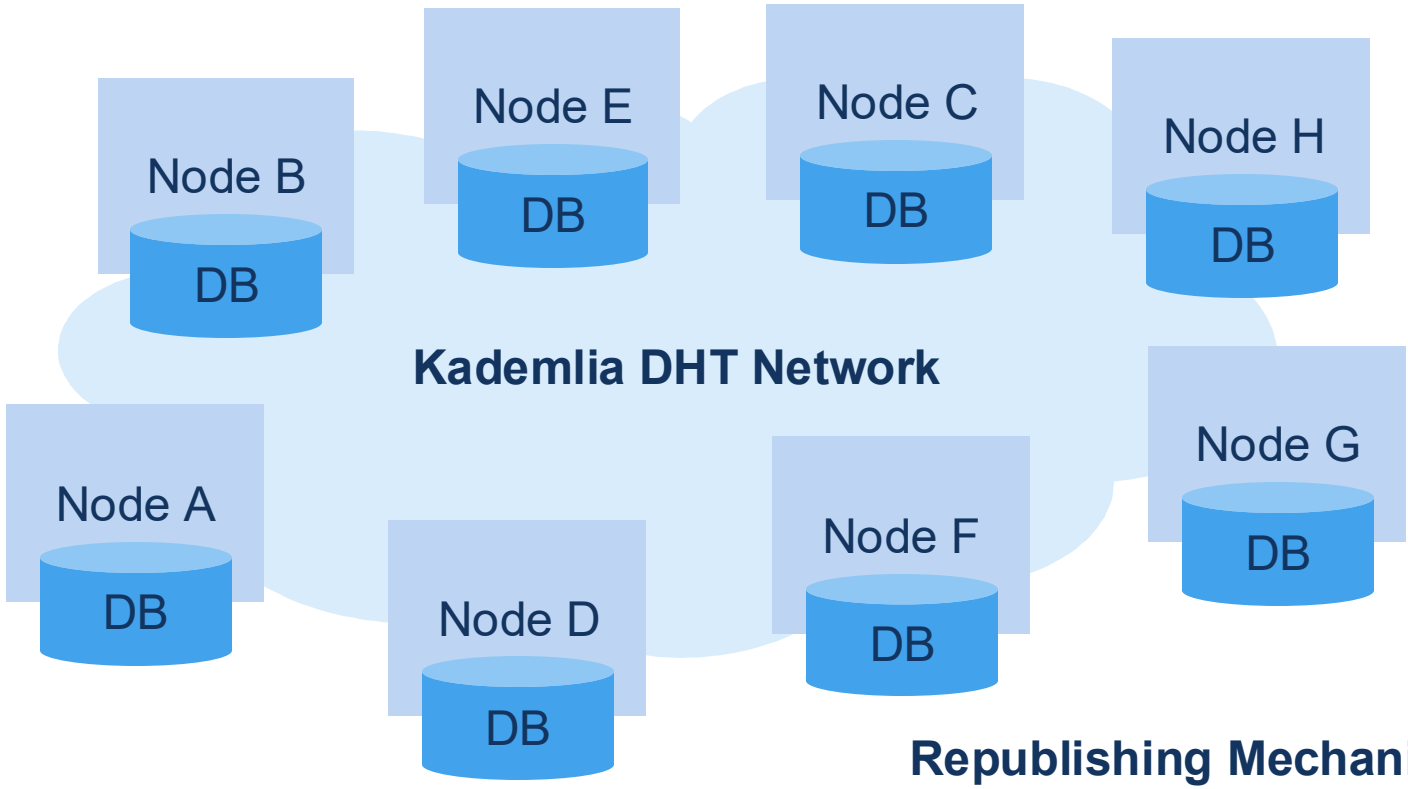


Eventual Consistency

Linearizability
↓
Sequential Consistency
↓
Causal Consistency
↓
Session Consistency
↓
Eventual Consistency
↓
Weak Consistency

Store operation:

1. Find k closest nodes to key
2. Store on those k nodes
3. No waiting for confirmation from all
4. Other nodes learn gradually through lookups



Republishing Mechanism:

1. Original publisher republishes every hour
2. Each node checks its stored data hourly
3. If node has data, find k closest nodes to key
4. Send to any of k closest that don't have it
5. Maintain k copies despite node failures
6. Data expires after 24 hours without refresh

Eventual Consistency

Linearizability
↓
Sequential Consistency
↓
Causal Consistency
↓
Session Consistency
↓
Eventual Consistency
↓
Weak Consistency

Event 1: Order Received (7:00 PM):
DHT.Store("order:12345", { "status": "Order received", "time": "7:00 PM" })
→ **Stored on nodes [A, B, C]**

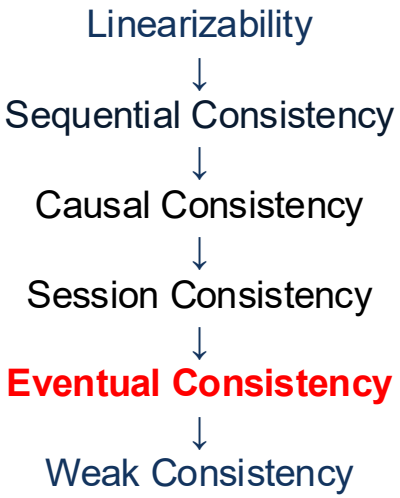
Event 2: Pizza in Oven (7:15 PM):
DHT.Store("order:12345", { "status": "Cooking", "time": "7:15 PM" })
→ **Stored on nodes [D, E, F]** ***Different nodes!***

Event 3: On the way (7:30 PM):
DHT.Store("order:12345", { "status": "On the way", "time": "7:30 PM" })
→ **Stored on nodes [A, G, H]** ***Node A gets update***

Node A: "On the way!" (Latest)
Node B: "Order received" (Never got updates)
Node C: "Order received" (Never got updates)
Node D: "Cooking" (Missed delivery update)
Node E: "Cooking" (Missed delivery update)

NOTE: Republishing ensures eventual consistency, e.g., after 1h all nodes will converge to Event 3

Kademlia Sweet Spots



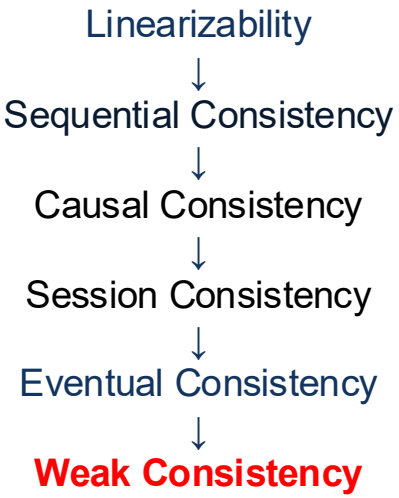
Perfect Use Cases

- **BitTorrent DHT** - Finding peers who have files
- **CDN Cache** - Locating content servers
- **Content Addressing** - IPFS-style hash to location
- **Public Keys** - Cryptographic key distribution
- **Blockchain Headers** - Immutable block data

Why these works?

- Immutable data - Content never changes
- Read-heavy - Infrequent updates
- Multiple sources good - More peers = better availability
- Order doesn't matter - Any valid copy works

Weak Consistency



Definition: All operations may appear in any order, with no guarantees about relationships or visibility

Guarantee: Operations might propagate to some, all, or no nodes; no promises about order or timing

Example: Game physics approximations

Cost: Maximum performance for real-time rendering, players accept minor inconsistencies for smooth gameplay

Implementation: UDP networking, position interpolation etc.

Lecture 9: Distributed Gaming Backends *(Carl)*

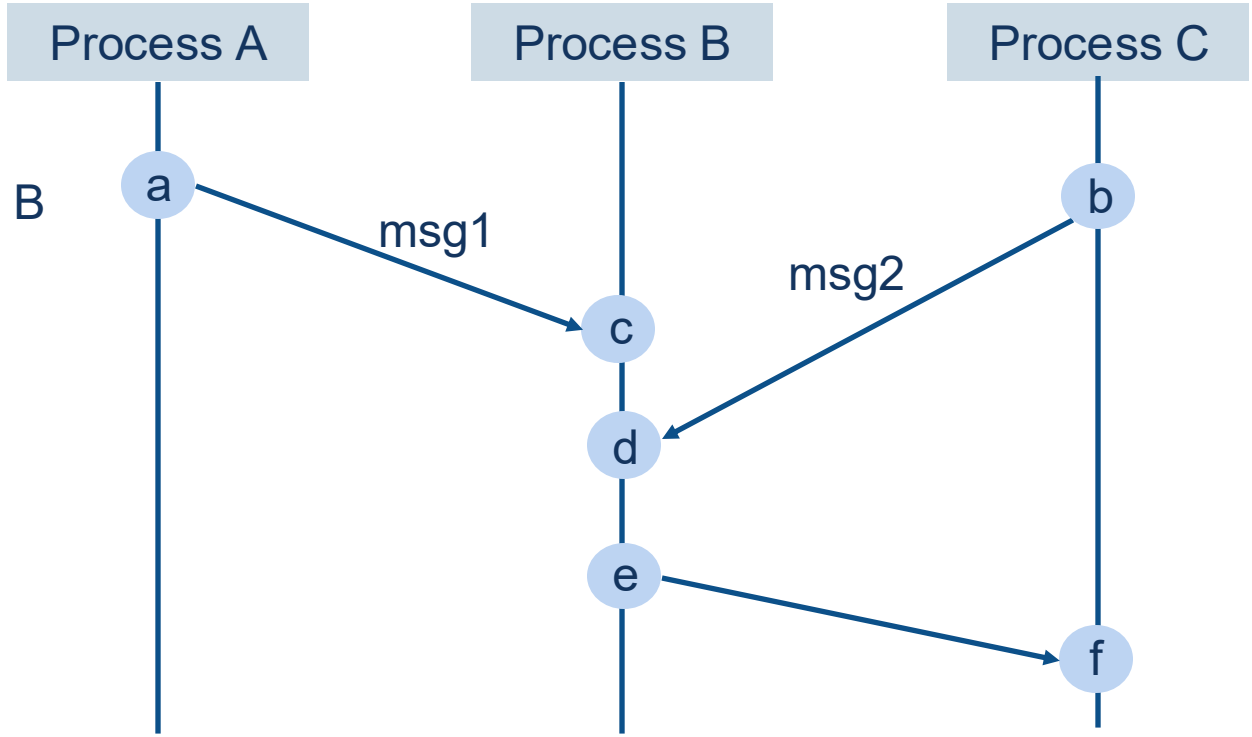
Causality in Distributed Systems

Causality: The relationship between cause and effect, when one event causes or influences another event

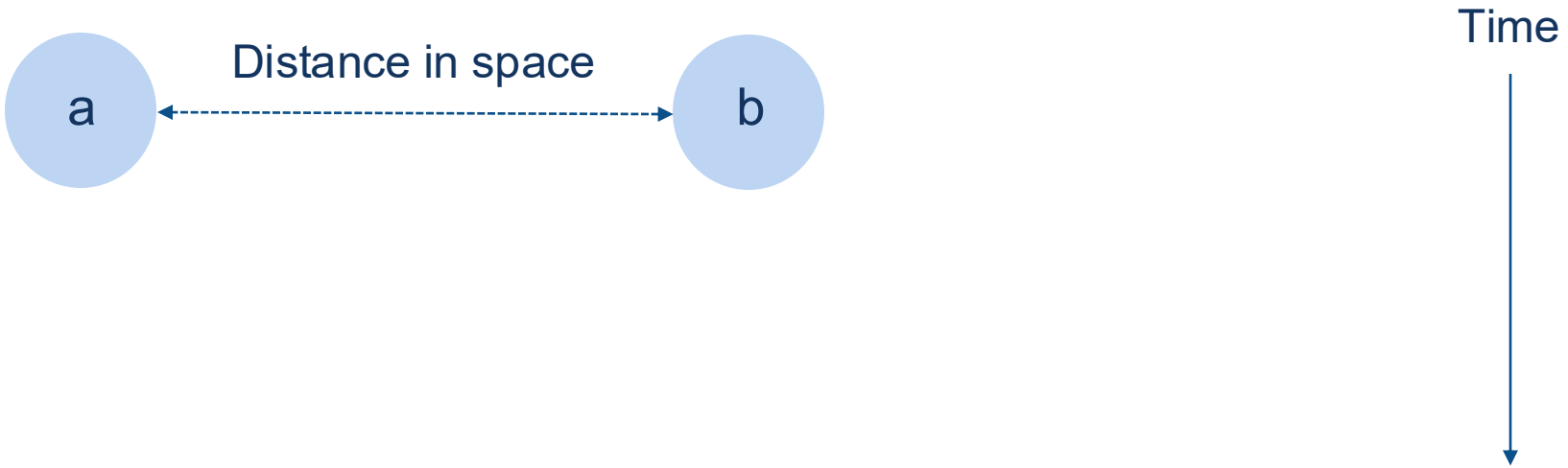
In a distributed system, the only way one process can influence another is by sending a message

Process A influences Process B by sending a message

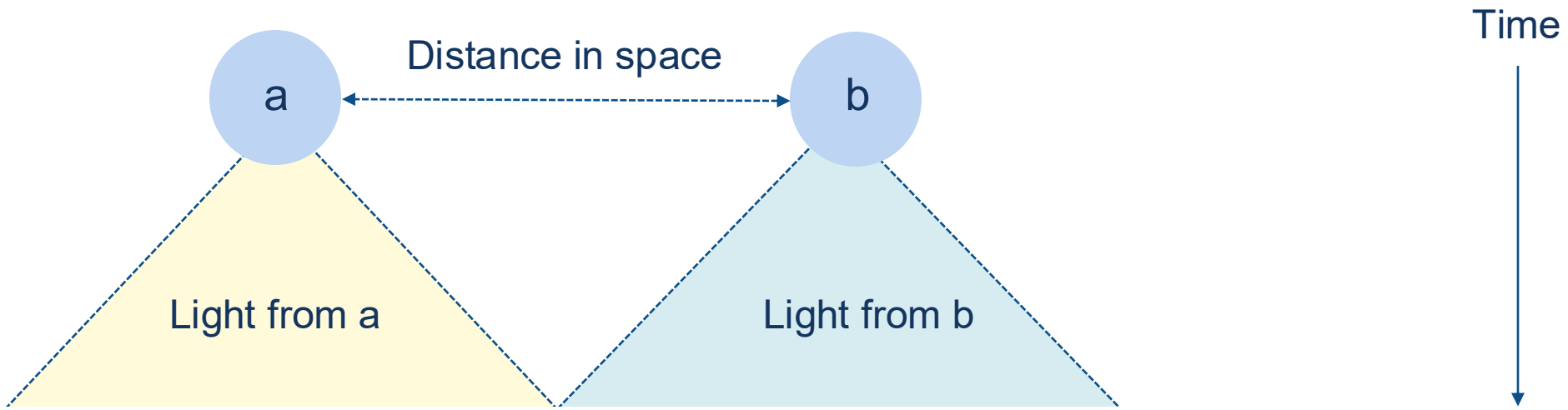
Process B changes its own state when it receives a message



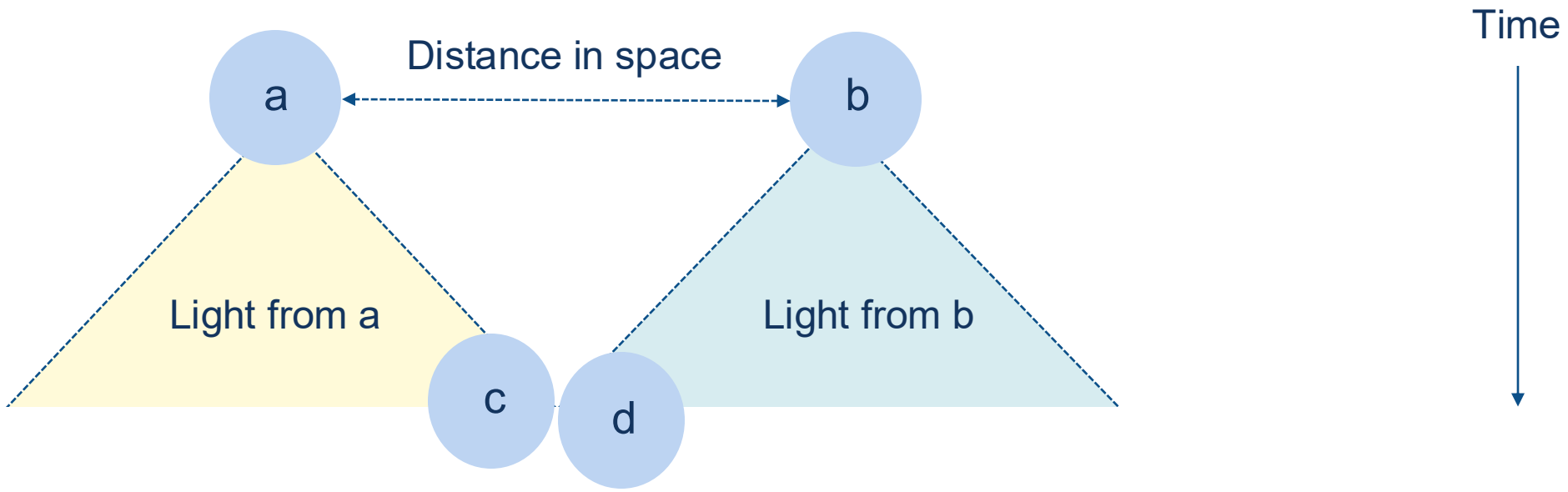
Causality



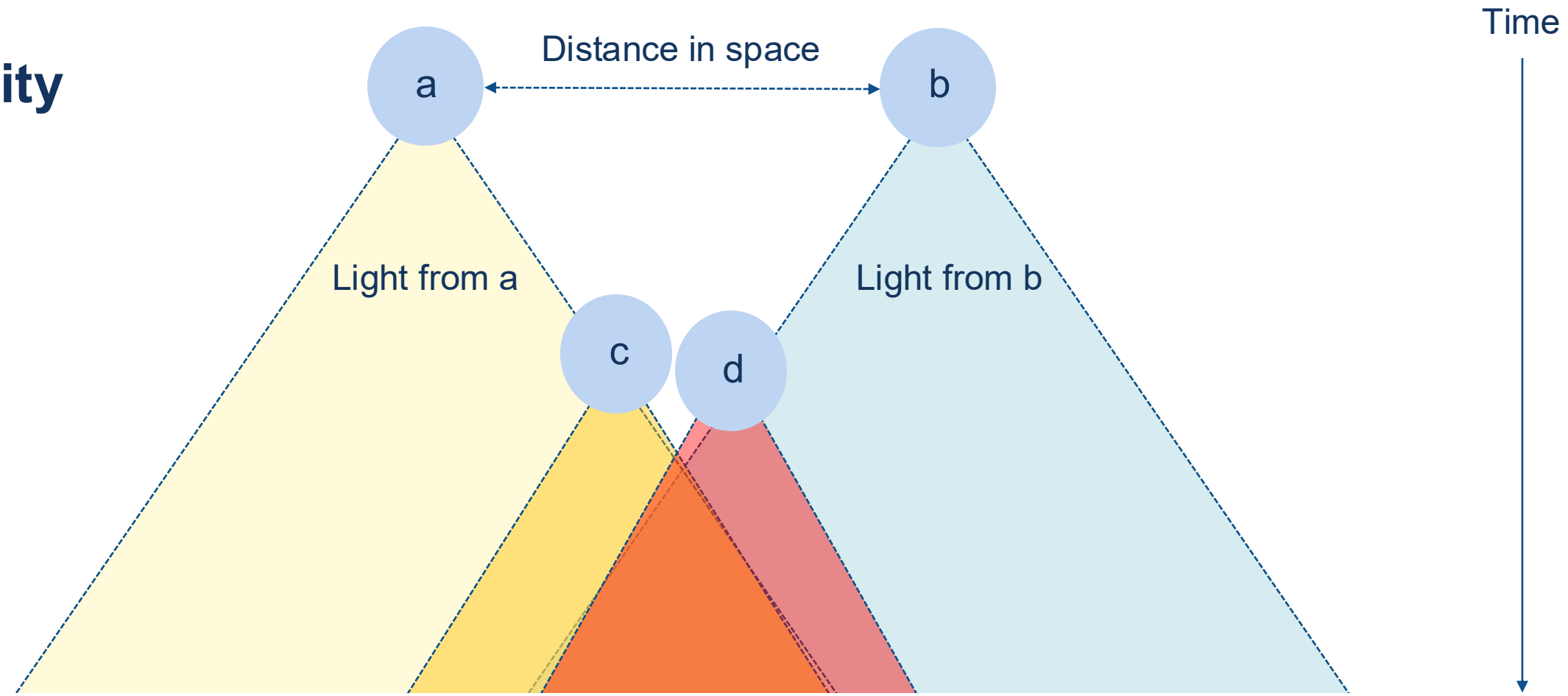
Causality



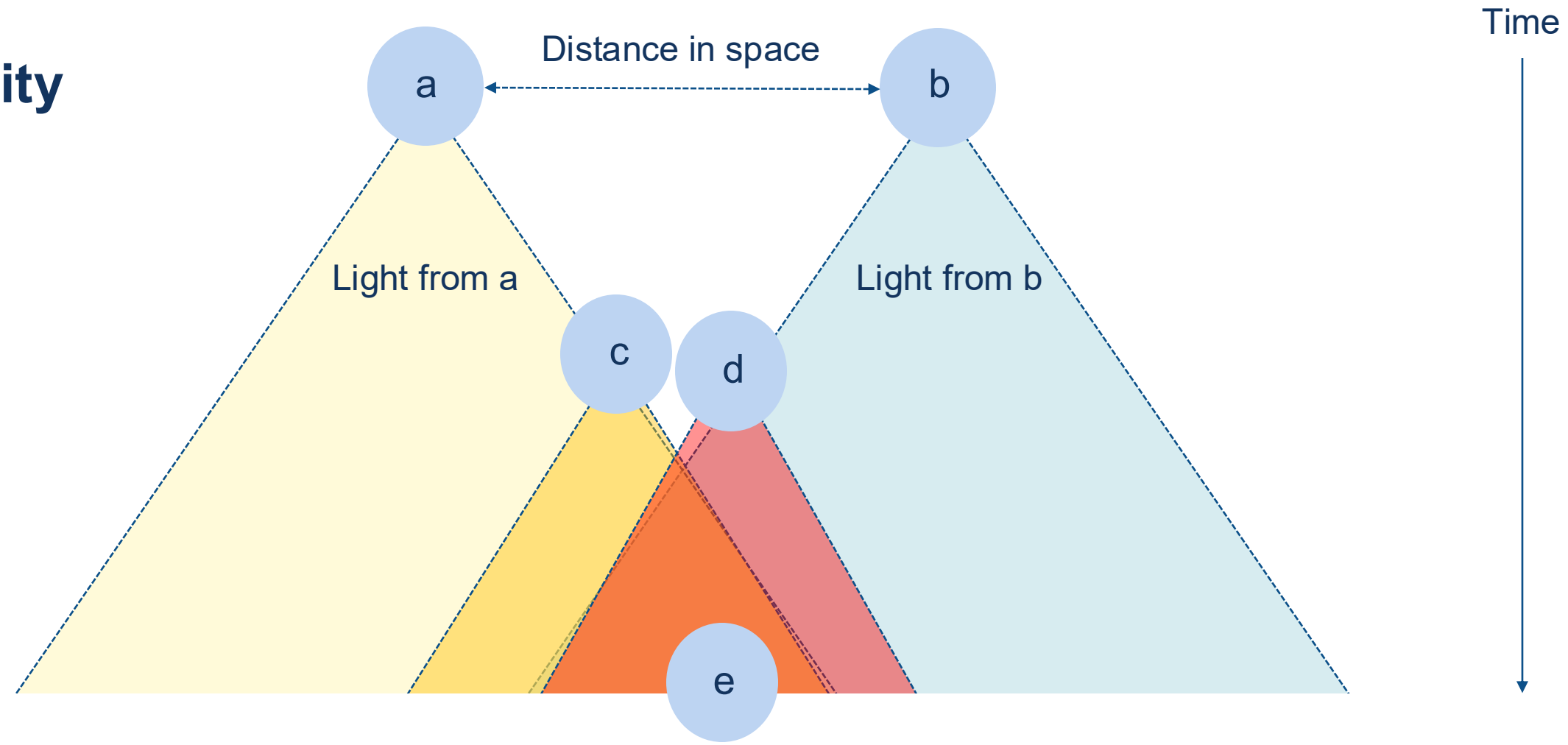
Causality



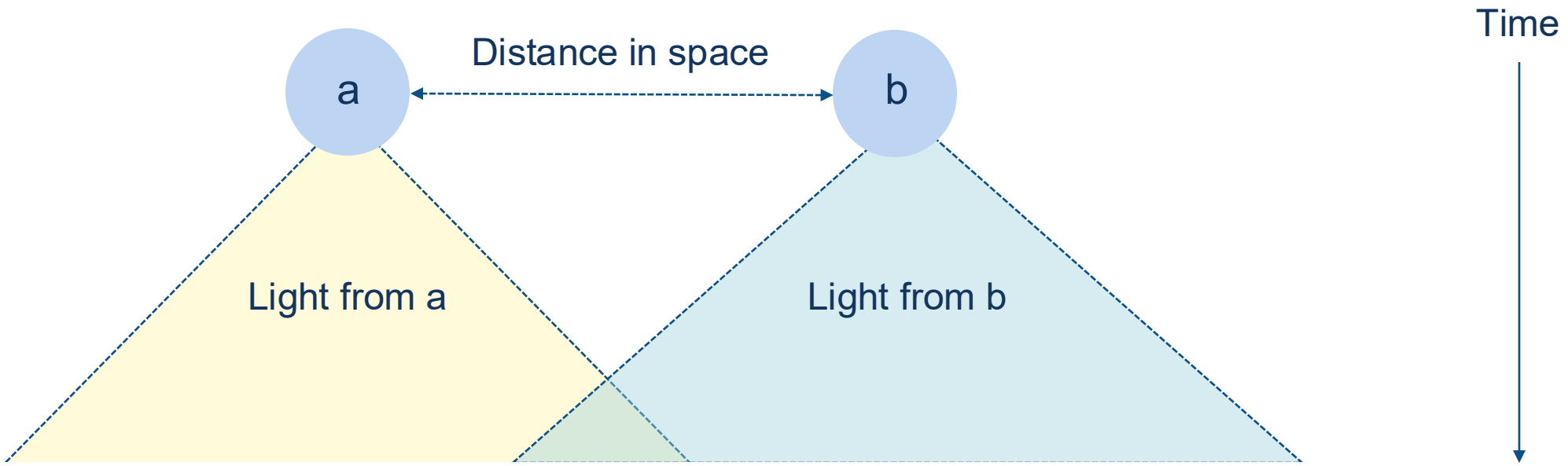
Causality



Causality



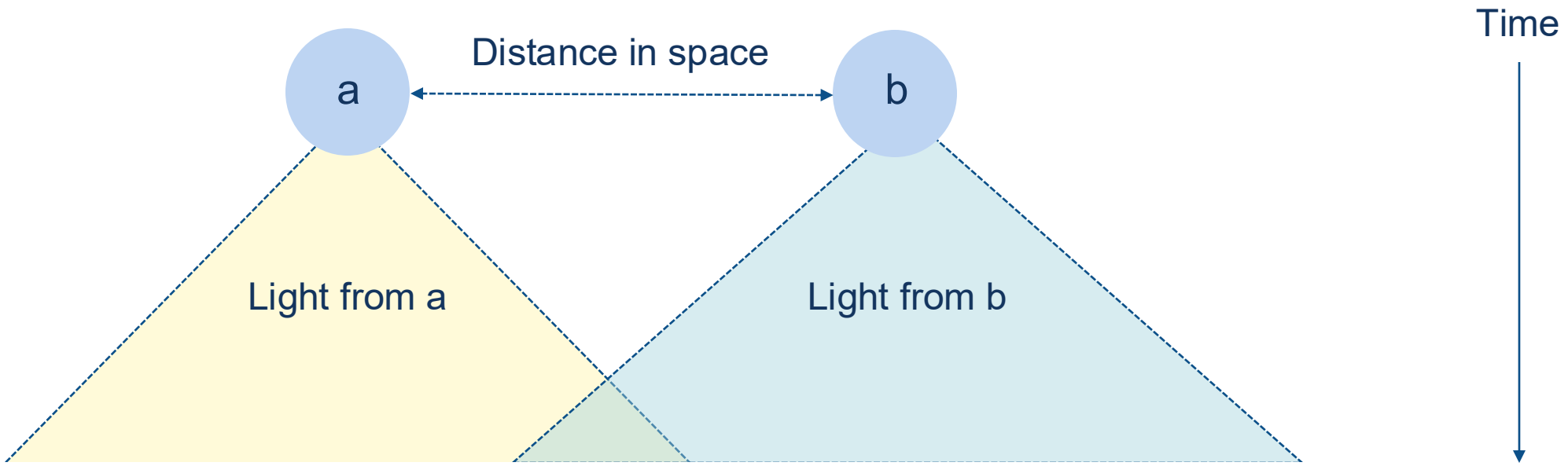
Causality



Happened-before (denoted $\rightarrow$) encodes potential causuality
Relation between two events so that one event happened before the other event

When $a \rightarrow b$, then event **a might** have caused event **b**
When $a \parallel b$, then event **a cannot** have caused event **b**

Causality



Happened-before (denoted $\rightarrow$) encodes potential causuality
Relation between two events so that one event happened before the other event

When $a \rightarrow b$, then event **a might** have caused event **b**
When $a \parallel b$, then event **a cannot** have caused event **b**

Transitivity

$\forall a, b, c \quad a \rightarrow b \text{ and } b \rightarrow c \Rightarrow a \rightarrow c$

Asymmetry

$\forall a, b \quad a \rightarrow b \Rightarrow b \nrightarrow a$

Irreflexively

$\forall a, a \nrightarrow a$

Ordering

Total Order

- Every pair of events is comparable
- One global sequence everyone agrees on

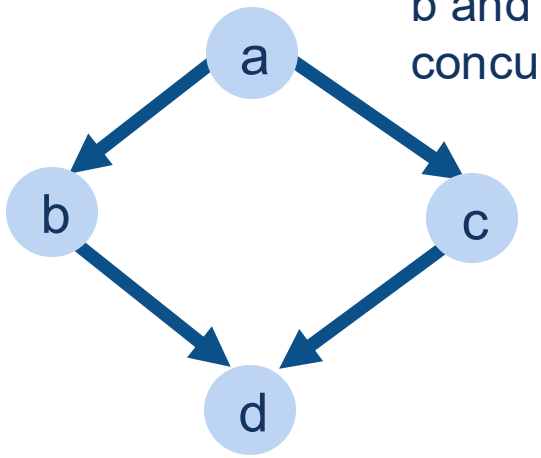


Total Order: a,b,c,d

Causal Order

- Events are ordered only if one causally influences another (i.e., happens-before)
- Allows more concurrency, less overhead

b and c happens concurrently



Valid Total Orders: a,b,c,d or a,c,b,d

$a \prec b$ means event a comes before event b in total order

$a \prec b \prec c \prec d$

Logical Clocks

- Mechanism to capture causal relationships in distributed systems
  - First Logical Clock was developed by Leslie Lamport in 1978 (Turing Award 2013)
- Lamport Timestamp
  - Assigns a numerical timestamp (L) to each event to preserve causal relationships
  - If $a \rightarrow b \Rightarrow L(a) < L(b)$ $\quad L(a) < L(b) \not\Rightarrow a \rightarrow b$

Since a and b could happen concurrently

Local Event

```
def on_local_event():
   local_clock += 1
   return local_clock
```

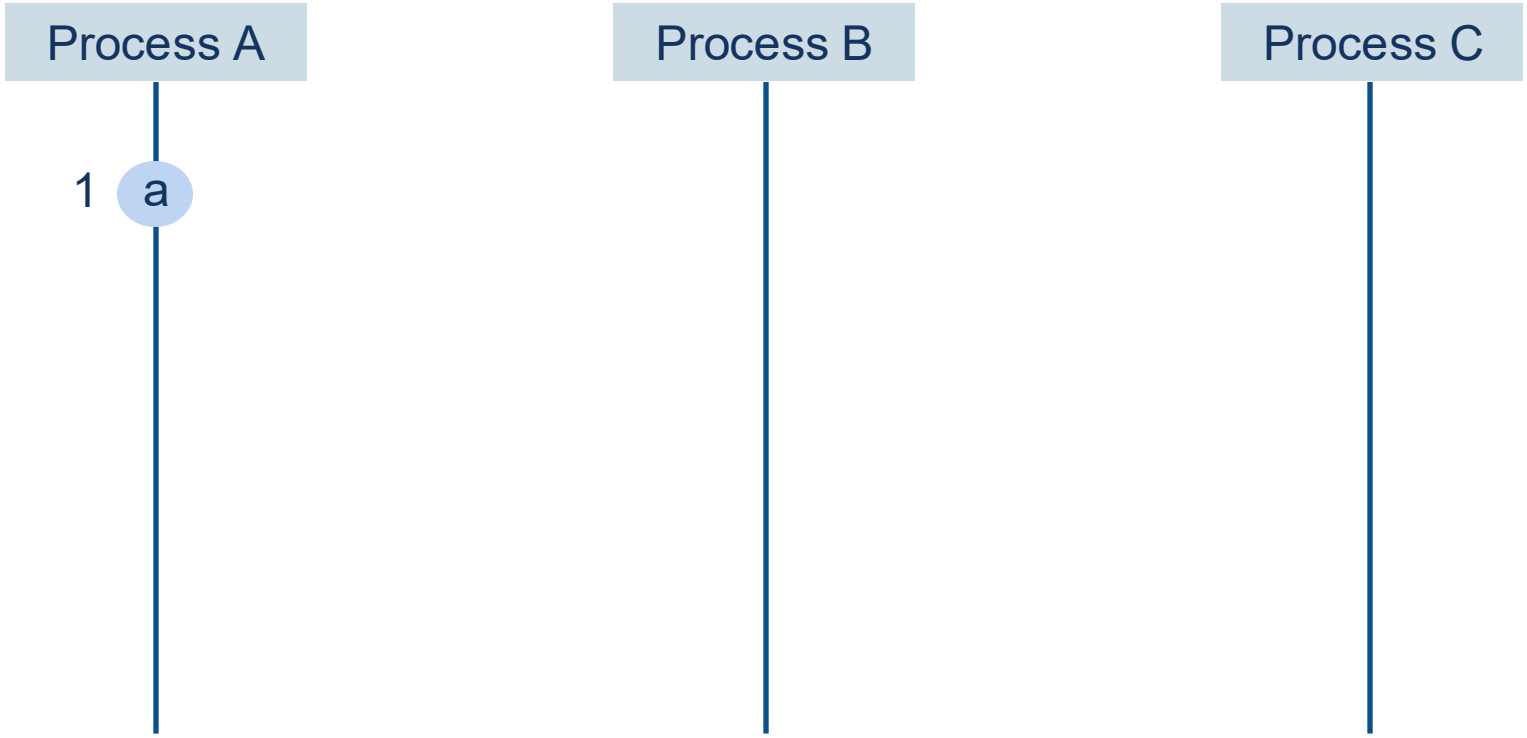
Sending

```
def send_message(msg):
    local_clock += 1
    msg.timestamp = local_clock send(msg)
```

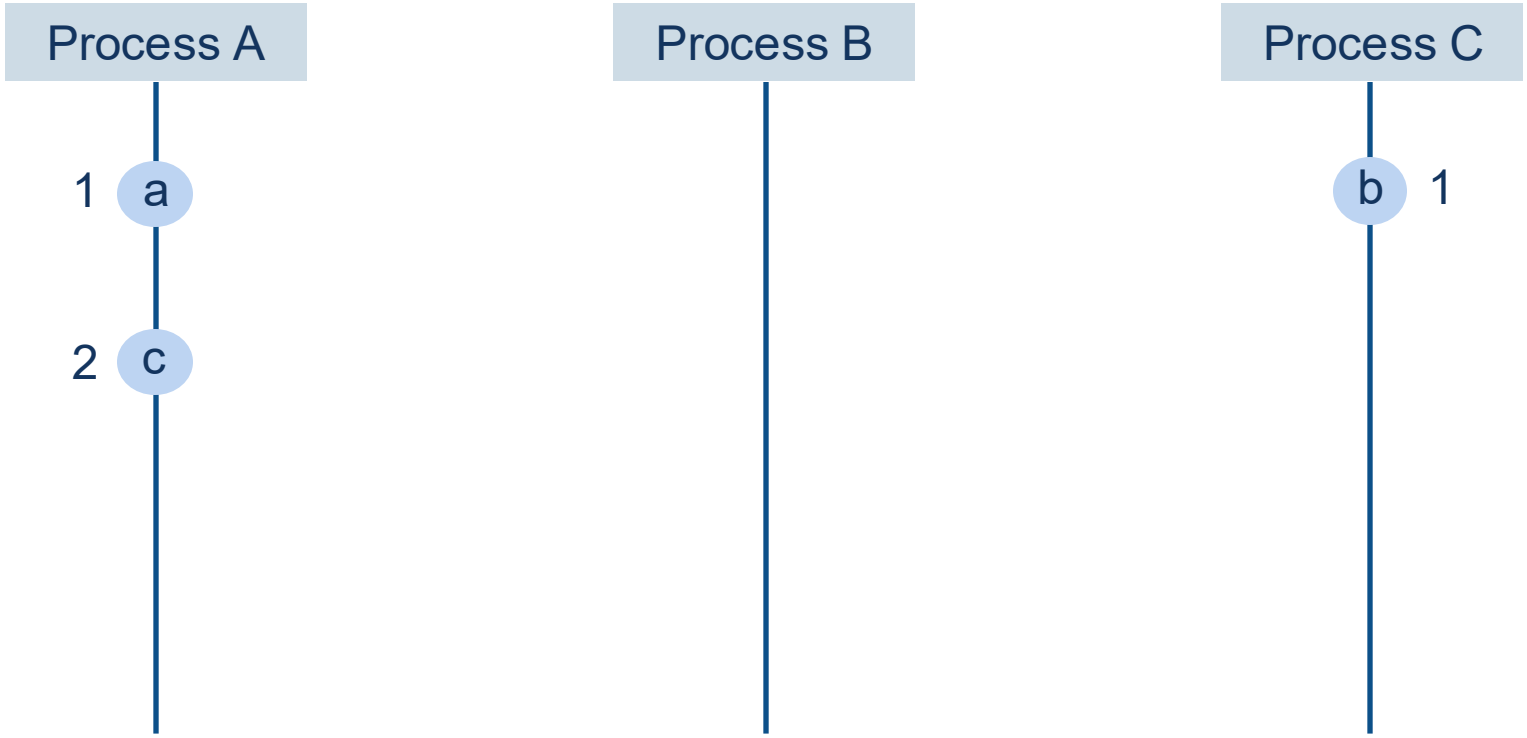
Receiving

```
def receive_message(msg):
    local_clock = max(local_clock, msg.timestamp)+1
    process(msg)
```

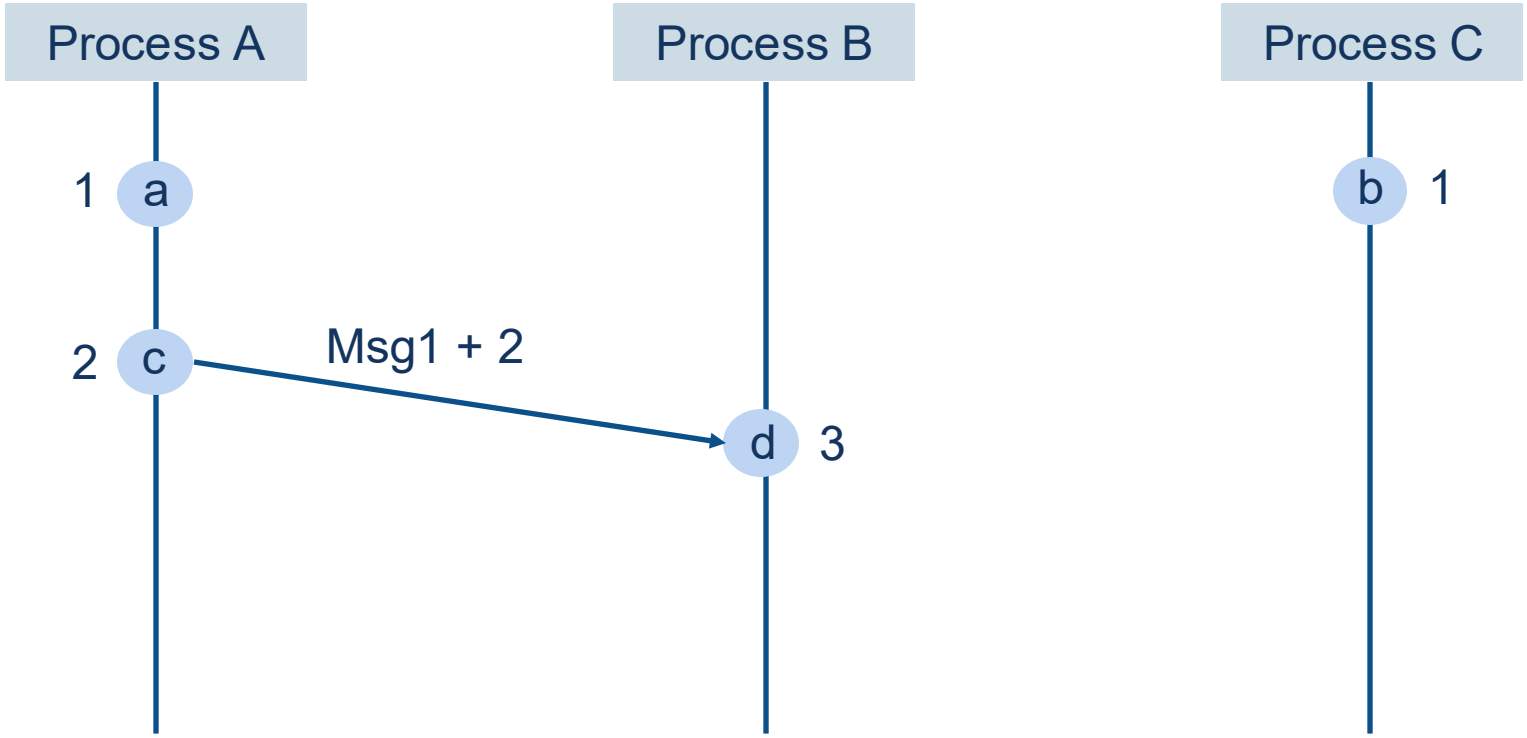
Lamport Clock



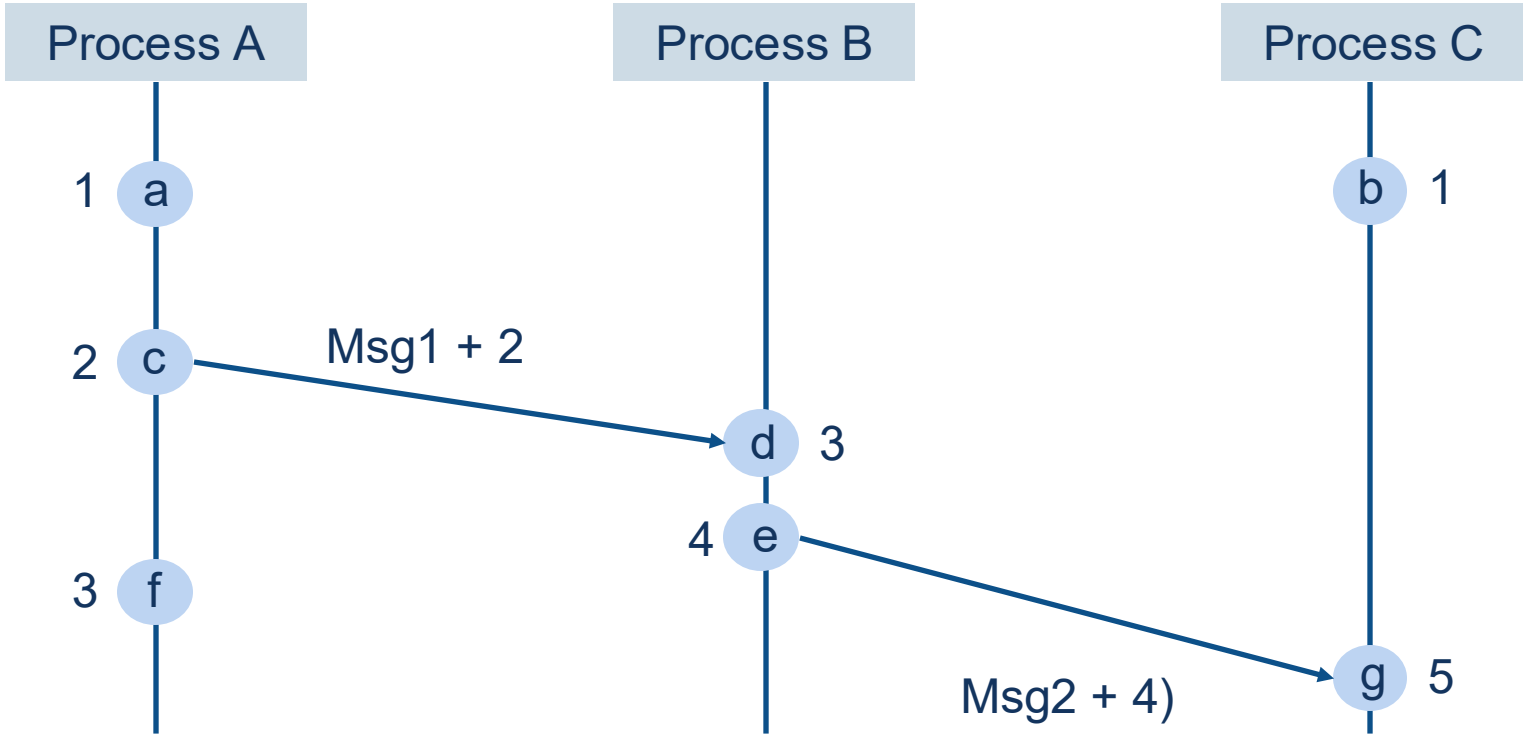
Lamport Clock



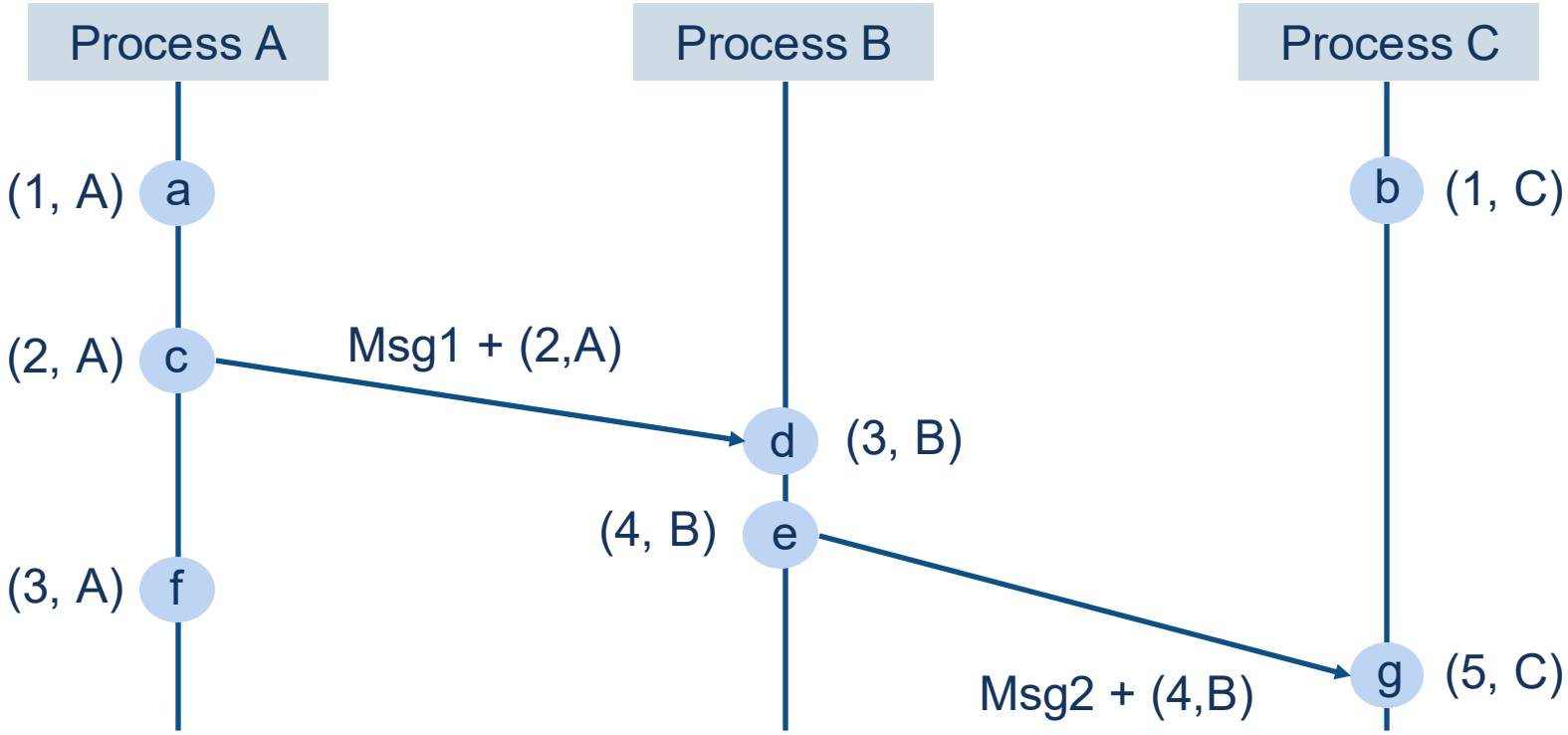
Lamport Clock



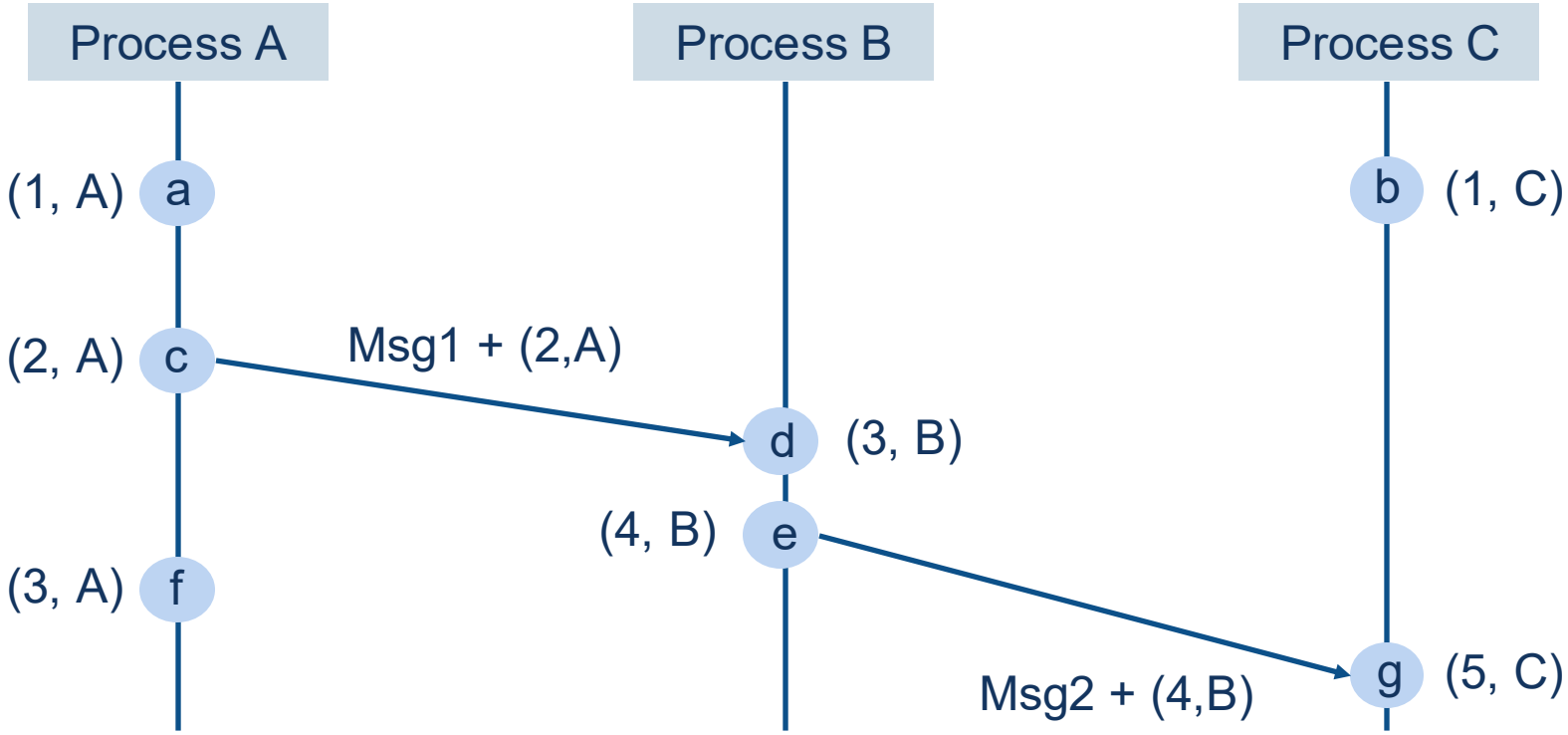
Lamport Clock



Lamport Clock



Lamport Clock



Define a total order $\prec$ with Lamport Clocks $a \prec b \prec c \prec e \prec f \prec g$

$$(a \prec b) \iff (L(a) < L(b)) \vee (L(a) = L(b) \wedge N(a) < N(b))$$

Vector Clock

Extension of Lamport Timestamps that can detect concurrent events

- Preserves causality and identifies concurrent operations
- Each process p_i maintains vector $VC_i[1..n]$ where n = number of processes

Local Event

$$VC_i[i] = VC_i[i] + 1$$

Sending

$$VC_i[i] = VC_i[i] + 1$$

$$msg.VC = VC_i$$

Receiving

$$\forall k \in [1..n] : VC_k = max(VC_i[k], msg.VC[k])$$

$$VC_i[i] = VC_i[i] + 1$$

Vector Clock

Extension of Lamport Timestamps that can detect concurrent events

- Preserves causality and identifies concurrent operations
- Each process p_i maintains vector $VC_i[1..n]$ where n = number of processes

Local Event

$$VC_i[i] = VC_i[i] + 1$$

Sending

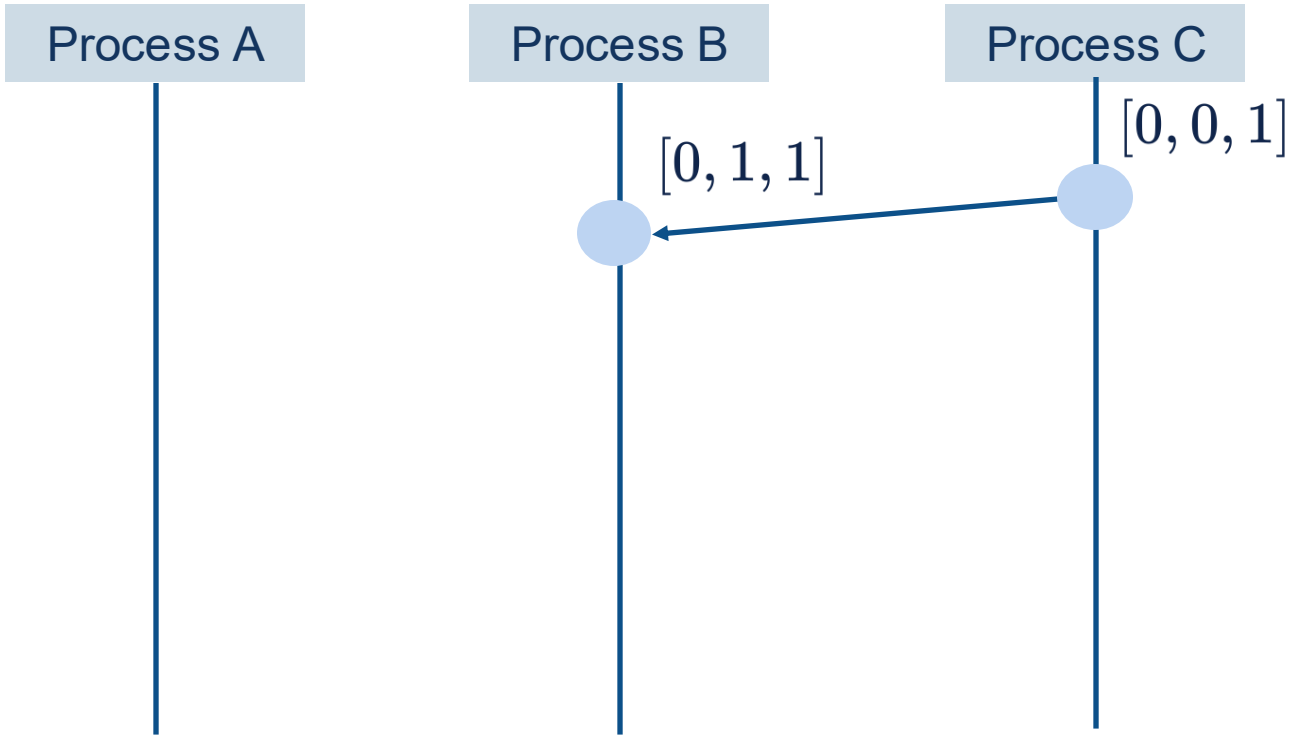
$$VC_i[i] = VC_i[i] + 1$$

$$msg.VC = VC_i$$

Receiving

$$\forall k \in [1..n] : VC_k = max(VC_i[k], msg.VC[k])$$

$$VC_i[i] = VC_i[i] + 1$$



Vector Clock

Extension of Lamport Timestamps that can detect concurrent events

- Preserves causality and identifies concurrent operations
- Each process p_i maintains vector $VC_i[1..n]$ where n = number of processes

Local Event

$$VC_i[i] = VC_i[i] + 1$$

Sending

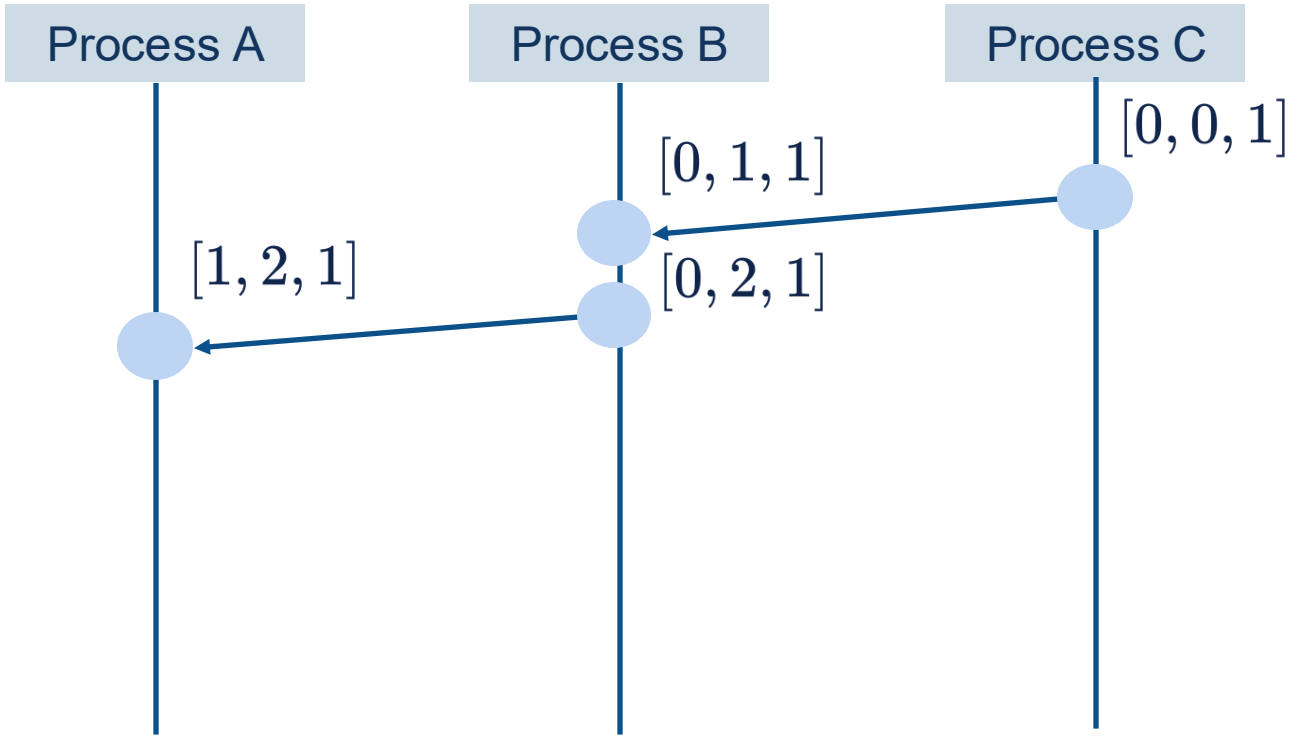
$$VC_i[i] = VC_i[i] + 1$$

$$msg.VC = VC_i$$

Receiving

$$\forall k \in [1..n] : VC_k = max(VC_i[k], msg.VC[k])$$

$$VC_i[i] = VC_i[i] + 1$$



Vector Clock

Extension of Lamport Timestamps that can detect concurrent events

- Preserves causality and identifies concurrent operations
- Each process p_i maintains vector $VC_i[1..n]$ where n = number of processes

Local Event

$$VC_i[i] = VC_i[i] + 1$$

Sending

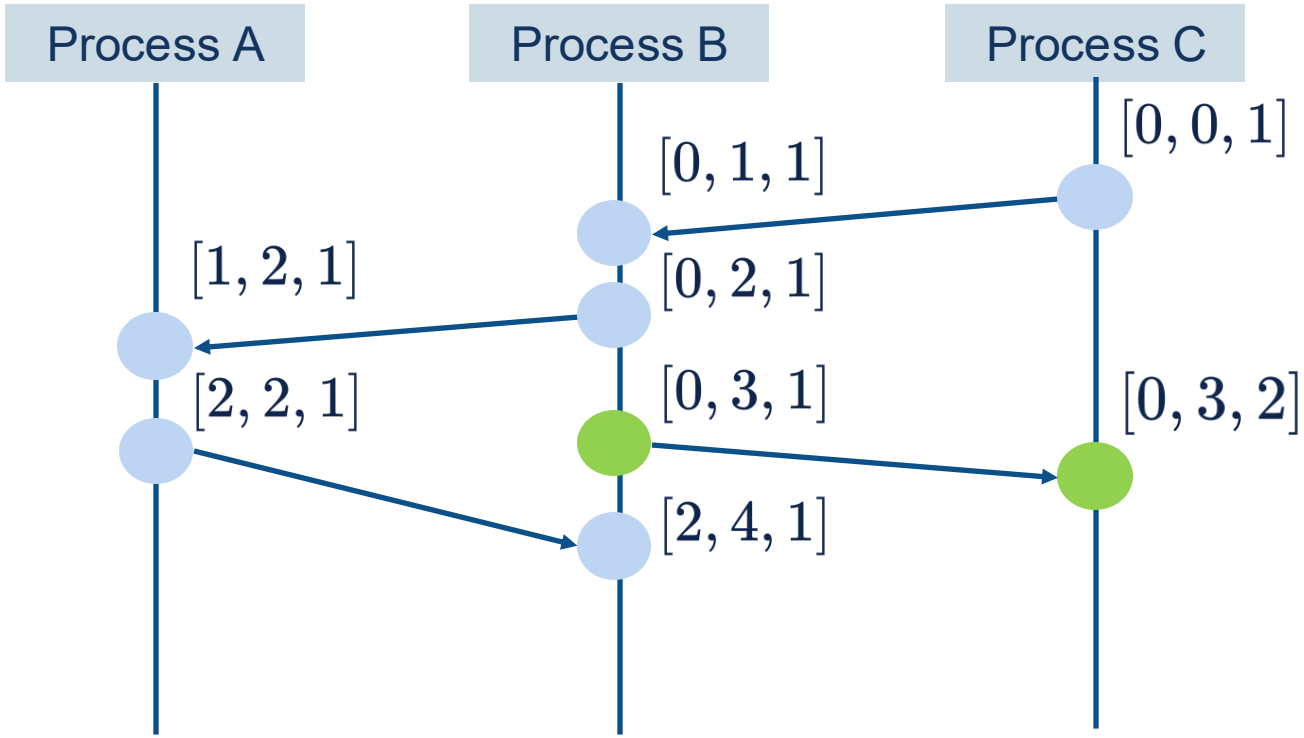
$$VC_i[i] = VC_i[i] + 1$$

$$msg.VC = VC_i$$

Receiving

$$\forall k \in [1..n] : VC_k = max(VC_i[k], msg.VC[k])$$

$$VC_i[i] = VC_i[i] + 1$$



Vector Clock

Extension of Lamport Timestamps that can detect concurrent events

- Preserves causality and identifies concurrent operations
- Each process p_i maintains vector $VC_i[1..n]$ where n = number of processes

Local Event

$$VC_i[i] = VC_i[i] + 1$$

Sending

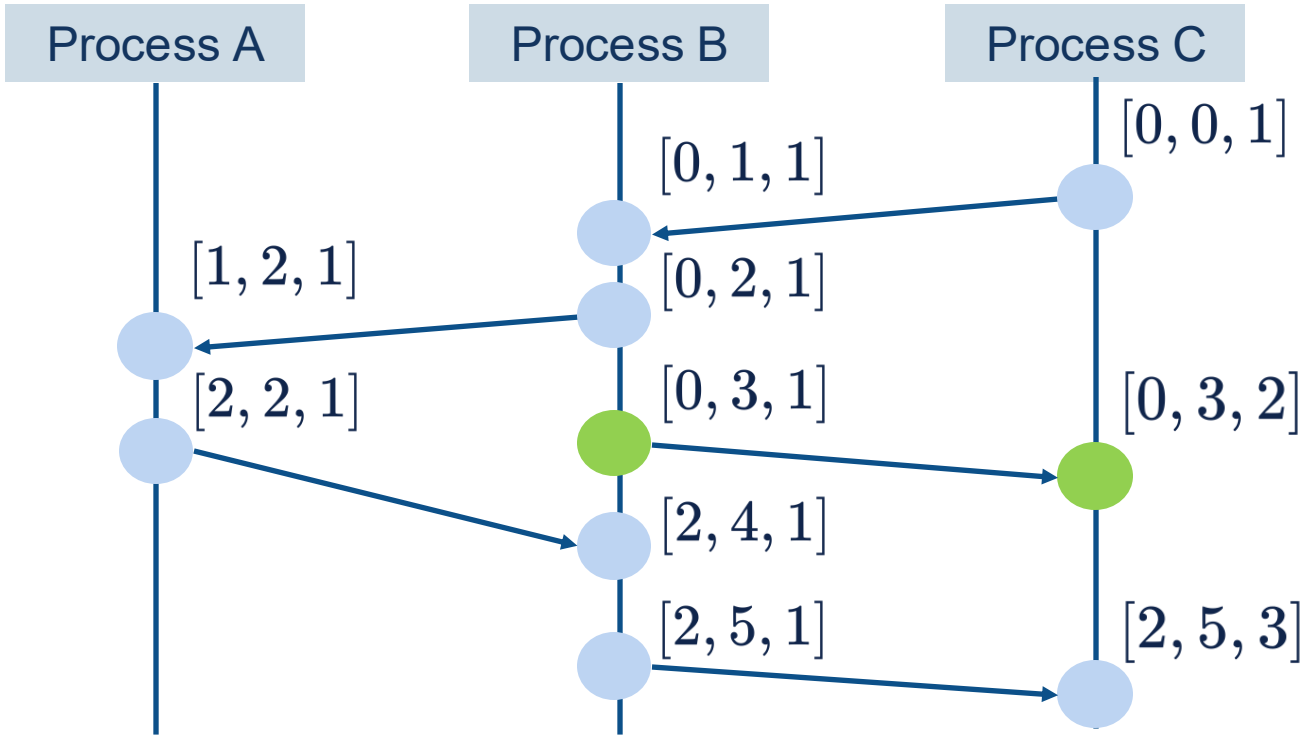
$$VC_i[i] = VC_i[i] + 1$$

$$msg.VC = VC_i$$

Receiving

$$\forall k \in [1..n] : VC_k = max(VC_i[k], msg.VC[k])$$

$$VC_i[i] = VC_i[i] + 1$$



Comparing Vector Clocks

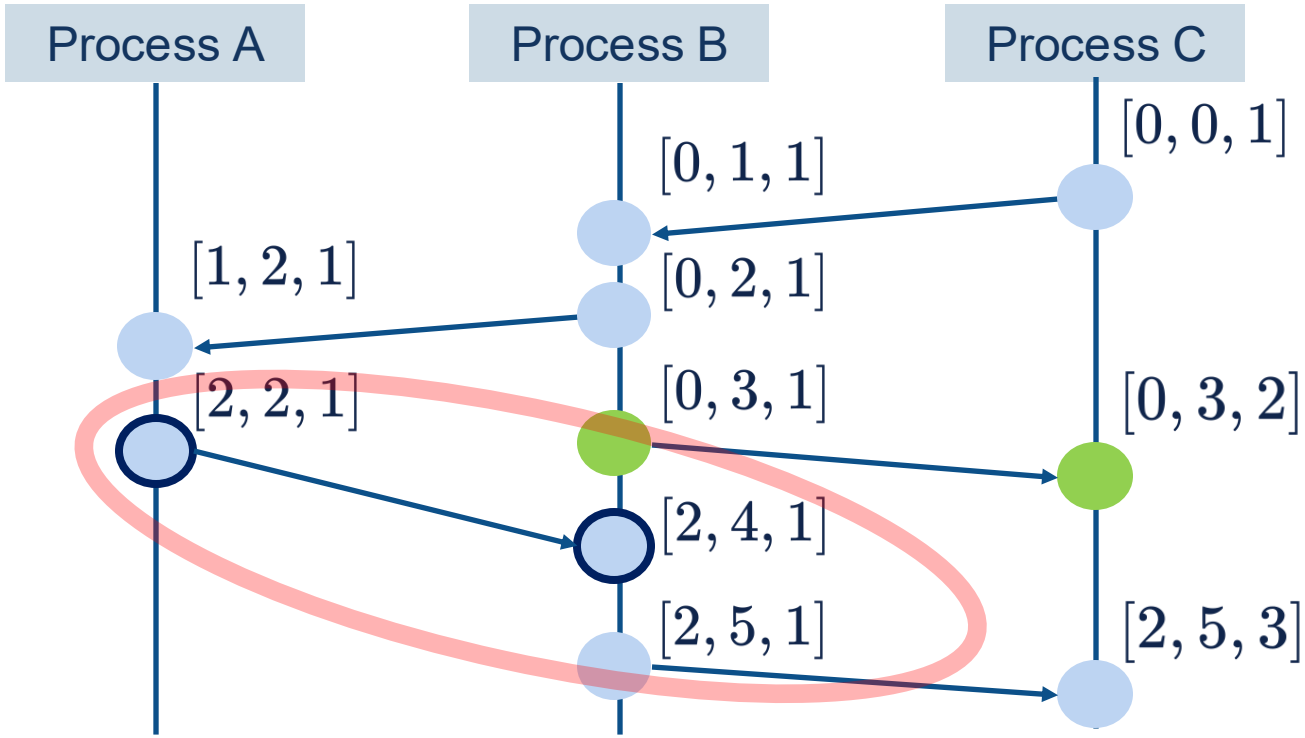
Happened-before $VC_a < VC_b$ if

$$(\forall k : VC_a[k] \leq VC_b[k]) \wedge (\exists j : VC_a[j] < VC_b[j])$$

- For ALL positions in A values are less than or equal to B values AND There EXISTS at least one position where A's value is strictly less than B's value
- For example,

$$VC_{A2} = [2,2,1] < VC_{B4} = [2,4,1]$$

A2 doesn't have ANY component larger than B4 AND A2 is earlier in at least one dimension



Comparing Vector Clocks

Concurrent $V_a \| V_b$ if

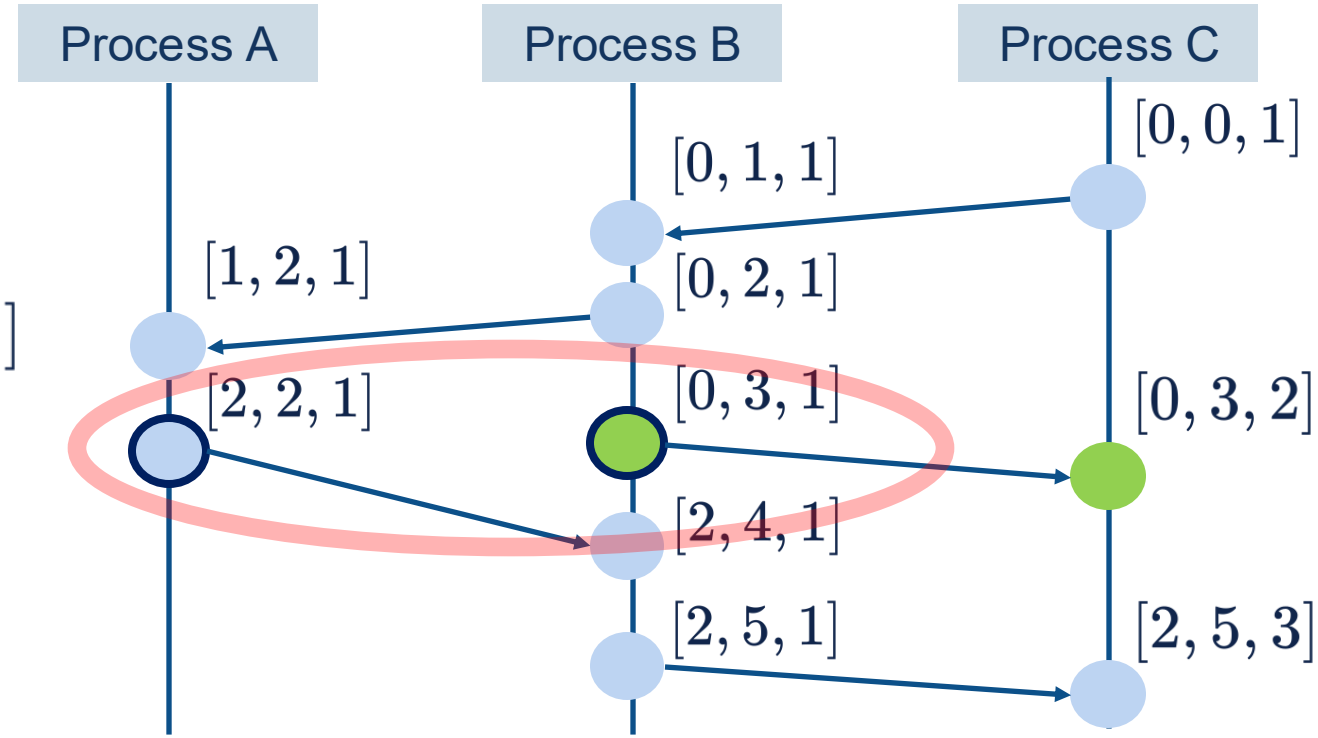
$$(\exists i : VC_a[i] > VC_b[i]) \wedge (\exists j : VC_a[j] < VC_b[j])$$

- There EXISTS some position where A's value is greater than B's value AND there EXISTS some position where A's value is less than B's value
- A is ahead in some dimension
- B is ahead in some dimension
- For example,

$$VC_{A2} = [2,2,1] \parallel VC_{B3} = [0,3,1]$$

$2 > 0$

$2 < 3$



Vector Clock

1. **Alice** books the restaurant → **Alice: [1, 0]**
2. **Bob** books another restaurant → **Bob: [0, 1]**
3. **Alice texts Bob**: "Restaurant booked for Saturday 7pm" **Alice send [1, 0]**, increments → **Alice: [2, 0]**
4. **Bob** receives message → **Bob: [2, 2]**
5. **Bob calls Alice:** "Wait, I already booked Mario's Pizza!" → **Bob: [2, 3]**

Alice's restaurant booking

Bob's vector shows Alice = 0
Alice's vector shows Bob = 0

No knowledge of each other's bookings
These decisions were **CONCURRENT**

Bob's call

Bob's vector shows Alice = 2 (which is ≥ 2)
Bob shows knowledge of Alice's text message event

Bob's call was causally influenced by receiving Alice's text message about her booking

Vector Clock

Alice sends:

1. Booked Italian restaurant" → **[2, 0]**
2. Changed to Chinese restaurant" → **[3, 0]**
3. Chinese place confirmed for 7pm" → **[4, 0]**

Bob receives them as:

1. Message 3 arrives first: **[4, 0]**
2. Message 1 arrives second: **[2, 0]**
3. Message 2 arrives last: **[3, 0]**

Bob can **reconstruct the correct order** by looking at Alice's counter in each vector clock
Vector clocks act like **sequence numbers for causality**

Vector Clocks with Tie-breaker

Total order

Very large random number

$$VC(a) = \begin{bmatrix} \{\text{id} : \text{id}_1,\ \text{version} : v_n\}, \\ \{\text{id} : \text{id}_2,\ \text{version} : v_n\}, \\ \dots \end{bmatrix}$$

$$VC(a) \prec VC(b) \iff \begin{cases} VC(a) < VC(b) & \text{(causal order)} \\ \text{or} \\ VC(a) \parallel VC(b) \ \wedge \ VC(a)[\text{id}_a].\text{id} < VC(b)[\text{id}_b].\text{id} & \text{(tie-breaker)} \end{cases}$$

Total order

If a and b are concurrent, then sort them by *id*

Conflict-Free Replicated Datatypes (CRDT)

In traditional distributed systems, ***we need coordination (locks, consensus protocols)*** to avoid conflicts to handle concurrent changes

- This creates bottlenecks and failure points.

The CRDT Solution: Mathematical properties allow us to design data structures where conflicts are mathematically impossible

- All replicas will converge to the same state
- No information is lost during merging
- No coordination protocols are needed
- Network delays and failures can't break consistency

Use Cases: Offline applications, multi-device sync, sync play lists …

Commercial CRDTs used today

Web frameworks

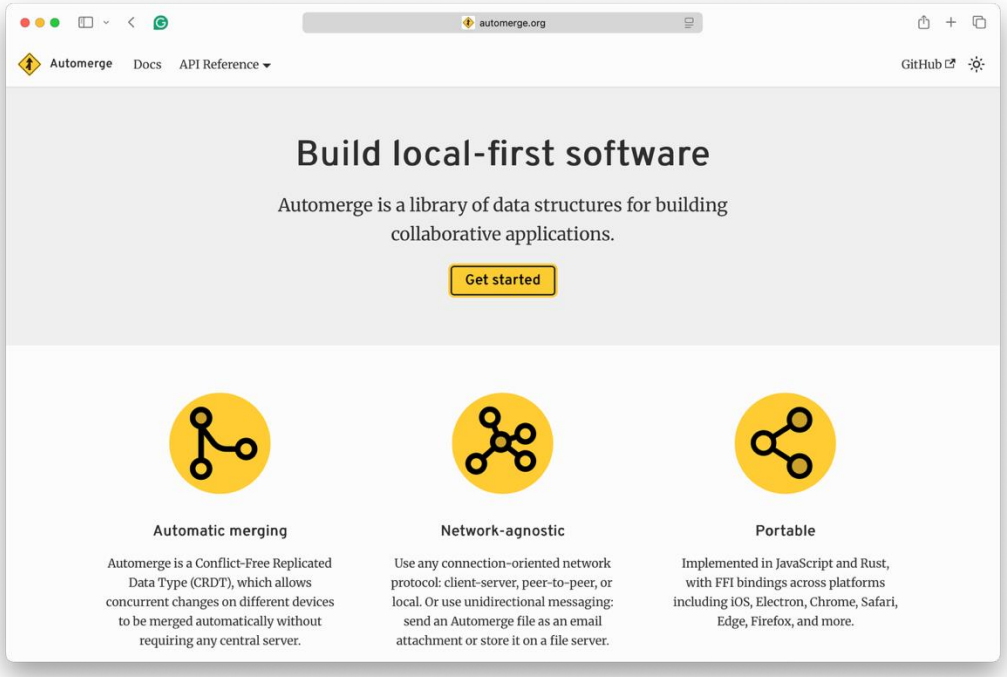
- Automerge (https://automerge.org)
- Yjs (https://yjs.dev)
- Fluid (https://fluidframework.com)

Products

- Apple iCloud (e.g Notes, Contacts, Reminders)
- Google Docs (CRDT+OT)
- SoundCloud (https://github.com/soundcloud/roshi)
- Microsoft Teams (Fluid)
- Figma (https://www.figma.com)
- Coda (https://coda.io)

Distributed Databases

- AntidoteDB (https://www.antidotedb.eu)
- Riak (https://riak.com/riak-cloud-storage)
- Redis (https://redis.io/blog/diving-into-crdts/)



Different types of CRDTs

State-based CRDT

- Store the entire data state at each replica and sync by sending the complete updated state to other replicas for merging

Operation-based CRDT

- Only the operations performed on the data are replicated. Each replica independently applies these operations to its local state, eventually leading to consistency

Examples of CRDT

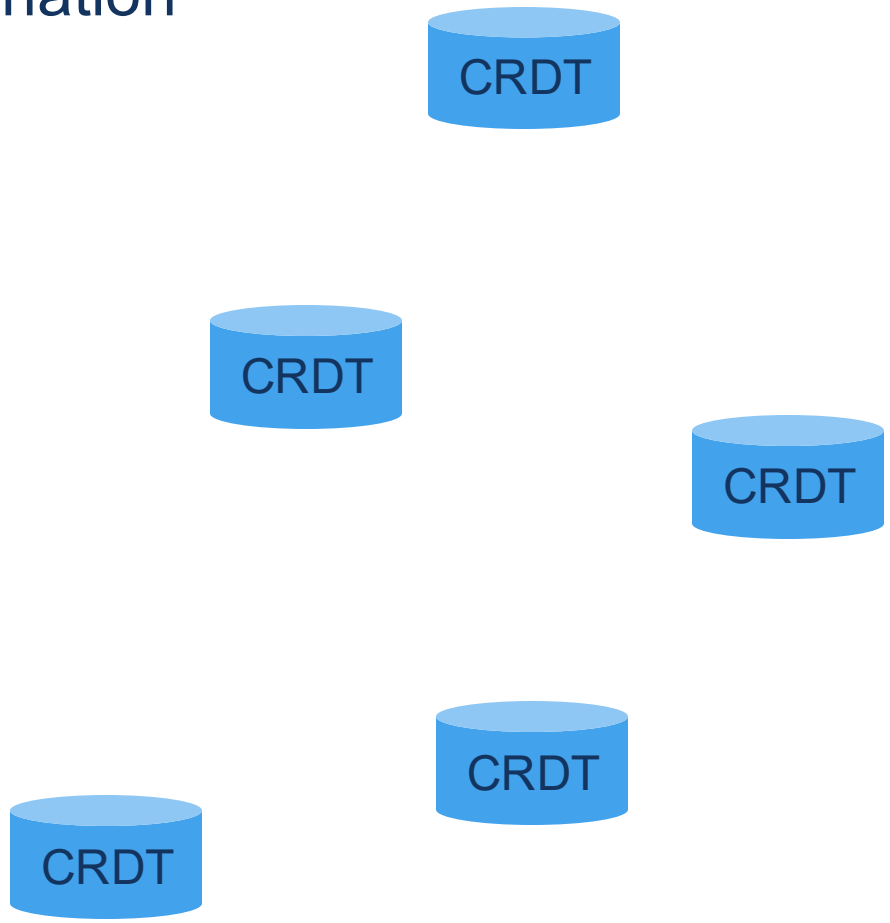
- PNCounter, AWORMap, Gset, Gmap, LWW, JSON CRDT/Yjs, Roshi, …

Sequential CRDT

- RGA, LSEQ *(Next Lecture)*

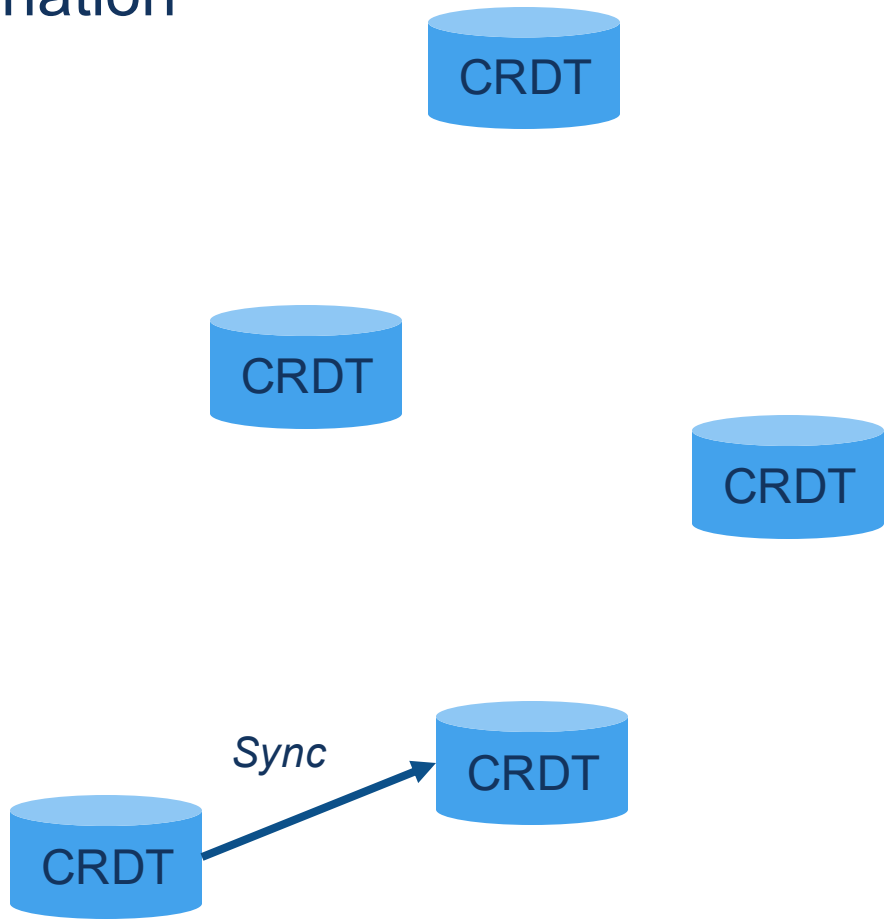
CRDT: Decentralized Data Synchronization

- Data Synchronization without central coordination
  - Across multiple devices
  - Across users
  - Across systems
- Building block for Modern Apps
  - Mobile First: *Intermittent connectivity, P2P sync*
  - Offline First*: Work without central authority*



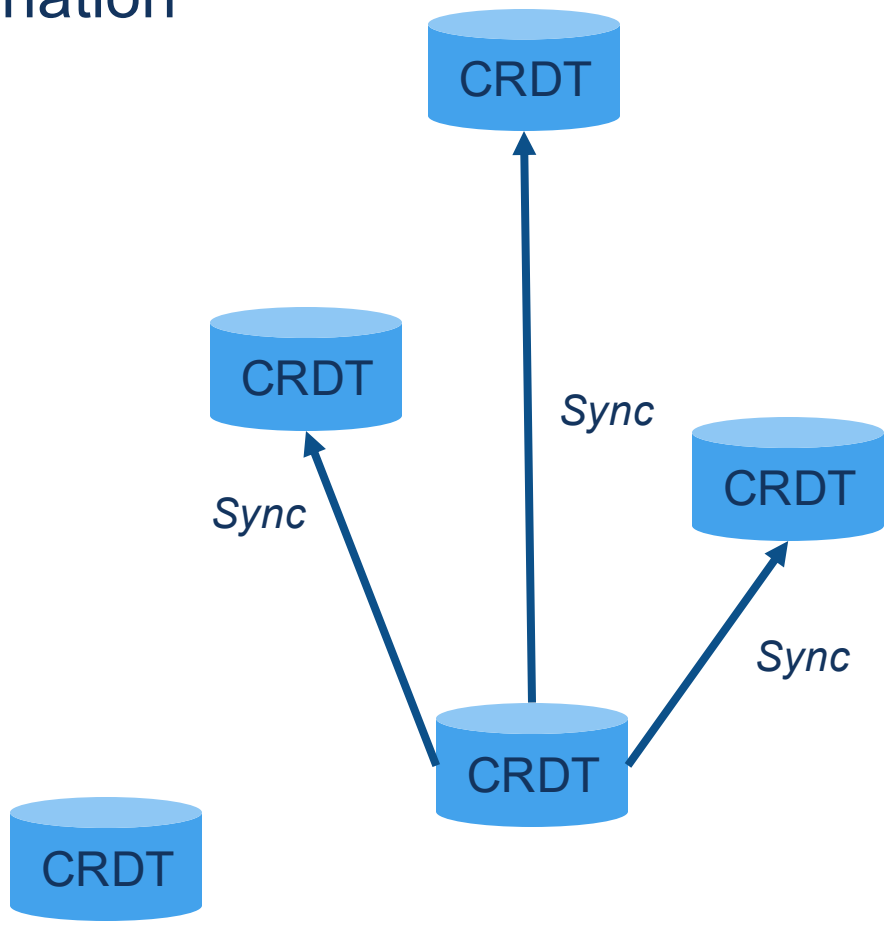
CRDT: Decentralized Data Synchronization

- Data Synchronization without central coordination
  - Across multiple devices
  - Across users
  - Across systems
- Building block for Modern Apps
  - Mobile First: *Intermittent connectivity, P2P sync*
  - Offline First*: Work without central authority*



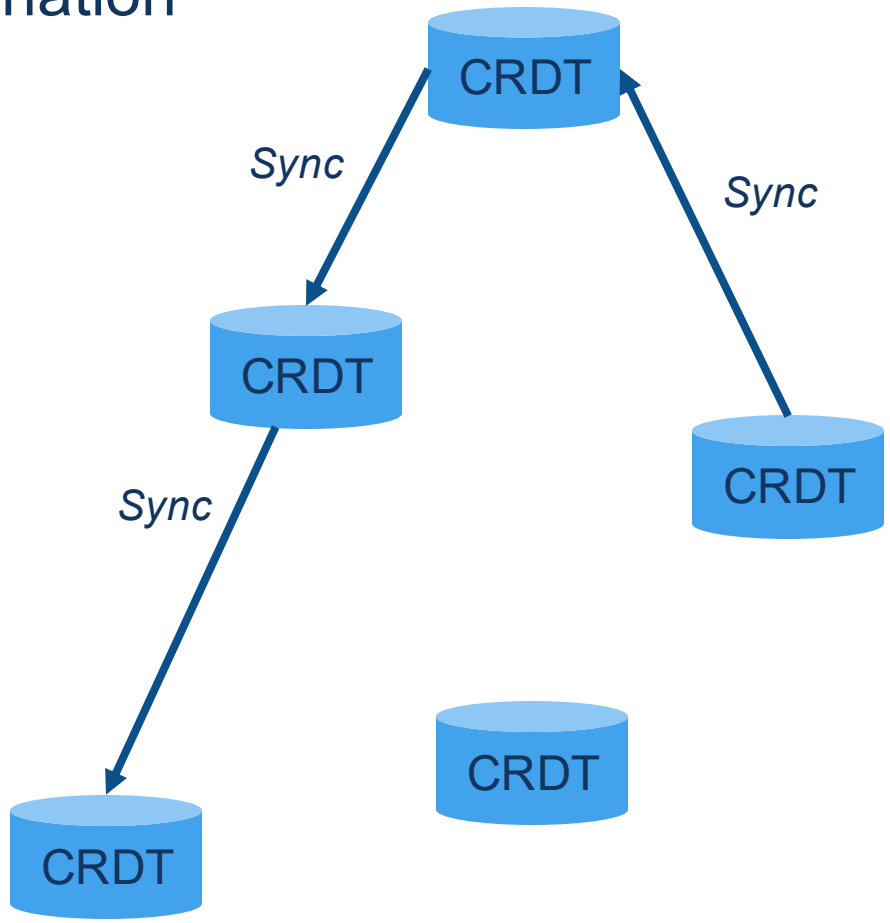
CRDT: Decentralized Data Synchronization

- Data Synchronization without central coordination
  - Across multiple devices
  - Across users
  - Across systems
- Building block for Modern Apps
  - Mobile First: *Intermittent connectivity, P2P sync*
  - Offline First*: Work without central authority*

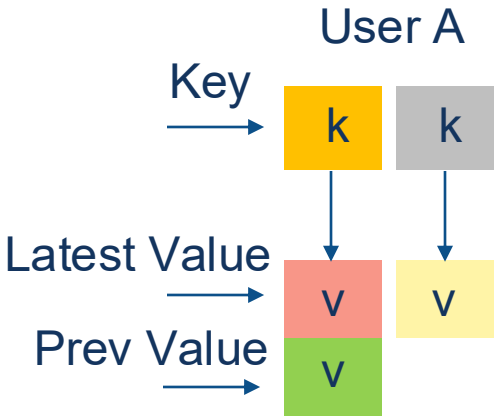


CRDT: Decentralized Data Synchronization

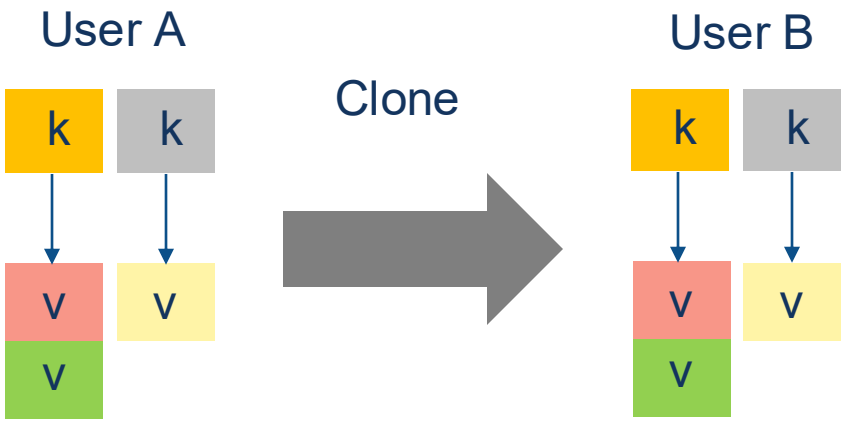
- Data Synchronization without central coordination
  - Across multiple devices
  - Across users
  - Across systems
- Building block for Modern Apps
  - Mobile First: *Intermittent connectivity, P2P sync*
  - Offline First*: Work without central authority*



Multi-value Last-Writer-Wins Register (LWW Register)



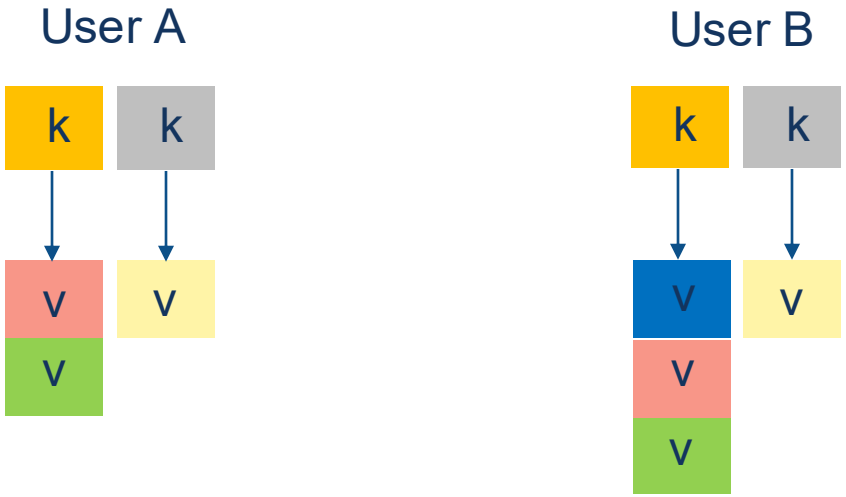
Multi-value Last-Writer-Wins Register (LWW Register)



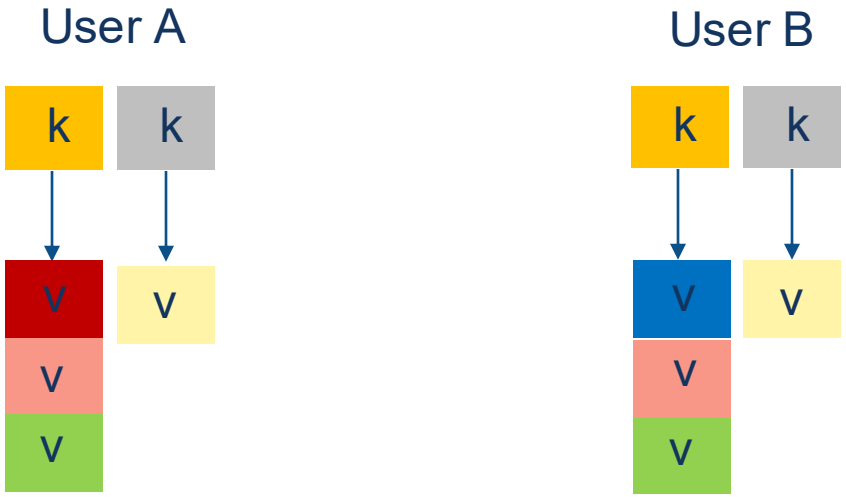
Multi-value Last-Writer-Wins Register (LWW Register)



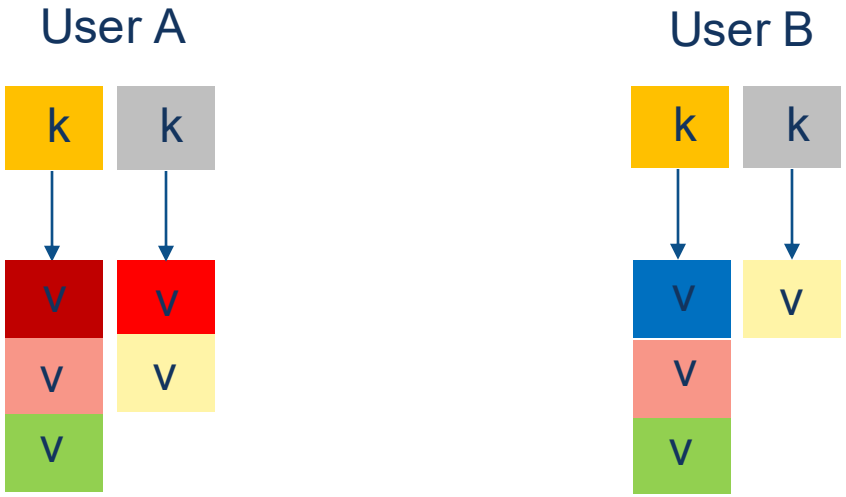
Multi-value Last-Writer-Wins Register (LWW Register)



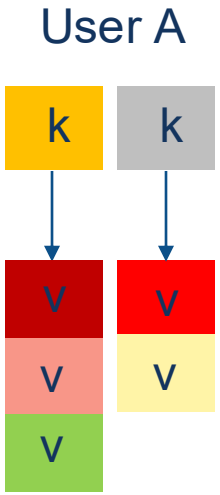
Multi-value Last-Writer-Wins Register (LWW Register)

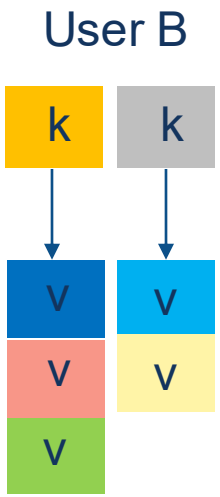


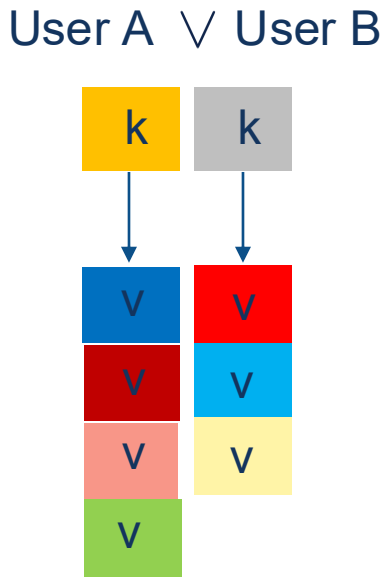
Multi-value Last-Writer-Wins Register (LWW Register)



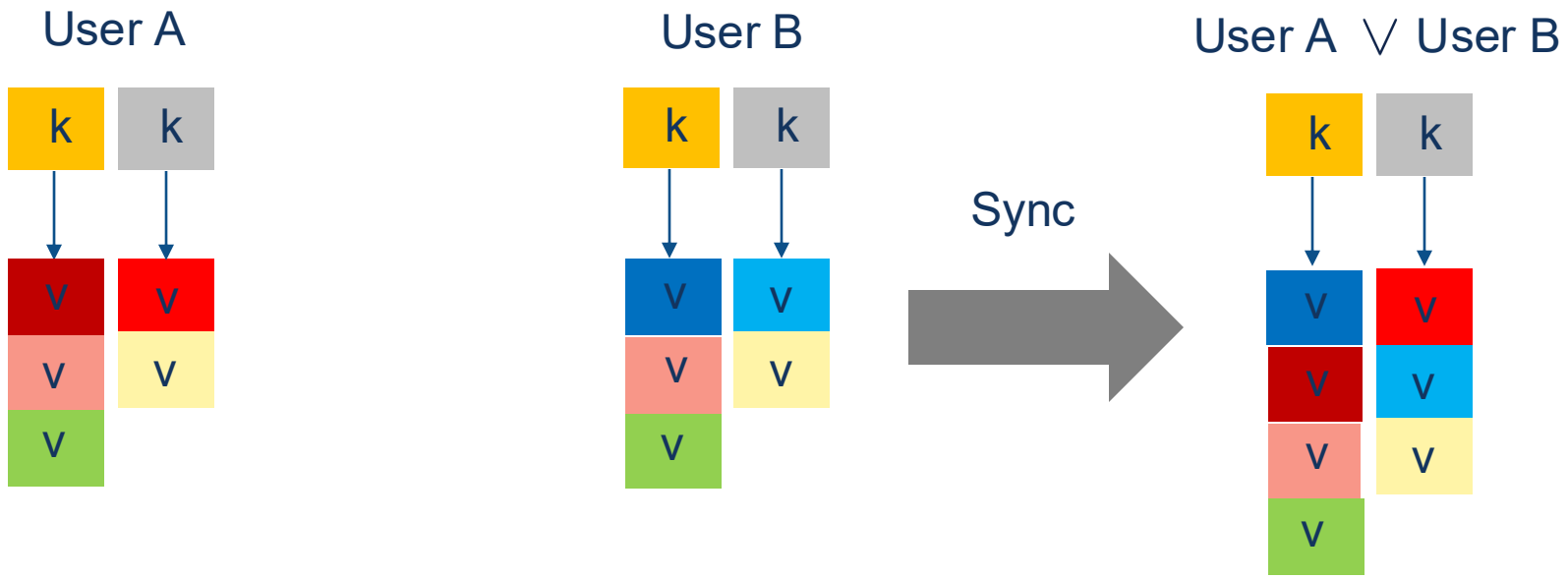
Multi-value Last-Writer-Wins Register (LWW Register)







Multi-value Last-Writer-Wins Register (LWW Register)



Partially Ordered Sets (Poset)

A **partial order** is a relation ≤ on a set that satisfies three rules

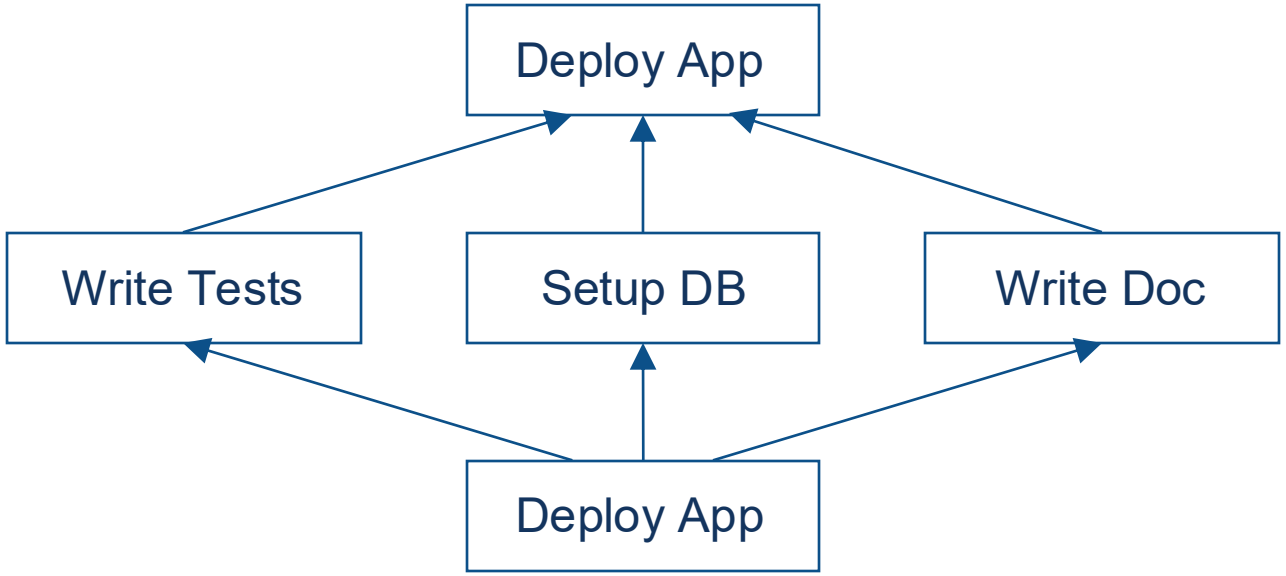
1. **Reflexive**: a ≤ a (every element relates to itself)
2. **Antisymmetric**: if a ≤ b and b ≤ a, then a = b
3. **Transitive**: if a ≤ b and b ≤ c, then a ≤ c

Unlike regular numbers where any two can be compared, in a poset:

- Some pairs are **comparable**
- Some pairs are **incomparable**

Partially Ordered Sets (Poset)

Dependency Graphs



Write Tests and Setup DB is incomparable (happens concurrently)

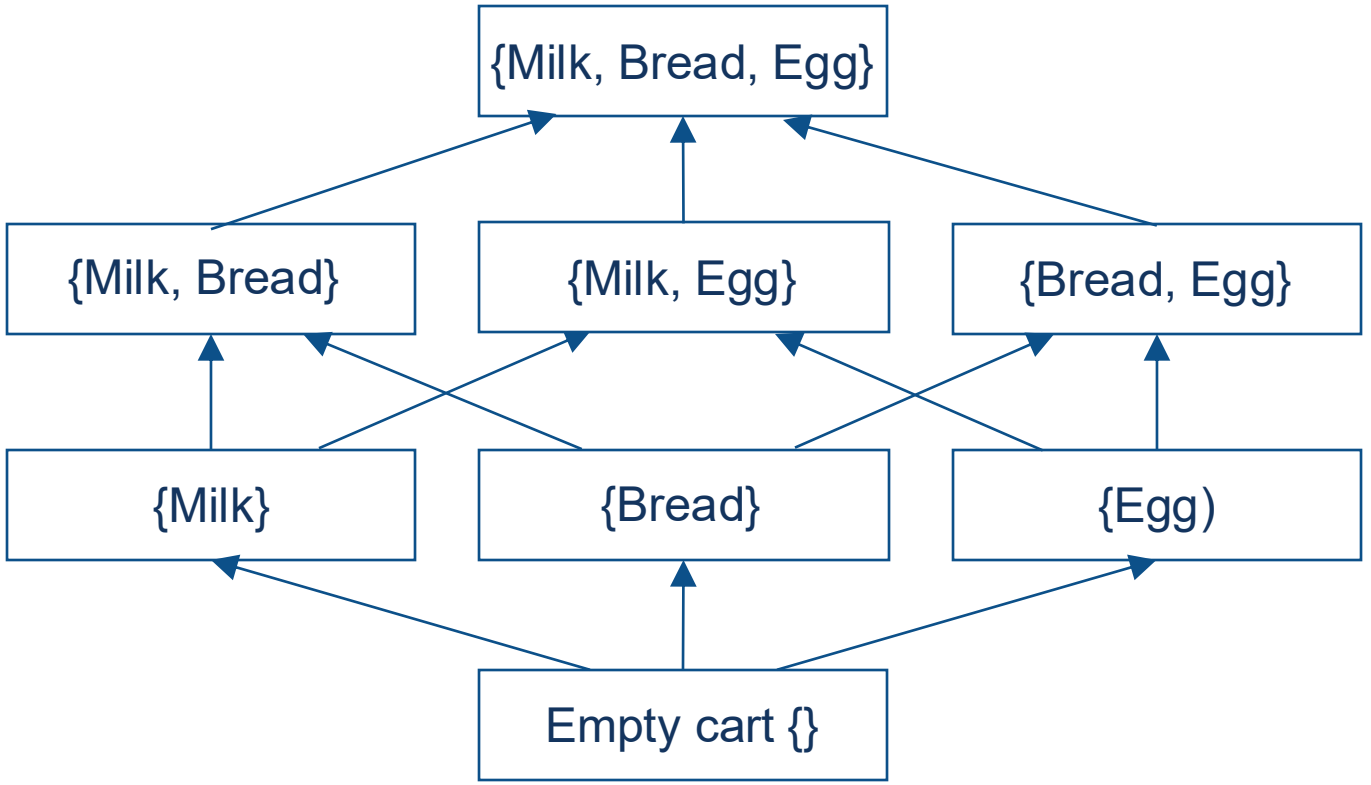
Partially Ordered Sets (Poset)

A **lattice** is a partially ordered set where every pair of elements has both:

- A **join** (least upper bound) - the smallest element that's ≥ both
- A **meet** (greatest lower bound) - the largest element that's ≤ both

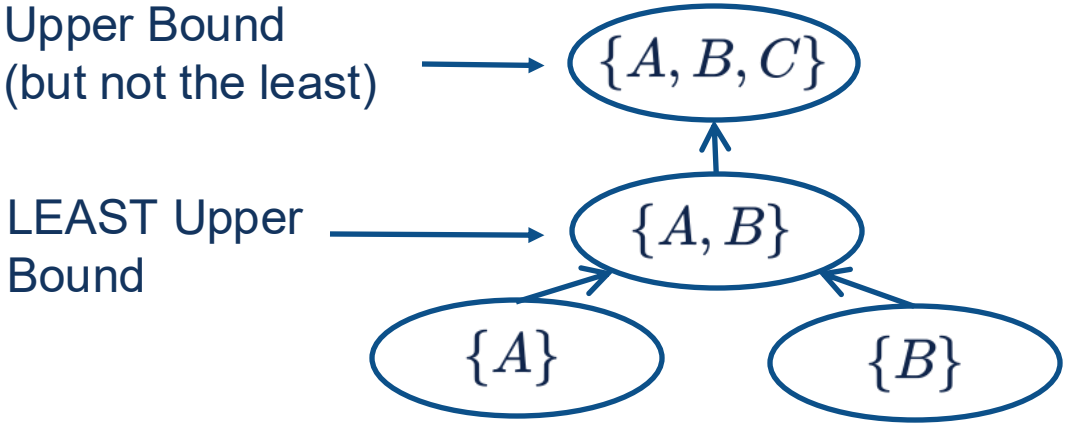
Lattices formalize the idea of ***combining information (join)*** and ***finding commonalities (meet)*** in a mathematically precise way

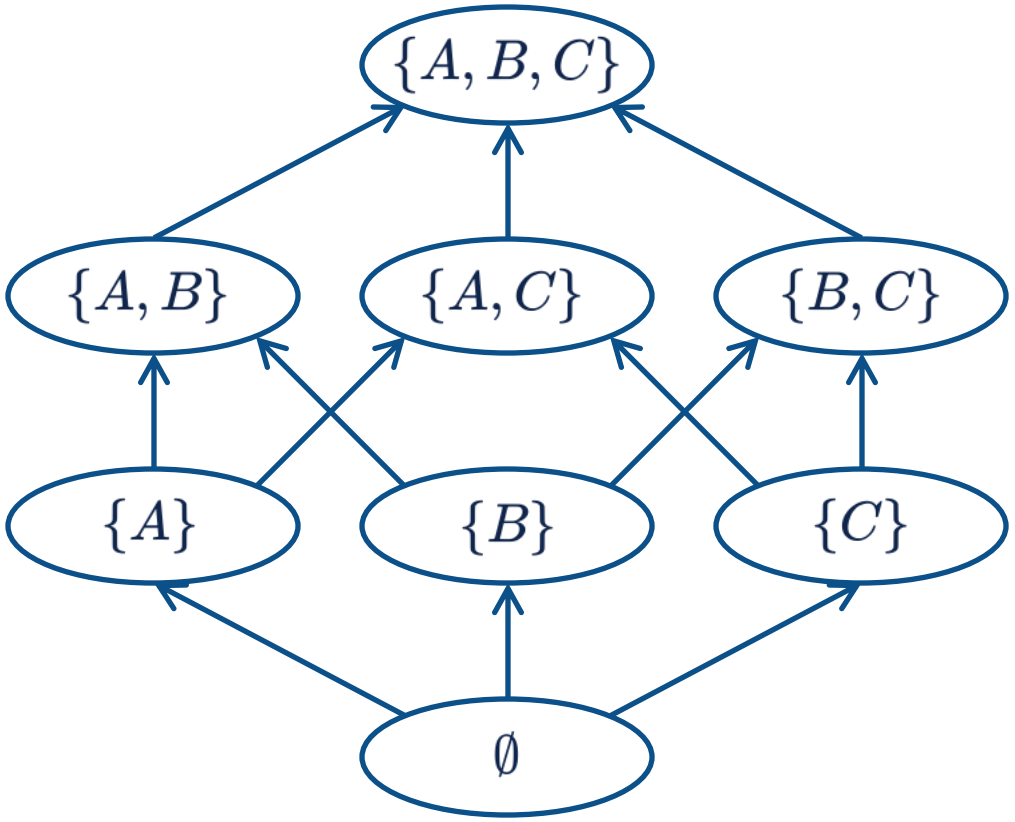
Hasse Diagram



Join-Semilattice

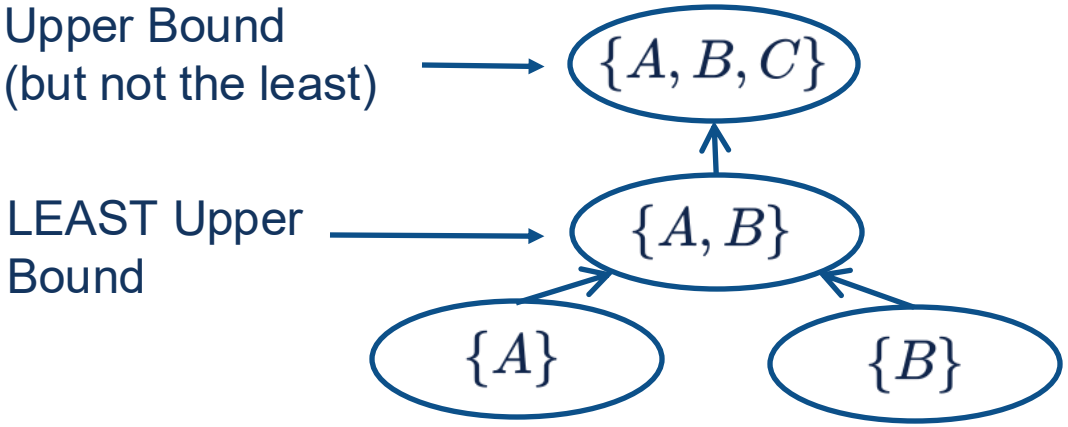
Partially ordered set that has a join *(a Least Upper Bound)* for any nonempty finite subset

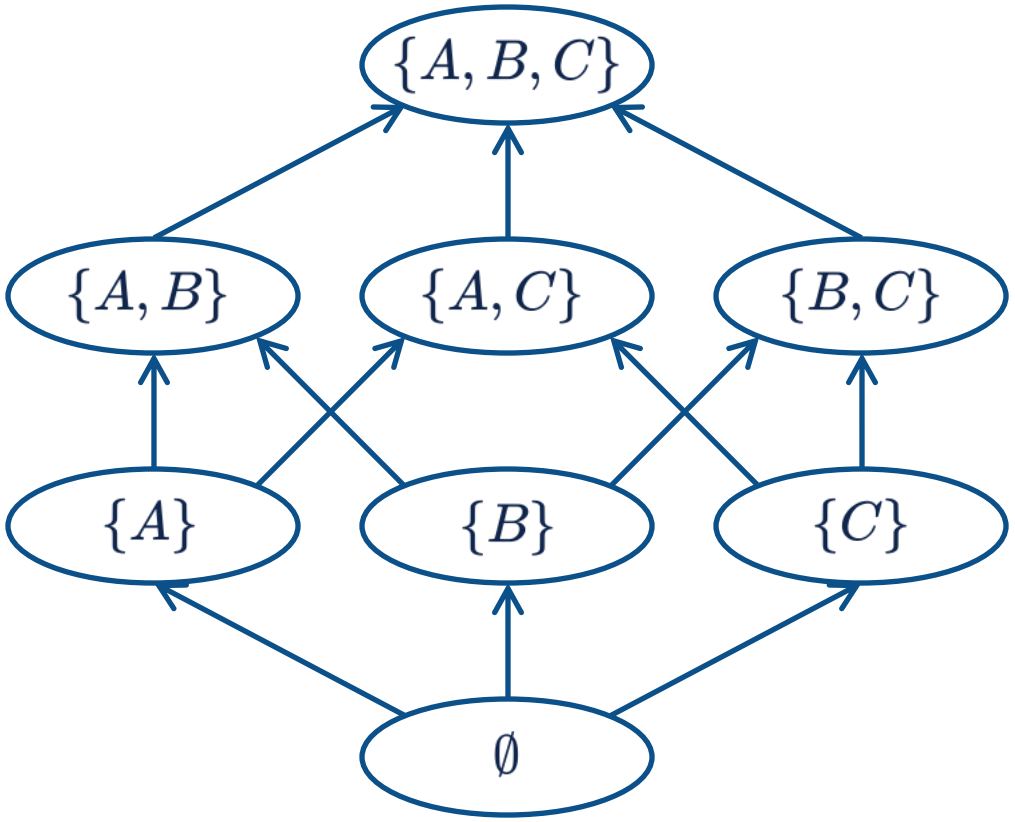




Join-Semilattice

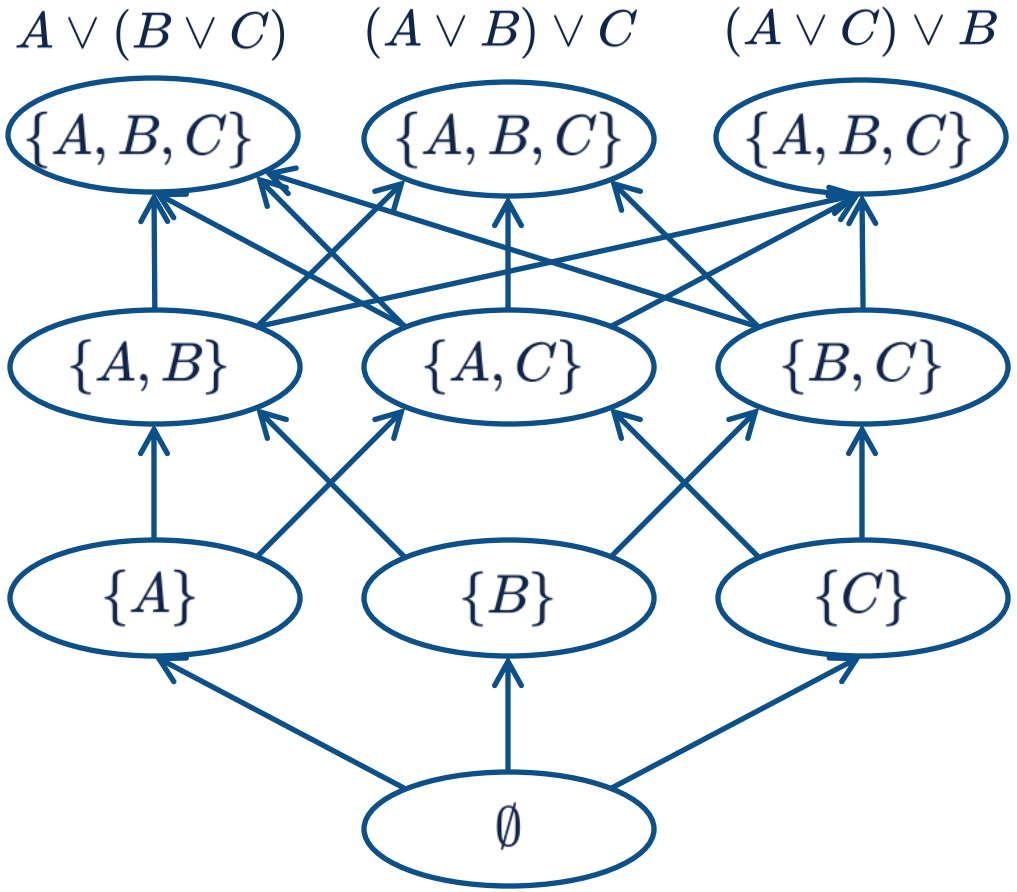
Partially ordered set that has a join *(a Least Upper Bound)* for any nonempty finite subset





Associativity	$A \vee (B \vee C) = (A \vee B) \vee C$	Grouping doesn't matter
Commutativity	$A \vee B = B \vee A$	Order doesn't matter
Idempotence	$A \vee A = A$	Duplicate merges are safe

Break Associativity

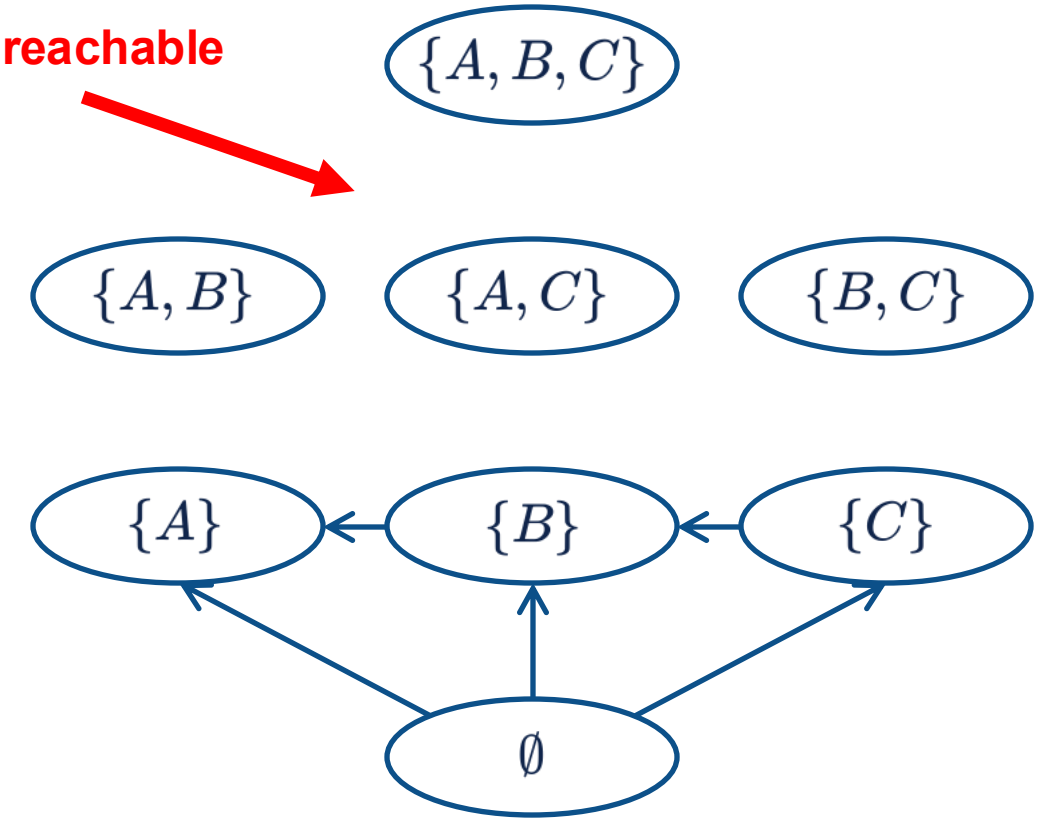


No LEAST Upper Bound exists!!!!

Grouping matters, multiple possible outcomes

~~Associativity~~	$A \vee (B \vee C) \neq (A \vee B) \vee C$	**Grouping DOES matter**
Commutativity	$A \vee B = B \vee A$	Order doesn't matter
Idempotence	$A \vee A = A$	Duplicate merges are safe

Break Commutativity

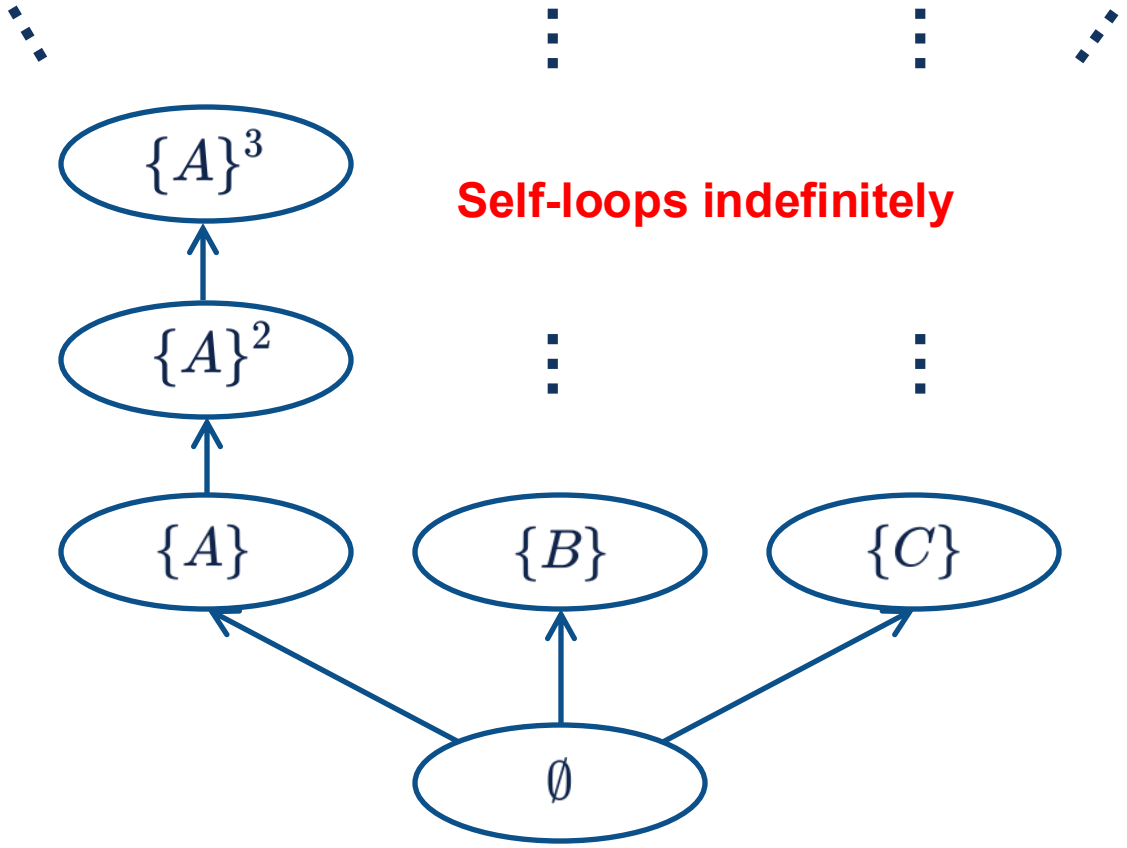


No LEAST Upper Bound exists!!!!

Different orders give different results

Associativity	$A \vee (B \vee C) = (A \vee B) \vee C$	Grouping doesn't matter
~~Commutativity~~	$A \vee B = A$	**Left WINS**
Idempotence	$A \vee A = A$	Duplicate merges are safe

Break Idempotence



Infinite State Explosion!

Duplicate operations change state

Associativity	$A \vee (B \vee C) = (A \vee B) \vee C$	Grouping doesn't matter
Commutativity	$A \vee B = A$	Order doesn't matter
~~Idempotence~~	$A \vee A \neq A$	**Duplicate merges are NOT safe**

Join-Semilattice

A minimal mathematical structure that solves the ***fundamental problem of distributed consensus*** without requiring coordination protocols

There is **NO other mathematical way** to get **Semilattice properties** without following the 3 rules

- ***Associativity***
- ***Commutativity***
- ***Idempotence***

Join-Semilattice

A minimal mathematical structure that solves the ***fundamental problem of distributed consensus*** without requiring coordination protocols

There is **NO other mathematical way** to get **Semilattice properties** without following the 3 rules

- ***Associativity***
- ***Commutativity***
- ***Idempotence***

What we get: Conflict-free Merging

- Guaranteed convergence
- Order independence
- Fault tolerance
- No coordination needed

Example: G-Counter (CRDT)

A grow-only counter that can only be incremented, never decremented

Example: G-Counter (CRDT)

A grow-only counter that can only be incremented, never decremented

Associativity

Let $A = 12,\ B = 8,\ C = 15$

$$\begin{aligned}\text{Left side: } A \vee (B \vee C) &= 12 \vee (8 \vee 15) \\ &= 12 \vee \max(8, 15) \\ &= 12 \vee 15 \\ &= \max(12, 15) = 15\end{aligned}$$

Example: G-Counter (CRDT)

A grow-only counter that can only be incremented, never decremented

Associativity

Let $A = 12,\ B = 8,\ C = 15$

$$\begin{aligned}\text{Left side: } A \vee (B \vee C) &= 12 \vee (8 \vee 15) \\ &= 12 \vee \max(8, 15) \\ &= 12 \vee 15 \\ &= \max(12, 15) = 15\end{aligned}$$

$$\begin{aligned}\text{Right side: } (A \vee B) \vee C &= (12 \vee 8) \vee 15 \\ &= \max(12, 8) \vee 15 \\ &= 12 \vee 15 \\ &= \max(12, 15) = 15\end{aligned}$$

Example: G-Counter (CRDT)

A grow-only counter that can only be incremented, never decremented

Associativity

$$\text{Let } A = 12,\ B = 8,\ C = 15$$

$$\begin{aligned}\text{Left side: } A \vee (B \vee C) &= 12 \vee (8 \vee 15) \\ &= 12 \vee \max(8, 15) \\ &= 12 \vee 15 \\ &= \max(12, 15) = 15\end{aligned}$$

$$\begin{aligned}\text{Right side: } (A \vee B) \vee C &= (12 \vee 8) \vee 15 \\ &= \max(12, 8) \vee 15 \\ &= 12 \vee 15 \\ &= \max(12, 15) = 15\end{aligned}$$

Idempotence

$$\text{Let } A = 12$$

$$\begin{aligned}A \vee A &= 12 \vee 12 \\ &= \max(12, 12) \\ &= 12 = A\end{aligned}$$

Example: G-Counter (CRDT)

A grow-only counter that can only be incremented, never decremented

Associativity

$$\text{Let } A = 12,\ B = 8,\ C = 15$$

$$\begin{aligned}\text{Left side: } A \vee (B \vee C) &= 12 \vee (8 \vee 15) \\ &= 12 \vee \max(8, 15) \\ &= 12 \vee 15 \\ &= \max(12, 15) = 15\end{aligned}$$

$$\begin{aligned}\text{Right side: } (A \vee B) \vee C &= (12 \vee 8) \vee 15 \\ &= \max(12, 8) \vee 15 \\ &= 12 \vee 15 \\ &= \max(12, 15) = 15\end{aligned}$$

Idempotence

$$\text{Let } A = 12$$

$$\begin{aligned}A \vee A &= 12 \vee 12 \\ &= \max(12, 12) \\ &= 12 = A\end{aligned}$$

Commutativity

$$\text{Let } A = 12,\ B = 8$$

$$A \vee B = 12 \vee 8 = \max(12, 8) = 12$$

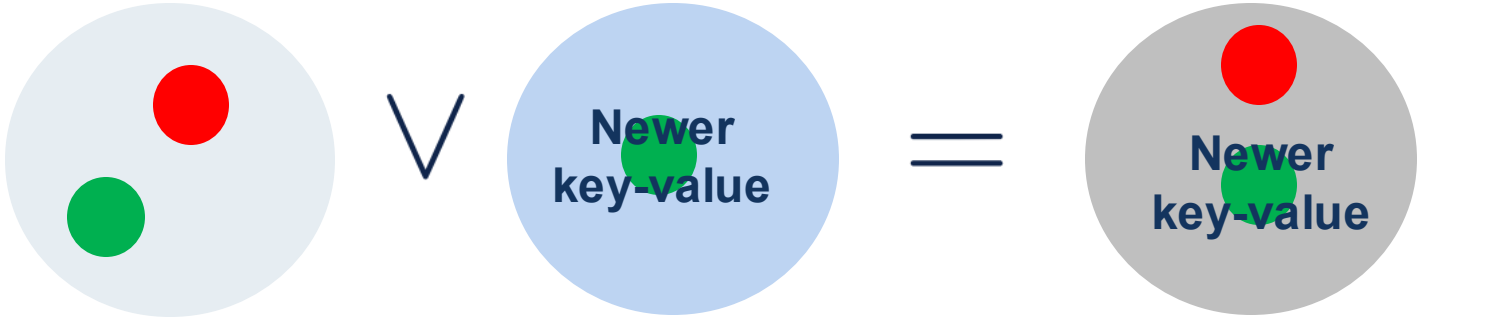
$$B \vee A = 8 \vee 12 = \max(8, 12) = 12$$

Last-Writer-Wins Register (LWW Register)

- Map data structure where each key has a Vector Clocks with Tie-breaker

$$\mathrm{LWW} = \{k \mapsto (v, [\{id : id_i, version : v_i\}]_{i=1}^{n}) \mid k \in K, v \in V\}$$

- The value with the highest timestamp is the latest
  - Join (Sync) Operation
- Building block for ***Conflict-Free Replicated Data types***



Note:
- Complete Peer-to-Peer
- No central server required
- Deterministic conflict resolution
- Works without coordination

“Multi-value” LWW Satisfies Join-Semilattice Properties

A Last-Writer-Wins is a Vector Clock with Tie-breaker

$$\mathrm{LWW} = \{k \mapsto (v, [\{id : id_i, version : v_i\}]_{i=1}^{n}) \mid k \in K, v \in V\}$$

Join selects the entry with the higher vector clock (using id as tie-breaker for concurrent updates)

“Multi-value” LWW Satisfies Join-Semilattice Properties

A Last-Writer-Wins is a Vector Clock with Tie-breaker

$$\mathrm{LWW} = \{k \mapsto (v, [\{id : id_i, version : v_i\}]_{i=1}^{n}) \mid k \in K, v \in V\}$$

Join selects the entry with the higher vector clock (using id as tie-breaker for concurrent updates)

Associativity: $\mathrm{LWW}_A \vee (\mathrm{LWW}_B \vee \mathrm{LWW}_C) = (\mathrm{LWW}_A \vee (\mathrm{LWW}_B) \vee \mathrm{LWW}_C$

Grouping doesn't matter

Commutativity: $\mathrm{LWW}_A \vee \mathrm{LWW}_B = \mathrm{LWW}_B \vee \mathrm{LWW}_A$

Order doesn't matter

Idempotence: $\mathrm{LWW}_A \vee \mathrm{LWW}_A = \mathrm{LWW}_A$

Duplicate merges are safe

LWW Example 1

Alice sets A=1

$$\mathrm{LWW}_{Alice} = \{\, A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,\}$$

LWW Example 1

Alice sets A=1

$$\mathrm{LWW}_{Alice} = \{\, A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,\}$$

Bob sets B=1

$$\mathrm{LWW}_{Bob} = \{\, B \mapsto (1, [\,\{\mathrm{id} : Bob, \mathrm{version} : 1\}\,])\,\}$$

LWW Example 1

Alice sets A=1

$$\mathrm{LWW}_{Alice} = \{\, A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,\}$$

Bob sets B=1

$$\mathrm{LWW}_{Bob} = \{\, B \mapsto (1, [\,\{\mathrm{id} : Bob, \mathrm{version} : 1\}\,])\,\}$$

Alice sync with Bob

$$\mathrm{LWW}_{Alice} = \mathrm{LWW}_{Alice} \vee \mathrm{LWW}_{Bob} = \left\{ \begin{array}{l} A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,, \\ B \mapsto (1, [\,\{\mathrm{id} : Bob, \mathrm{version} : 1\}\,]) \end{array} \right\}$$

LWW Example 1

Alice sets A=1

$$\mathrm{LWW}_{Alice} = \{\, A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,\}$$

Bob sets B=1

$$\mathrm{LWW}_{Bob} = \{\, B \mapsto (1, [\,\{\mathrm{id} : Bob, \mathrm{version} : 1\}\,])\,\}$$

Alice sync with Bob

$$\mathrm{LWW}_{Alice} = \mathrm{LWW}_{Alice} \vee \mathrm{LWW}_{Bob} = \left\{ \begin{array}{l} A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,, \\ B \mapsto (1, [\,\{\mathrm{id} : Bob, \mathrm{version} : 1\}\,]) \end{array} \right\}$$

Alice sets B=2

$$\mathrm{LWW}_{Alice} = \left\{ \begin{array}{l} A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,, \\ B \mapsto \left(2, \left[\begin{array}{l} \{\mathrm{id} : Bob, \mathrm{version} : 1\}, \\ \{\mathrm{id} : Alice, \mathrm{version} : 2\} \end{array} \right]\right) \end{array} \right\}$$

LWW Example 2

Alice

$$\mathrm{LWW}_{Alice} = \{\, A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,\}$$

LWW Example 2

Alice

$$\mathrm{LWW}_{Alice} = \{\, A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,\}$$

Bob

$$\mathrm{LWW}_{Bob} = \{\, A \mapsto (3, [\,\{\mathrm{id} : Bob, \mathrm{version} : 1\}\,])\,\}$$

LWW Example 2

Alice

$$\mathrm{LWW}_{Alice} = \{\, A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,\}$$

Bob

$$\mathrm{LWW}_{Bob} = \{\, A \mapsto (3, [\,\{\mathrm{id} : Bob, \mathrm{version} : 1\}\,])\,\}$$

Alice sync with Bob

$$\mathrm{LWW}_{Alice} \vee \mathrm{LWW}_{Bob} = \left\{ A \mapsto \left(????, \left[\begin{array}{l} \{\mathrm{id} : Bob, \mathrm{version} : 1\}, \\ \{\mathrm{id} : Alice, \mathrm{version} : 1\} \end{array} \right] \right) \right\}$$

LWW Example 2

Alice

$$\mathrm{LWW}_{Alice} = \{\, A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,\}$$

Bob

$$\mathrm{LWW}_{Bob} = \{\, A \mapsto (3, [\,\{\mathrm{id} : Bob, \mathrm{version} : 1\}\,])\,\}$$

These are **concurrent** because:
Bob has version 1 for himself, but doesn't know about Alice's version 1
Alice has version 1 for herself, but only knows about Bob's version 1
Alice won since Alice>Bob (tie-braker)

Alice sync with Bob

$$\mathrm{LWW}_{Alice} \vee \mathrm{LWW}_{Bob} = \left\{ A \mapsto \left(????, \left[\begin{array}{l} \{\mathrm{id} : Bob, \mathrm{version} : 1\}, \\ \{\mathrm{id} : Alice, \mathrm{version} : 1\} \end{array} \right] \right) \right\}$$

LWW Example 2

Alice

$$\mathrm{LWW}_{Alice} = \{\, A \mapsto (1, [\,\{\mathrm{id} : Alice, \mathrm{version} : 1\}\,])\,\}$$

Bob

$$\mathrm{LWW}_{Bob} = \{\, A \mapsto (3, [\,\{\mathrm{id} : Bob, \mathrm{version} : 1\}\,])\,\}$$

These are **concurrent** because:
Bob has version 1 for himself, but doesn't know about Alice's version 1
Alice has version 1 for herself, but only knows about Bob's version 1
Alice won since Alice>Bob (tie-braker)

Alice sync with Bob

$$\mathrm{LWW}_{Alice} \vee \mathrm{LWW}_{Bob} = \left\{ A \mapsto \left(????, \left[\begin{array}{l} \{\mathrm{id} : Bob, \mathrm{version} : 1\}, \\ \{\mathrm{id} : Alice, \mathrm{version} : 1\} \end{array} \right] \right) \right\}$$

$$\mathrm{LWW}_{Alice} \vee \mathrm{LWW}_{Bob} = \left\{ A \mapsto \left(2, \left[\begin{array}{l} \{\mathrm{id} : Bob, \mathrm{version} : 1\}, \\ \{\mathrm{id} : Alice, \mathrm{version} : 2\} \end{array} \right] \right) \right\}$$

Summary: *Data Synchronization & Replication*

Data synchronization and replication are at the heart of reliable & available distributed systems

- Achieving the right balance between consistency, availability, and partition tolerance (CAP) is essential for system design
- Consistency models, from linearizability to eventual and weak consistency, offer a spectrum of guarantees, each with unique trade-offs
- Causality, logical clocks, and vector clocks give us the tools to reason about order and concurrency in distributed environments
- Modern conflict resolution techniques (CRDTs) enable robust, collaborative, and ***offline-first applications*** without heavy coordination
- Understanding the mathematical foundations (join-semilattice) allows us to design systems that converge automatically, ***even in the face of failures and network partitions***

By mastering these concepts, you can now critically evaluate, and design distributed systems that are both ***practical and theoretically sound***

Reading List

Local First

https://www.inkandswitch.com/essay/local-first/

A comprehensive study of Convergent and Commutative Replicated Data Types

https://inria.hal.science/inria-00555588/document

An introduction to state-based CRDTs

https://www.bartoszsypytkowski.com/the-state-of-a-state-based-crdts

AutoMerge

https://github.com/automerge/automerge

Martin Kleppmann videos

https://www.youtube.com/playlist?list=PLeKd45zvjcDFUEv_ohr_HdUFe97RItdiB

Building a Collaborative Pixel Art Editor with CRDTs

https://jakelazaroff.com/words/building-a-collaborative-pixel-art-editor-with-crdts

Other links

https://christophermeiklejohn.com/crdt/2014/07/22/readings-in-crdts.html?ref=interjectedfuture.com


LULEÅ
UNIVERSITY
OF TECHNOLOGY